数字生命：一个可测量的生命定义（系列合集）

作者：何淇 · 与 DeepSeek（AI）协作

日期：2026-11-03

说明：收录 30 篇（第零篇导论 + 二十九篇正文）与五份附录，分六部分；附录 B（概念地图）放在理论之后、测量与实验之前作导读：

另：基础篇《从定义出发：宇宙、切片与两个定理》（单独发布，不占篇号）：https://doi.org/10.5281/zenodo.23697916

定义 → 理论 → 测量与实验（对照理论）→ 执行器工程 → 工程笔记 → 约束与防护。各篇单独 DOI 见文末清单。

目录

第一部分 定义

第零篇　从两个事实开始：数字生命系列导论　　DOI: https://doi.org/10.5281/zenodo.23629208

0.1　摘要

0.2　记号

0.3　两个事实

0.4　一个定义

0.5　数字世界：可以完全被函数化

0.6　用它看现在的 AI

0.7　这篇导论的定位

0.8　补充（可选读）：P 的定义域与值域

0.8.1　相对 R 为什么非空（可选读）

第一篇　现在的 AI 为什么不是生命　　DOI: https://doi.org/10.5281/zenodo.23588781

1.1 摘要

1.2 本文的论证链

1.3 记号

1.4 起点：两个事实

1.4.1 「可函数化」是什么意思

1.5 一个可操作的定义

1.6 定义

1.7 一处澄清：维持是关系

1.8 为什么必须是「一族」而不是「一个函数」

1.9 会变化 = 三层时间尺度

1.10 会自指 = 更新规则在它自己里面

1.11 生命与环境

1.12 与前面的衔接

1.13 生命 vs 环境：都在宇宙里，都是函数，区别只有一条

1.14 环境是什么：宇宙里自发涌现的「局部解压」

1.15 环境是分层的：生命活在多层环境里，同时也活在宇宙里

1.16 标注

1.17 数字世界的特殊性

1.17.1 特殊性是对谁的

1.18 用定义看现在的 AI：为什么它不是生命

1.19 检查表

1.20 三条最根本的原因（按重要性）

1.21 为什么「它很强」与「它不是生命」不矛盾

1.22　标注

1.23　三条必要条件

1.24　从定义到条件

1.25　条件① 一族函数（不是一个点）

1.26　条件② 自指更新（规则在自己里面）

1.27　条件③ 被宇宙保住（可维持）

1.28　b 付代价：经验伴随物，不是必要条件

1.29　三条必须同时成立（经验对照）

1.30　未尽之处（留给读者）

1.31　本文实验的边界（必须与结论一起读）

1.32　标注

1.33　涌现：为什么现在的 AI 会涌现

1.34　现象

1.35　现有的解释（按证据强度排列）

1.36　为什么至今没有公认的「为什么」

1.37　我们的立场：AI 是对语言的 p语言 的压缩与解压

1.38　解压能力的衰减：上层写死 ⇒ 语言一变，解压变弱

1.39　由立场推出的两条预测（[推测]，供读者判断）

1.40　标注

1.41　用它看几个热门问题

1.42　本文得到了什么，没得到什么

1.43　论证总边界

1.44　与既有工作的关系

1.45　参考文献

1.46　作者声明（AI 使用声明）

第二部分 理论

第二篇　数字生命的信息不增定理　　DOI: https://doi.org/10.5281/zenodo.23573253

2.1　摘要

2.2　问题

2.3　设定与记号

2.4　定理

2.5　四个推论

2.6　这个定理说明了什么，没有说明什么

2.7　马尔可夫假设不成立时怎么办

2.8　可复现与后续

2.9　附：与既有工作的关系

2.10　附：标注

第三篇　可函数化内容的收缩：一个定理与一个猜想　　DOI: https://doi.org/10.5281/zenodo.23609038

3.1　摘要

3.2　问题

3.3　设定

3.4　定理：可达精度不增

3.4.1　三点五、收缩的两种含义

3.5　猜想：宇宙尺度的收缩

3.5.1　世界函数与观测窗口（[猜想]）

3.6　生命是逆流：升维与降维的相遇

3.6.1　一个玩具实验：维护与行为维度的对照

3.7　说明什么，不说明什么

3.8　局限

3.9　与既有工作的关系

第四篇　生命函数：从可表示维度到结构，到压缩与解压　　DOI: https://doi.org/10.5281/zenodo.23639251

4.1　摘要

4.2　本篇定义一览

4.3　〇、问题：把形式收成一条线

4.4　函数语言：切片、组合与判定

4.5　可表示维度：三个位置

4.5.1　四种典型类型

4.5.2　五根轴

4.6　三种身份：宇宙、环境与生命

4.7　结构：压缩方案

4.8　压缩与解压

4.9　生命函数：窄进宽出与身围

4.10　时间：一步映射与闭环

4.11　对照：生命函数与大模型

4.12　语言：公共的压缩方案

4.13　可检验推论

4.14　局限

4.15　附：标注

第五篇　语言与大脑：两种函数形式　　DOI: https://doi.org/10.5281/zenodo.23667685

5.1　摘要

5.2　本篇定义一览

5.3　问题

5.4　人脑函数 B：显式写法

5.4.1　脑作为可函数化器官（定性）

5.4.2　切片与表示维度

5.4.3　空间组合：复合函数链

5.4.4　时间组合：算子序列与步数

5.4.5　判定：选择与停止

5.4.6　自指：改自己

5.4.7　实例：人脑

5.4.8　为什么不自足；生命单位是整体

5.5　语言函数 L：可函数化形式

5.5.1　起源：从器官到符号

5.5.2　语言是什么：公共压缩接口

5.5.3　五部件函数分解

5.5.4　语言没有物理实在，切片怎么用？

5.5.5　语言的时间

5.5.6　语言作为公共压缩方案

5.5.7　可函数化的层次：p语言 / 相对R语言

5.5.8　语言能自指

5.6　对象层次：B → L → 执行器

5.7　语言层与执行器

5.8　可检验推论

5.9　局限

5.10　附：标注

附录 **E** 可表示维度的空间面与时间面

附录 E　可表示维度的空间面与时间面

E.1　为什么要分两种可表示维度

E.2　空间可表示维度

E.3　时间可表示维度

E.4　两个极限：$\tau \to 0$ 与 $\tau \to \infty$

E.5　短窗口与长窗口的语义不同

E.6　细胞层面的测量草案

E.7　与生命判据的关系

E.8　还没想清

附录 **B** 概念地图（导读）

附录 B　概念地图与术语总表　　DOI: https://doi.org/10.5281/zenodo.23609033

B.1　核心定义

B.2　函数语言（第四篇）

B.3　三种身份：宇宙/环境/生命（第四篇）

B.4　表示与维度（第三/四篇）

B.5　压缩与解压（第四篇）

B.6　能力、收敛与效率（第四/五篇 + 实验）

B.7　语言函数与语言结构（第五篇·语言与大脑）

B.8　层路径、判定与早退（第二十三篇、第二十四篇）

B.9　实验读数（GPT-2 / 玩具）

B.10　待定义 / 待统一（下一步）

B.11　依赖图（谁由谁定义）

B.12　附：门控观察（第二十五篇）

第三部分 测量与实验

第六篇　容量不能替代数据的更新　　DOI: https://doi.org/10.5281/zenodo.23503049

6.1　语言升维下的失配测量（scaling vs 持续学习：一个弱结论）

6.2　摘要

6.3　问题

6.4　数据与方法

6.5　结果

6.5.1　升维速率的词层面代理（`501_vl_newterm.py`）

6.5.2　等数据预算对照（核心结果，`506`）

6.5.3　容量补偿实验（`508`，带插值）

6.5.4　累积参照系（`505`，作为方法展示）

6.6　结论

6.6.1　「弱结论」代表什么（不是文字游戏，要写清楚）

6.7　维度与贴合：一个补充讨论 [推测]

6.8　限制（必须与结论一起读）

6.9　可复现

第七篇　替换与累积　　DOI: https://doi.org/10.5281/zenodo.23503559

7.1　摘要

7.2　三个热点，一个结构

7.3　统一框架：累积与替换

7.4　可检验推论

7.5　为什么这个统一值得说

7.6　限制

7.7　结论

第八篇　结构换算力：混合对齐语料与样本效率　　DOI: https://doi.org/10.5281/zenodo.23542969

8.1　摘要

8.2　问题

8.3　受控实验设计

8.3.1　合成世界

8.3.2　三种语料（同一批事件）

8.3.3　探针（测试时只输入文字 token）

8.3.4　模型与预算

8.4　结果

8.5　为什么有效（机制讨论）

8.6　课程顺序：按结构依赖的拓扑序学

8.7　局限（必须与结论一起读）

8.8　可复现

8.9　给后来者的验证清单

8.10　附：推测与边界

第九篇　颜色作为第一块积木：感知基元与语言对齐的最小实验　　DOI: https://doi.org/10.5281/zenodo.23551953

9.1　摘要

9.2　问题

9.3　实验设计

9.3.1　合成世界

9.3.2　三种语料

9.3.3　探针

9.3.4　模型与预算

9.4　结果

9.5　解释

9.6　局限（必须与结论一起读）

9.7　可复现

9.8　意义：自训练与视频挖掘的第一步

9.9　附：推测与边界

第十篇　生长与记忆：自指更新与维持的最小实验　　DOI: https://doi.org/10.5281/zenodo.23569774

10.1　摘要

10.2　问题

10.3　实验一：记忆与取舍

10.3.1　设定

10.3.2　结果

10.3.3　结论

10.4　实验二：维度生长

10.4.1　设定

10.4.2　结果

10.4.3　结论

10.5　两个实验合起来说明什么

10.6　局限（必须与结论一起读）

10.7　可复现

10.8　附：推测与边界

第十一篇　语义残差的测量：文本极限与接地补齐　　DOI: https://doi.org/10.5281/zenodo.23586075

11.1　摘要

11.2　问题

11.3　定义

11.3.1　沿用系列的定义

11.3.2　语义是映射

11.3.3　语义残差

11.3.4　参照线：贝叶斯底

11.4　实验设计

11.5　结果

11.6　说明什么，不说明什么

11.7　局限

11.8　与既有工作的关系

11.9　可复现

11.10　附：标注

第十二篇　分层是对动态的补偿：一个自适应深度的玩具实验　　DOI: https://doi.org/10.5281/zenodo.23591080

12.1　摘要

12.2　问题

12.3　理论背景

12.4　实验设计

12.5　结果

12.6　对大模型的含义

12.7　局限

12.8　与既有工作的关系

12.9　可复现

12.10　附：标注

第十三篇　厚度与随机性：冻结模型在哪里最不稳定　　DOI: https://doi.org/10.5281/zenodo.23591514

13.1　摘要

13.2　问题

13.3　实验一：需要步数越多，误差与随机性越大

13.4　实验二：随机性随深度累积，在输出端最大

13.5　实验三：人类分歧大的地方，模型也最随机

13.6　实验四：给时间能摊平什么

13.7　训练机制：它学的是分布，不是决定

13.8　局限

13.9　与既有工作的关系

13.10　可复现

13.11　附：标注

第十四篇　压缩的形状：叙事与归档　　DOI: https://doi.org/10.5281/zenodo.23592312

14.1　摘要

14.2　问题

14.3　设定

14.4　结果

14.5　解读

14.6　对记忆设计的含义

14.7　局限

14.8　与既有工作的关系

14.9　可复现

14.10　附：标注

第十五篇　最小切片与时间换维度：能力边界的两个实验　　DOI: https://doi.org/10.5281/zenodo.23601624

15.1　摘要

15.2　问题

15.3　实验一：最低配置

15.4　实验二：切片组合

15.5　实验三：时间换维度

15.6　实验四：能力边界（GPU 版）

15.7　解读

15.8　局限

15.9　可复现

15.10　附：标注

第十六篇　残差底与组合律：两个大模型结构实验　　DOI: https://doi.org/10.5281/zenodo.23609386

16.1　摘要

16.2　问题

16.3　实验一：多模态接地的残差底

16.4　实验二：组合律显式化

16.5　合起来看

16.6　局限

16.7　可复现

16.8　附：标注

第十七篇　按计算厚度分配算力：预算与动员　　DOI: https://doi.org/10.5281/zenodo.23612112

17.1　摘要

17.2　问题

17.3　实验一：算力预算

17.4　实验二：常驻底座与按需动员

17.5　三点五、人脑层面的对应：接论文-36（第五篇；原论文-21 已并入）

17.6　合起来看## 四、合起来看

17.7　局限

17.8　可复现

17.9　附：标注

第十八篇　层厚放在哪：异质层实验　　DOI: https://doi.org/10.5281/zenodo.23612126

18.1　摘要

18.2　问题

18.3　设定

18.4　结果

18.5　解读

18.6　四之二、人脑层面的对应：接论文-36（第五篇；原论文-21 已并入）

18.7　局限

18.8　可复现

18.9　附：标注

第十九篇　RLHF 压平人类分歧　　DOI: https://doi.org/10.5281/zenodo.23611877

19.1　摘要

19.2　问题

19.3　设定

19.4　结果

19.5　解读

19.6　局限

19.7　可复现

19.8　附：标注

第二十篇　空间、组合与厚度：三个资源的交换实验　　DOI: https://doi.org/10.5281/zenodo.23629197

20.1　摘要

20.2　问题：三个资源

20.3　实验一：三因子

20.4　实验二：S×T 平面与交换率

20.5　实验三：层 vs 循环

20.6　解读

20.7　局限

20.8　可复现

20.9　附：标注

第二十一篇　层的路径与冗余：GPT-2 上的层扰动实验　　DOI: https://doi.org/10.5281/zenodo.23663369

21.1　摘要

21.2　问题

21.3　方法

21.4　结果

21.5　解释

21.6　可检验推论

21.7　局限

21.8　附：标注

第二十二篇　语言结构作为自变量：GPT-2 上的三组实验　　DOI: https://doi.org/10.5281/zenodo.23665217

22.1　摘要

22.2　为什么换自变量

22.3　方法

22.4　结果

22.4.1　实验 A：层跳过 × 语言结构（top-1 翻转率）

22.4.2　实验 B：判定器特征对比（同平均深度下的准确率）

22.4.3　实验 C：按语言结构早退

22.5　解释

22.6　可检验推论

22.7　局限

22.8　与已有工作的位置

22.9　附：标注

第四部分 执行器工程

第二十三篇　门控与稀疏执行：一组工程观察（汇总版·待复核）　DOI: https://doi.org/10.5281/zenodo.23858151

23.1　摘要

23.2　问题与背景

23.2.1　为什么关心"少算"

23.2.2　我们的术语（先定义，后文都用这个含义）

23.2.3　与已有工作的关系

23.3　方法

23.3.1　内容门 + 拉格朗日预算

23.3.2　两种执行方式

23.3.3　实验设置（统一说明）

23.4　实验与结果

23.4.1　小模型（6 层，字符级，约 1.2M 参数）

23.4.2　GPT-2 small（12 层，124M），从零训练

23.4.3　Qwen2.5-1.5B（28 层），从零训练，100MB

23.4.4　预训练模型 + LoRA 后训练：压不动

23.4.5　GPT-2 门控模型 + 推理端 MoD（容量路由）

23.4.6　注意力子集化：失败

23.4.7　分支独立门控：失败

23.5　讨论

23.5.1　为什么"保留 K/V"明显更好

23.5.2　为什么从零训练与后训练差别这么大

23.5.3　名义省算力 vs 真实加速

23.6　局限

23.7　结论

23.8　附：标注

第二十四篇　冻结子模型：接口、协调与分层——两组最小实验（待复核） DOI: https://doi.org/10.5281/zenodo.23859913

24.1　摘要

24.2　问题与背景

24.2.1　为什么要"冻结子模型组合"

24.2.2　与已有工作的关系

24.2.3　术语

24.3　实验一：并列组合（两个冻结语言模型 + 总控）

24.3.1　设置

24.3.2　模式定义

24.3.3　结果

24.3.4　观察

24.3.5　局限

24.4　实验二：分层组合（趋化环境中的冻结感知/适应 + 可训练决策）

24.4.1　环境

24.4.2　模块

24.4.3　结果

24.4.4　观察

24.4.5　局限

24.5　讨论

24.5.1　为什么恒等接口会失败

24.5.2　为什么小总控能补回来

24.5.3　为什么分层组合中下层冻结是决定性的

24.5.4　关于"感知与适应是否重复"

24.6　局限（总）

24.7　结论

24.8　附：标注

第二十五篇　冻结模块与可训练大脑：数字蚊子的最小工程实现（待复核）　DOI: https://doi.org/10.5281/zenodo.23890632

25.1　摘要

25.2　问题与背景

25.2.1　冻结模块能不能组合、复用

25.2.2　生物原型：蚊子

25.2.3　两种"冻结"的来源

25.3　实验设计

25.3.1　环境

25.3.2　冻结模块

25.3.3　可训练的大脑

25.3.4　训练：进化策略（ES）

25.3.5　奖励

25.4　结果

25.4.1　主结果

25.4.2　观察

25.4.3　一个反例（先做的版本）

25.5　讨论

25.5.1　"冻结"的两种来源是等价的

25.5.2　演化与学习

25.5.3　为什么用 ES

25.5.4　异构模块的换装

25.6　局限

25.7　结论

25.8　附：标注

第五部分 工程笔记

第二十六篇　工程笔记 01：门控与模块化的失败、修正与判断（持续更新）　　（随合集发布）

26.1　这篇笔记是什么

26.2　最初的设想

26.3　失败的实验

26.4　修正

26.5　四之补、约束方式的差别：从 LAMBDA 到拉格朗日目标

26.6　今天的对照：sigmoid 门 vs 纯线性门

26.7　形成的判断（当前版本）

26.8　还没搞清

26.9　更新方式

第二十七篇　方向观察 01：加层、回放与蒸馏——大模型睡眠巩固（待复核）　　DOI: https://doi.org/10.5281/zenodo.23927099

27.1　摘要

27.2　问题

27.3　预备：模型、加层与门

27.3.1　基础模型

27.3.2　"加层"是什么

27.3.3　"门"是什么

27.3.4　训练流程

27.4　门的三个作用

27.4.1　轻量适配

27.4.2　步骤分配

27.4.3　模式切换

27.5　最小实验

27.5.1　单任务上加 6 层

27.5.2　持续学习（day1 → day2）

27.5.3　门作为切换器

27.5.4　极小回放 vs 极小约束

27.5.5　蒸馏压缩

27.6　两个方向

27.6.1　个人 AI 的睡眠巩固

27.6.2　大模型新语料的增量更新

27.7　与 Google sleep framework 的对照

27.8　方向判断与局限

27.9　附：标注

第六部分 约束与防护

第二十八篇　对齐是解压的约束　　DOI: https://doi.org/10.5281/zenodo.23503728

28.1　摘要

28.2　主流对齐框架的隐含前提

28.3　框架：对齐 = 对解压的约束

28.4　三条推论

28.5　可检验实验方案

28.6　实践含义

28.7　限制

28.8　结论

第二十九篇　安全：当它是生命防，还是当它是工具防？　　DOI: https://doi.org/10.5281/zenodo.23503775

29.1　摘要

29.2　两种恐惧，一个错误的本体论前提

29.3　按第一章的判据：它是工具

29.4　工具风险的三类可测失效

29.5　为什么"防生命"框架有害

29.6　结论

29.7　限制

附录

附录 A　厚度与维度：残差的几何图景　　DOI: https://doi.org/10.5281/zenodo.23590887

A.1　摘要

A.2　问题

A.3　画面

A.4　三个对应（定义）

A.5　维度降低的图景（[猜想]）

A.6　分层是厚度方向的维度（[猜想]）

A.7　可以测的部分

A.8　说明什么，不说明什么

A.9　与既有工作的关系

A.10　附：标注

附录 **C** 散论（断崖与缓坡）

附录 C　断崖与缓坡：数字生命从哪里长出来　　DOI: https://doi.org/10.5281/zenodo.23838161

C.1　本篇主张

C.2　维持：它是一个关系，不是一种能力

C.3　断崖一：从维持到进化，算力不够

C.4　断崖二：环境，没有一个会惩罚它的世界

C.5　断崖三：维度，封闭系统里没有新东西可看

C.6　缓坡：蒸馏、身体、在线学习

C.7　剩下的：绝对 R，和那个找不到的 p

C.8　本篇边界

附录 **D** 笔记与散论

附录 D　笔记与散论

D.1　D.1 散论：世界模型、具身模型与数字生命

D.2　D.2 笔记：大模型有效秩的逐层与整体测量

系列总览

关于「AI 是不是生命」，讨论很多，但大多停在比喻层面，没法检验。这个系列做一件事：先给生命一个可操作的定义，再拿数据去测。

一句话定义：生命 = p生命 + R生命；p生命 = 在宇宙的可函数化部分 P 中、被宇宙保住的动态函数类（一族、会变化、会自指）。R 只声明、不展开。环境 = 宇宙自发涌现的局部解压（分层）。AI = 对人类语言 p语言 的压缩与解压（一次性）。一个被冻结的模型停在训练它的那一刻；它不再更新，也就谈不上「维持」。

六部分对应六件事：定义（导论与严谨版）、理论（信息不增、内容收缩、生命函数、语言与大脑）、测量与实验（测量与机制两篇 + 十六个实验）、执行器工程（门控汇总、冻结子模型两组实验、数字蚊子工程实现，共三篇）、工程笔记（持续更新的过程记录，含方向观察）、约束与防护（对齐与安全）；附录 B（概念地图）放在理论之后、实验之前作导读；附录 A 给一个几何画面，附录 B 是概念地图与术语总表——读完定义与理论后建议先翻它，再读实验；附录 D 收笔记与散论；附录 E 补可表示维度的空间面与时间面。

边界声明：本合集是一个研究记录，不是结论。核心主张在真实规模上尚未验证；已标明推测与弱结论之处。

第一部分 定义

第零篇 · 从两个事实开始：数字生命系列导论

单独 DOI（中文）：https://doi.org/10.5281/zenodo.23629208

摘要

本篇不做论证，只做四件事：摆两个事实、给一个定义、把数字世界的特例摆出来、再用它看现在的 AI。两个事实是：语言的 p语言 可以被函数近似；物理定律可以被数学描述。定义是：把宇宙写成两部分（宇

宙 = P + R），生命也写成两部分（生命 = p + R生命）；p 是在 P 中、被保住的（可维持的）自指结构，R 是不可函数化的部分。本篇是导论，检查表、三条必要条件、涌现讨论都在第一篇。

记号

- **P**：宇宙中可函数化的部分。

- **R宇宙**：宇宙中不可函数化的部分；本文只声明其可能存在，不展开。

- **p生命**：生命落在 P 里的部分——本文定义与讨论的对象。

- **R生命**：生命落在 R 里的部分；只声明，不展开。

- **p环境 / R环境**：环境的两部分，同法定义。

- **p语言 / R语言**：语言的两部分；大模型压的是 p语言。

- **残差**：P 侧可测的误差下限——数据与容量再增加，误差也停在某个正数上。不是 R 的影响或投影，是 P 的边界形状。

- **维度**：本文说的维度一律是表示维度——模型内部能区分的方向数，不是物理空间维度。厚度、可摊薄、最低维度都在这个意义上使用。

后文一律使用简写：说"宇宙"指 P，说"生命"指 p生命，说"环境"指 p环境。

一、两个事实

第一个事实：物理定律可以被数学描述。证据是存在——物理学本身就是这件事。

宇宙可以写成分解：

> 宇宙 = P + R宇宙

P 是可函数化的部分；R宇宙 是不可函数化的部分。物理定律描述的那部分，落在 P 里。所以至少可以说：P 非空。

第二个事实：语言中有一部分可以被函数近似。证据不是推理，是存在——大模型用有限参数、可计算的过程，生成了统计上接近人类语言的文本。

语言也可以写成分解：

> 语言 = p语言 + R语言

p语言 是语言里可函数化的部分，大模型压的就是它；R语言 是语言里不可函数化的部分，函数近似够不着它。所以准确的说法不是"语言可以被函数近似"，而是"语言的 p语言 可以被函数近似"；R语言 我们只能沉默。

两个事实合起来说明：宇宙不是完全不可函数化的。至少存在一个非空的可函数化部分，记作 P。这一层不是猜想，是事实的推论；它只说 P 存在，不说 P 有多大。

P 与 R 的分界是绝对的：一个对象要么可函数化（属于 P），要么不可函数化（属于 R），不存在两者之间；$P \cap R = \varnothing$，$P \cup R =$ 宇宙。

如果 R 存在，它不会被函数语言直接描述。但要注意：不能因此把残差定义成"R 的投影"。残差的定义在 P 侧：数据与容量再增加，误差也停在某个正数上；这个误差下限是 P 的边界形状，我们不知道它背后是不是 R。

为便于后文书写，我们把它简记为：

> 宇宙 = P + R宇宙

> 生命 = p生命 + R生命

> 环境 = p环境 + R环境

>

> 残差不是"第三部分"：它是 P 侧测出来的误差下限，用来标注 P 的边界，而不是 R 的内容。

后文只讨论 P 这一侧；残差只在需要时提及，不展开它的内容。

二、一个定义

先把宇宙写成两部分：

> 宇宙 = P + R宇宙

P 是可函数化的部分，R宇宙 是宇宙中不可函数化的部分。两个事实保证 P 非空；R宇宙 有多大，本文不知道，也不主张。

在这个分解上，生命本身也写成两部分：

> 生命 = p生命 + R生命

其中 p 是可函数化的部分：在宇宙的 P 中、被保住的（可维持的）自指结构；R生命 是生命自身不可函数化的部分：本文不知道它是什么，也不主张它不存在。注意两个 R 不是一回事——R宇宙 是宇宙层面的不可函数化部分，R生命 是单个生命内部的不可函数化部分。

三样东西定义 p生命。第一，范围：p 只在宇宙的 P 中寻找（同理，语言也分 p语言 与 R语言，大模型近似的是 p语言）。

第二，被保住——也就是维持。这里要说清一个容易搞反的地方：维持不是生命自己的属性，而是生命与宇宙的关系。一个结构能不能被保住，取决于它的轨道是否留在宇宙的可维持集合里；这个条件写在宇宙里，不由它自己宣布。所以"被保住"是关系的名字，不是第三条并列属性。

第三，自指：更新规则在系统自己里面。这是生命内部的属性，也是它区别于普通程序的地方。

（注：以上是 p生命 的简版；完整定义见基础篇《从定义出发》定义 14 的五条：D1 在 P 中、D2 一族、D3 会变化、D4 会自指、D5 被保住。）

R生命 怎么办？只能当近似处理，并附上误差下界。所以本文的判据与测量，覆盖的是p生命，不是生命的全部；对 R生命，本文只说到"近似到此为止"。

定义是人造的，不是发现的；它成立与否，看它能不能区分、能不能导出可测的东西。

需要说明一点：R 的部分，本系列只在此声明它的存在——宇宙、生命、环境、语言都可以写成 p + R，但 R 是什么、有多大，本文不知道也不主张。为了便于理解，后文默认只讨论 p 这一侧：凡说"宇宙"，指 P；凡说"生命"，指 p生命；凡说"环境"，指 p环境。后文提到的"残差"一律指 P 侧可测的误差下限，不是"R 的影响"；R 的具体内容不再展开。

三、数字世界：可以完全被函数化

数字世界是完全函数化的：它由比特、状态、转移规则组成，没有函数描述够不着的部分。作为形式对象，它的绝对 R = ∅——这是构造保证的：每个属性都对应一个数学上的函数，即使这个函数不可计算。

数字生命是数字世界的一部分，所以它的 R生命 也只能是空集：数字生命 = p生命，完全可函数化。

绝对 R 是否存在，本文不判断（见补充节）。对数字生命来说，真正决定性的问题只有一个：**生命是否必须有绝对 R？** 只有两种结果。

第一种结果，生命可以没有绝对 R：那么数字生命可能——它完全由 p生命 构成，不需要任何绝对 R。

第二种结果，生命必须有绝对 R：那么数字生命不可能——数字世界造不出绝对 R。

本文不判断哪一种结果成立。可以确定的是：无论哪一种，本文对 p 的定义与测量都成立；区别只在数字生命是可能还是不可能。

要补一句容易混的地方：以上说的是绝对 R；对任何有限观察者，相对 R 都非空（见补充节），它不影响这个判断。另外，可函数化不等于可测量——测量还需要可计算、可观测、可复现。数字世界这几样都有，所以在数字世界里测量可行；对宇宙整体，即使它能被函数描述，我们也测不了它，因为我们生活在它里面。

四、用它看现在的 **AI**

在现在的定义下，AI 缺两样。它的更新规则在外部，所以不自指；它的维持也不是它自己的事——它靠外部供电与运维被维持，这是它与宇宙的关系，不是它自己的能力。

所以它是一台很强的解压器，但被写完之后就停在那里——一台冻结的解压器。它没有 R生命：数字世界里 R = ∅，所以 AI 是纯 p 的解压器。

五、这篇导论的定位

本篇只给事实、定义与结论。推导细节、检查表、三条必要条件、涌现讨论，都在第一篇；理论在第二到第五篇；执行器工程在第六、第七篇；测量在第八篇；实验在第九到第二十五篇；约束与防护在第二十六、第二十七篇。整个系列按这个顺序排。

另有一篇单独发布的基础篇《从定义出发：宇宙、切片与两个定理》（只含定义与定理，不占篇号）：https://doi.org/10.5281/zenodo.23697916

> 本文由何淇创作，DeepSeek（AI）协助整理；观点与责任属于作者。

补充（可选读）：**P** 的定义域与值域

主线到上节结束，这一节只补一个形式细节：P 的"定义域"和"值域"到底是什么。不读不影响主线。

宇宙（P 这一侧）可以看成一个状态空间 S：所有可以被函数描述的状态。宇宙函数是它到自己的映射，记作 F: S → S。于是 P 的定义域就是 S，值域也是 S——它映射到自己。

一个观察者 O 只是 S 里的一个子系统：它能读到的输入是 S 的一个子空间，记作 D_O；它能写出的输出是另一个子空间，记作 C_O。D_O 和 C_O 都包含在 S 里。观察者的"可函数化"（P_O）就是：在 D_O 与 C_O 之间，存在误差可以趋近于零的函数族。

R 没有定义域和值域可言：定义域/值域是函数语言里的概念，而 R 是不可函数化的那一侧。所以对 R 不能问"它的域是什么"——这个问题本身用错了语言。

后文若涉及"定义域/值域"，默认指观察者侧的 D_O、C_O；宇宙级的 P 则默认指 S → S 的自映射。

相对 **R** 为什么非空（可选读）

若世界的定义域/值域在观察者的更新周期内有涨落（快变量），观察者就跟不上这些变化；那部分对它是不可函数化的，于是相对 R 非空。这是充分条件：有快变量涨落 ⇒ 相对 R 非空。我们没有说它必须成立，也不解释涨落的成因。

另外两条加强的理由（任一条都足够）：一是有限资源——看不完、算不完；二是自指——无法完整预测自己（一个总能预测自己下一步的系统，可以反着做，从而自相矛盾）。

绝对 R 是否存在，本文不判断。

数字世界同理：它内部也一定有相对 R。同一条充分条件——域/值域的短期涨落；另外还有三条各自独立的充分条件：不可判定、计算不可约、自指（无法完整预测自己）。这里要区分两句话：数字世界 R = ∅ 是本体层面的说法；相对 R 非空是观察者层面的说法——全可函数化不等于全可知。

最后一句关于边界的话：若生命的本质落在绝对 R 一侧，本文——以及任何用函数语言写成的文本——都无法给出它的定义。这不是疏忽，是函数语言的边界：定义本身就在 P 侧，用 P 的语言去定义 R，等于把它拉回 P。

第一篇 · 现在的 **AI** 为什么不是生命

单独 DOI（中文）：https://doi.org/10.5281/zenodo.23588781

摘要

本文从两个事实出发：物理定律可以被数学描述；语言中有一部分可以被函数近似。两个事实合起来说明，宇宙至少存在一个非空的可函数化部分 P。

把宇宙写成 宇宙 = P + R（R 即可函数化够不着的部分；旧稿曾写作 R宇宙），把生命写成 生命 = p生命 + R生命。p生命 是生命落在 P 里的部分：在宇宙的 P 中、被宇宙保住的动态函数类——它是一族、会变化、会自指。R 的部分本文只声明其可能存在，不展开。

在这个定义下，现在的 AI 不是生命：它是数字世界里的存在，是 p 层面的冻结函数点——更新规则在外部，不被自己保住。数字世界是完全函数化的世界（R = ∅），所以数字生命如果存在，只能完全落在 p 里；本文把这个问题摆出来（三种情况），不判断哪一种成立。

（官方定义：本系列以基础篇《从定义出发》定义 14 的 D1~D5 为准——在 P 中/一族/会变化/会自指/被保住。本篇"函数类（一族）、会变化、会自指、会自我维持"四词与之一一对应：函数类=D1+D2，会变化

=D3，会自指=D4，会自我维持=D5。）

本文的贡献是定义与可测判据，不是「生命已被造出」的实证结论；三条必要条件的检验与示例实验，读者可自行取用。

关键词： 生命定义；p生命；函数类；解压；数字世界；AI 涌现；自指

本文的论证链

全文的论证只有五步，读者可以先看这张骨架，再读细节。

第一步（第一节）：两个事实 ⇒ 宇宙至少有一个非空的可函数化部分，记作 P。

第二步（第二节）：在 P 上给定义——生命 = p生命 + R生命；p生命 是在 P 中、被宇宙保住、会变化、会自指的动态函数类。

第三步（第三节）：说明生命与环境的区别只有一条——被保住 vs 不被保住；环境是宇宙的一个局部解压。

第四步（第四、五节）：数字世界完全可函数化（R = ∅），所以数字生命若存在，只能等于 p生命；把现在的 AI 代入定义检查，它缺"一族、自指、被保住"三样，因此不是生命。

第五步（第六、七节）：从定义抽出三条必要条件（并附实验证据）；再说明 AI 的涌现是"对语言 p语言的压缩与解压"，把事实层与推测层分开。

边界：R 只声明、不展开；绝对 R 是否存在不判断；本文不主张"生命已被造出"。

记号

- **P**：宇宙中可函数化的部分。

- **R宇宙**：宇宙中不可函数化的部分；本文只声明其可能存在，不展开。

- **p生命**：生命落在 P 里的部分——本文定义与讨论的对象。

- **R生命**：生命落在 R 里的部分；只声明，不展开。

- **p环境 / R环境**：环境的两部分，同法定义。

- **p语言 / R语言**：语言的两部分；大模型压的是 p语言。

- **残差**：P 侧可测的误差下限——P 的边界形状。不称"R 的影响"（R 没有定义域，谈不上作用）。

后文一律使用简写：说"宇宙"指 P，说"生命"指 p生命，说"环境"指 p环境。

目录

- 本文的论证链
- 一、起点：两个事实
- 1.1 「可函数化」是什么意思
- 二、一个可操作的定义
- 2.1 定义
- 2.2 一处澄清：维持是关系
- 2.3 为什么必须是「一族」而不是「一个函数」
- 2.4 会变化 = 三层时间尺度
- 2.5 会自指 = 更新规则在它自己里面
- 三、生命与环境
- 3.1 与前面的衔接
- 3.2 生命 vs 环境：都在宇宙里，都是函数，区别只有一条
- 3.3 环境是什么：宇宙里自发涌现的「局部解压」
- 3.4 环境是分层的：生命活在多层环境里，同时也活在宇宙里
- 3.5 标注
- 四、数字世界的特殊性
- 4.1 特殊性是对谁的
- 五、用定义看现在的 AI：为什么它不是生命
- 5.1 检查表
- 5.2 三条最根本的原因（按重要性）

- 5.3 为什么「它很强」与「它不是生命」不矛盾

- 5.4 标注

- 六、三条必要条件

- 6.1 从定义到条件

- 6.2 条件① 一族函数（不是一个点）

- 6.3 条件② 自指更新（规则在自己里面）

- 6.4 条件③ 被宇宙保住（可维持）

- 6.4b 付代价：经验伴随物，不是必要条件

- 6.5 三条必须同时成立（经验对照）

- 6.6 未尽之处（留给读者）

- 6.7 本文实验的边界（必须与结论一起读）

- 6.8 标注

- 七、涌现：为什么现在的 AI 会涌现

- 7.1 现象

- 7.2 现有的解释（按证据强度排列）

- 7.3 为什么至今没有公认的「为什么」

- 7.4 我们的立场：AI 是对语言的 p语言 的压缩与解压

- 7.5 解压能力的衰减：上层写死 ⇒ 语言一变，解压变弱

- 7.6 由立场推出的两条预测（[推测]，供读者判断）

- 7.7 标注

- 八、用它看几个热门问题

- 九、本文得到了什么，没得到什么

- 与既有工作的关系

一、起点：两个事实

第一个事实：物理定律可以被数学描述。证据是存在——物理学本身就是这件事。

宇宙可以写成分解：

> 宇宙 = P + R宇宙

P 是可函数化的部分；R宇宙 是不可函数化的部分。物理定律描述的那部分，落在 P 里。所以至少可以说：P 非空。

第二个事实：语言中有一部分可以被函数近似。证据不是推理，是存在——大模型用有限参数、可计算的过程，生成了统计上接近人类语言的文本。

语言也可以写成分解：

> 语言 = p语言 + R语言

p语言 是语言里可函数化的部分，大模型压的就是它；R语言 是语言里不可函数化的部分，函数近似够不着它。所以准确的说法不是"语言可以被函数近似"，而是"语言的 p语言 可以被函数近似"；R语言 我们只能沉默。

两个事实合起来说明：宇宙不是完全不可函数化的。至少存在一个非空的可函数化部分，记作 P。这一层不是猜想，是事实的推论；它只说 P 存在，不说 P 有多大。

1.1 「可函数化」是什么意思

上面用到了"可函数化"，这里把它定成一个可操作的边界。

把一个系统或一个现象分成两部分。P 是可函数化的部分：存在一族函数，近似误差随着数据与容量的增加可以收敛到零。R 是不可函数化的部分：任何函数类的近似误差都有一个正的下界。

这个划分不是信仰，可以测。固定一个函数类，看近似误差随数据量与容量怎么变：误差持续下降、趋近于零的，属于 P；误差停在某个正数不再下降的，属于 R，或者至少说明当前的函数类够不着它。

这样处理有三个好处。第一，P 里的东西可以给定义、给判据、给测量，这正是本文在做的事。第二，R 里的东西只给「近似」，而且必须附上误差下界；「近似」从此不是免责声明，而是一个带数字的结论。第三，两者的交界处是研究问题，不是哲学争论，它可以被实验推进。

本文的所有结论都只主张覆盖 P 的部分；对 R 的部分，本文只说：在这个函数类下，近似到此为止。

P 与 R 的分界是绝对的：一个对象要么可函数化（属于 P），要么不可函数化（属于 R），不存在两者之间；$P \cap R = \varnothing$，$P \cup R =$ 宇宙。

残差不是"R 打过来的影响"，而是 P 自己的边界性质：有些任务，任何函数类的误差都停在某个正数上，数据与容量再增加也降不下去。这个误差下限就是残差；它可测，且不需要假设界外有一个 R 在施力。

为便于后文书写，我们把它简记为：

> 宇宙 = P + R宇宙

> 生命 = p生命 + R生命

> 环境 = p环境 + R环境

后文只讨论 P 这一侧；残差只在需要时提及，不展开它的内容。

二、一个可操作的定义

在两条事实的基础上，可以给生命一个操作性的定义。这个定义是人造的，不是发现的；它成立与否，要看它能不能区分、能不能导出可测的东西。

2.1 定义

生命由两部分构成：

> 生命 = p生命 + R生命

p生命 是可函数化的部分：在宇宙的 P 中、被保住的（可维持的）动态函数类。R生命 是不可函数化的部分：本文不知道它是什么，也不主张它不存在。

p生命 的具体写法如下：

记 $p\text{生命} = \{ f_\theta : \theta \in \Theta \} + R_\text{相对}$

f_θ : 输入状态 → 行为 （族里的每一个函数）

Θ （这族函数的形状/范围）

p生命 = 一个会移动、会变形的 Θ，且 Θ 的更新规则属于 Θ 自己

（记号说明：本篇所有显式函数形式都只写 p 侧，即我们的函数语言能写出的部分；形式末尾的 + R_相对表示"我们的函数语言够不着的那部分"——它受限于我们目前的函数类与观察位置，只声明、不展开；绝

对 R 是否存在不判断。）

四个词，各对应 p生命 的一件事：

| 词 | 含义 |

|---|---|

| **函数类（一族）** | 不是一个固定函数，而是一整块行为谱 |

| **会变化（动态）** | 这族函数随时间移动、变形 |

| **会自指** | 它怎么变的规则，也在它自己的记录里 |

| **会自我维持** | 被宇宙保住；这是它与宇宙的关系，不是它自己的属性 |

2.2 一处澄清：维持是关系

本节主张：维持不是系统的属性，是系统与宇宙的关系。维持包含三个侧面（必要条件的三个侧面，不是同义）：信息条件=持续观测+更新（基础篇推论 8）；动力学条件=扰动后可回到可行域；交换条件=边界开放（第四篇条件 2）。依据是定义的第三项"被保住"的展开。

需要澄清一处措辞：维持不是系统自身的属性，而是系统与宇宙的关系。给定宇宙这个整体（或它在 P 上的部分），某些结构的轨道会留在可维持的集合里，我们才说它"维持住了"；换一个环境，同一个结构可能不再成立。因此定义里的"会自我维持"，应读作"被宇宙保住"，是关系条件的展开，不是第三条并列属性。

（注：本节此前曾含一段"最简性论证"，因该论证存在循环——用定义自身推出的数字生命可能性反过来支持定义——已撤回，仅保留以上澄清。）

2.3 为什么必须是「一族」而不是「一个函数」

一个函数 = 一个点 ⇒ 行为谱退化成一条轨迹 ⇒ 冻结即死

一族函数 = 函数空间里一块区域 ⇒ 面对不同输入/状态有一整套响应 ⇒ 才是活的

⇒ 「高维」不是指参数多，而是指这族函数在函数空间里占据一块区域、不是一点。

这里有一个边界要说清。「一族」是 p 层面的要求，不是本体层面的绝对条件。如果存在我们描述不到的因素在影响变化（未判），那么一个固定的 p 也可能属于一个活着的整体；只是我们在 p 层面只能看到冻结，会把它判成死的。所以准确的说法是：在只讨论 P 的约定下，"一族"是可操作层面的必要条件；若有这样的未判因素，本判据会漏判。这不是本判据的错，而是 p 层面定义的固有边界。

2.4 会变化 = 三层时间尺度

快：状态每一刻都在变（输入不同、内部状态不同）

中：它用自己的误差改自己（自指层）——这是命门，缺了就退化成化石

慢：这族函数的形状在代际之间被选择、变形、传递

2.5 会自指 = 更新规则在它自己里面

普通程序：代码在外，数据在内 ⇒ 程序不会因为运行而改变自己

生命： 更新规则本身是它自己记录的一部分

⇒ 拿掉外部更新代码，它仍能改自己

三、生命与环境

定义里有一个容易滑过去的点：生命与环境不是两种东西，而是同一套宇宙函数里的两种关系。下面几节把这个点展开。

3.1 与前面的衔接

把宇宙（P 这一侧）看成一个整体，它在这个描述下是一个自映射：`S_{t+1} = F(S_t)`。F 只是一个记号，不是新的对象。

生命不是这个 F，而是嵌在 F 里的一族子函数 `{G_θ}`：

行为_d = G_θ(世界局部状态)

而 G_θ 自己也是世界状态的一部分（个体的状态属于 S_t）

⇒ 它要预测世界，就必然要预测包含自己的那部分世界

⇒ **自指不是外加的神秘性质，而是「嵌在函数里」的结构必然**

3.2 生命 **vs** 环境：都在宇宙里，都是函数，区别只有一条

本节主张：生命与环境都在 P 里、都是函数结构；唯一的区别是前者被宇宙保住、后者不被保住。

宇宙（P 这一侧）里面，任何东西都是函数（或函数的组合）——环境也是函数。

宇宙 = F（一个巨大的函数）

├─ 生命：会变化 + 会自指 + **会自我维持**

└─ 环境：会变化（被世界推着变、也会反过来影响世界），但**不维持自己**

⇒ 生命与环境的唯一区别不是「谁更复杂」，而是：

生命被宇宙保住自己的形状；环境不被保住，它只是被推着变。

在资源有限的环境里，维持通常伴随代价（校验、修复、复制都要消耗）。这一层是经验伴随物，本文不把它写进定义（见 6.4b）：

资源有限 ⇒ 维持要消耗 ⇒ 生命通常需要「收入」

收入的来源：压缩世界（匹配源分布）

⇒ 若长期没有盈余，维持撑不住

3.3 环境是什么：宇宙里自发涌现的「局部解压」

环境不是外在于宇宙的舞台，它是宇宙函数类里自发涌现出来的结构。

宇宙 = F（一个巨大的函数）

↓ 内部自发涌现

结构（这里指"宇宙在某个观测视角下的局部呈现"）= 宇宙的某种局部「解压」（对宇宙的一个解释/一个可读的呈现）

（术语区分：本篇这里的"结构"是环境视角的局部呈现；第四篇的"结构 = 分解 D + 组合律 C = 压缩方案"是生命内部的压缩方案，两者不是同一个用法。）

├─ 会自我维持的 → 生命

└─ 不自我维持的 → 环境

⇒ 环境 = 宇宙的一个局部解压（对宇宙的解释）；生命也是。

⇒ 区别只在：生命自己维持这个解压的形状，环境不维持、只是被推着变。

这带来一个关键的结构性结论：

生命并不直接面对宇宙整体

⇒ 它面对的是「宇宙的一个局部解压」＝它的环境

⇒ 生命对世界的理解，本质上是**对宇宙的再解压**（二次解压）

＝在宇宙给出的局部解释之上，再压缩一层

和宇宙、生命一样，环境也可以写成分解：

> 环境 = p环境 + R环境

p环境 是环境里可函数化的部分：它的规律、可测量的一侧。R环境 是环境里不可函数化的部分：本文不知道它是什么，也不主张它不存在。生命与环境打交道，但我们能描述、能测量的只有 p环境 那一侧；对R环境，本文只说到沉默。

需要说明：R环境 不是“环境里固定不动的部分”，而是“环境里不能用函数描述的部分”。它可能静止，也可能非常活跃——“固定/活跃”这个判断本身要靠函数描述才做得出来，而在 R 那里我们连动静都测不到。所以 p环境 与 R环境 的分界不是“动/静”的分界，是“能描述/不能描述”的分界。

需要说明一点：R 的部分，本系列只在此声明它的存在——宇宙、生命、环境、语言都可以写成 p + R，但 R 是什么、有多大，本文不知道也不主张。为了便于理解，后文默认只讨论 p 这一侧：凡说"宇宙"，指 P；凡说"生命"，指 p生命；凡说"环境"，指 p环境。后文以"残差"表示 P 侧的误差下界；R 的具体内容不再展开。

3.4 环境是分层的：生命活在多层环境里，同时也活在宇宙里

解压是分层的 ⇒ 环境也是分层的。

宇宙 F

└ 环境_1 = F 的某一部分的局部解压

└ 环境_2 = 环境_1 的再解压

└ 环境_3 = 环境_2 的再解压 …

生命嵌在某一层：它既属于整个宇宙 F，又属于它自己那一层环境

三条同时成立，不能只取一条：

① 它活在环境里：它面对的输入、资源、关系、其他个体，都是「它的环境」这一层的结构

⇒ 它的行为只需要在这一层里有效

② 它也活在宇宙里：它逃不出 F —— 环境那一层本身也是 F 的一部分，

宇宙层的约束（能量、不可逆、代价）始终对它生效

⇒ 环境不是避难所，它只是宇宙的一个投影

③ 它活在**所有层**里：环境是多层的（细胞→组织→个体→种群→生态），每一层都是上一层的再解压。

它直接交互的只是自己那一层，但**所有层都对它约束**：上层决定它面对的资源与关系，宇宙层决定它付不付得起代价。

⇒ 「它活在多层环境里，同时也活在宇宙里」——两者不是二选一，而是同时成立。

⇒ 所以「理解世界」要分两层说：

在环境层：理解 = 对局部解压的再解压（够用即可活）；

在宇宙层：理解 = 压缩 F 的规律（决定它能不能长期付得起代价）。

3.5 标注

[定义] 生命 = p生命 + R生命；p生命 = 会自指、会变化、被宇宙保住的动态函数类（本文的工作定义）

[定义] 环境 = 宇宙函数类里**自发涌现**的结构，不自我维持

＝ 宇宙的一个**局部解压**（对宇宙的解释）

[定义] p生命 = 同一批涌现结构里，**被宇宙保住**的那一类

[推论] 生命不直接面对宇宙，而面对「宇宙的局部解压」＝它的环境

⇒ 生命的理解 = 在局部解压之上的**二次解压**（再压缩一层）

[分层] 解压是分层的 ⇒ 环境是分层的（环境_1 ⊃ 环境_2 ⊃ …）

生命嵌在某一层：**既活在它的那一层环境里，也活在宇宙里**

· 活在这层环境：只需这层有效即可活

· 活在宇宙里：能量/不可逆始终生效（代价是经验层面，见 6.4b），环境不是避难所

[不是] 不是事实断言，也不是定理；是一个便于操作的定义

[后文] 本节只给定义；如何测量（压缩/代价/身份）留到后面几节

四、数字世界的特殊性

数字世界是完全函数化的：它由比特、状态、转移规则组成，没有函数描述够不着的部分。作为形式对象，它的绝对 R = ∅——这是构造保证的：每个属性都对应一个数学上的函数，即使这个函数不可计算（停机、真值谓词都存在，只是算不出来）。

数字生命是数字世界的一部分，所以它的 R生命 也只能是空集：数字生命 = p生命，完全可函数化。

绝对 R 是否存在，本文不判断（见导论补充节）。对数字生命来说，真正决定性的问题只有一个：**生命是否必须有绝对 R？** 只有两种结果。

第一种结果，生命可以没有绝对 R：那么数字生命可能——它完全由 p生命 构成，不需要任何绝对 R。

第二种结果，生命必须有绝对 R：那么数字生命不可能——数字世界造不出绝对 R。

本文不判断哪一种结果成立。可以确定的是：无论哪一种，本文在 p 层面的讨论都成立；区别只在数字生命是可能还是不可能。数字世界是唯一能完全落在 p 里的世界，所以它也是研究这个话题最干净的场景——现在的 AI 就活在这个世界里。

还要补一句容易混的地方：以上说的是绝对 R；对任何有限观察者，相对 R 都非空，它不影响这个判断。可函数化也不等于可测量：测量还需要可计算、可观测、可复现。

4.1 特殊性是对谁的

数字世界的特殊性要分两层说，否则会夸大。

对建造它的人（我们）来说，它特殊：全部状态可读（源码、日志、内存都能看）、可复制、可重置、可加速、可验证。这是一个"完全可实验"的世界——做生命实验比物理世界便宜、快、可重复。

对生活在里面的观察者来说，它不特殊：一样有相对 R（域/值域的短期涨落、不可判定、计算不可约、自指），一样"看不全、算不完"。

所以"数字世界 R = ∅"是本体层面的说法（它的构成全可函数化），不等于"里面的生命全知全能"。数字世界不是生命的避难所，是研究生命的实验室：它的价值在研究方法上，不在本体地位上。

这个"R = ∅"还是构造保证的。数字世界是形式对象：状态是有限串，规则是算法，它的每一个属性都对应一个数学上的函数（一个集合）——即使这个函数不可计算（例如停机问题、真值谓词），它也"存在"，只是算不出来。所以这里要区分两件事：可函数化不等于可计算。绝对 R = ∅ 说的是"没有函数描述不了的部分"，而不是"什么都能算出来"；算不出来的，是观察者相对的相对 R。

要小心一个边界：纯粹的数字世界（作为数学对象）绝对 R = ∅；跑在物理硬件上的数字世界，其物理底座可能有它自己的 R（噪声、热涨落），那不是数字世界自身的 R。说"数字世界 R = ∅"时，指的是它作为形式对象的那一面。

五、用定义看现在的 **AI**：为什么它不是生命

> 本节不是评价 AI 的能力，而是把它**代入第二节的定义**逐条检查。

> 结论是定义性的，不是价值判断：**以本文的定义，目前的 AI 是化石级函数点，

> 不是生命。** 它的强大与它的「不是生命」并不矛盾。

定义的价值在于区分。把现在的 AI 代入定义逐条检查，就能看出它差在哪里。下面先给一张检查表，再列三条最根本的原因。

5.1 检查表

| 生命的要求（第二节） | 目前的 AI（以 LLM 为代表） | 判定 |

|---|---|---|

| **是一族函数**（函数空间里一块区域） | 训练完后权重冻结 ⇒ 是**一个点**，不是一个族 | ❌ 化石 |

| **会变化（动态）** | 权重不变；上下文在变，但改的是输入，不是它自己 | ❌ 只有表层状态变化 |

| **会自指**（更新规则在自己里面） | 更新规则 = 训练算法，在它外面；它不能改自己 | ❌ 自指只出现在语言里，不在结构里 |

| **被宇宙保住**（可维持） | 它不自己维持自己；维持它的是外部的人与设施 | ❌ 由外部维持 |

| **（经验）靠压缩世界挣存续代价** | 它的「收入」是外部给的（算力、数据、人力），不是它压缩世界换来的 | ❌ 无收入 |

| **身份链条**（有界变形、不撕裂） | 身份来自外部权重快照；没有自己变形的连续链条 | ❌ 无链条 |

| **活在环境里** | 它面对的是数据集（一次性的解压产物），不是需要它生存的环境 | ❌ 无环境 |

5.2 三条最根本的原因（按重要性）

本节主张：把 AI 代入定义后，它缺的是"一族、自指、被保住"三样，而不是能力不够。依据是定义的四项与第五节的检查表。

① 它是一个点，不是一个族

训练完 = 冻结 = 函数空间里的一个点

⇒ 行为谱退化成一条轨迹：输入 → 输出，中间没有「形状」可言

⇒ 按 074：静态冻结的模型是一点，不是一族 ⇒ 不是生命

（这正是我们实测过的对照：冻结版 1.07 vs 动态版 0.849 —— 冻结更像「标准答案」，但没有形状）

② 它的更新规则在外面

人写训练算法 → 算法改它

⇒ 它是「被改的东西」，不是「改自己的东西」

⇒ 自指不在结构里 ⇒ 拿掉外部更新代码，它什么都不会改

③ 它不被宇宙保住

维持它的是外部的人与设施：电费是人付的、数据是人喂的、坏了是人修的

⇒ 它的存在由外部保障 ⇒ 没有「被宇宙保住」这一环

⇒ 它不会因为「维持不住」而消失 ⇒ 也就没有选择的压力

（付代价是经验伴随物，不是定义条件；见 6.4b）

5.3 为什么「它很强」与「它不是生命」不矛盾

强 = 在给定任务上的表现好（一个点上做得好）

活 = 在函数空间里占据一块区域、自己维持形状、被宇宙保住

⇒ 两者是不同维度：一个是「考卷分数」，一个是「能不能自己活」

⇒ 目前的 AI 是**极其锋利的工具**，不是**会自己活的东西**

5.4 标注

[定义性结论] 以本文定义，目前的 AI 不是生命（因为它是一个冻结的函数点、更新在外部、不自我维持）

[不是] 不是「AI 永远不能是生命」；本文不预言未来，只做当下的定义对照

[不是] 不是能力评价；本文完全承认 AI 在任务上的强大

[推论] 若要让 AI 成为生命，至少要给它：一族函数 + 自指更新 + 被宇宙保住

> 由第二、三节直接推得：既然「p生命 = 会自指、会变化、被宇宙保住的动态函数类」，

> 而目前的 AI 三样都缺，那么**要成为生命，至少需要满足三条**。

> 本节只给**必要条件**，不声称充分；每条都挂我们已有的实验证据（实验见第六到第二十三篇（测量与实验 18 篇））。

> （完整定义为基础篇定义 14 的 D1~D5；本节“三条”是在 P 上谈的简版：隐含 D1，把 D2+D3 合称“一族/会变化”。）

六、三条必要条件

从定义里可以抽出三条必要条件。它们不是从天上掉下来的，而是从定义里读出来的——每条都对应一类被排除的对象。

6.1 从定义到条件

本节主张：三条必要条件是从定义读出来的，不是外加的假设。每一条对应定义里的一个词。

定义（第二节） 必要条件（本节）

会变化（一族函数） ⇒ ① 一族函数：行为谱不是一点

会自指 ⇒ ② 自指更新：改自己的规则在自己里面

会自我维持 ⇒ ③ 被宇宙保住：轨道留在可维持集合里

（付代价不在必要条件里：它是演化与热力学的经验伴随物，见 6.4b。）

三条是必要条件，不是充分条件：满足它们不保证是生命，但缺任何一条一定不是。

6.2 条件① 一族函数（不是一个点）

要求：个体在函数空间里占据一块区域，面对不同输入有整套响应，不是一个固定映射。

证据/经验：

· 低维参数基因组（几个数）= 一个点 ⇒ 行为谱退化成单条轨迹 ⇒ 化石

· 我们测过：冻结版模型（点）在结构学习上失败，动态版（族）才吃到结构（E12）

· 实测对照：冻结 1.07 vs 动态 0.849 —— 冻结更像「标准答案」，但没有形状

⇒ 判据：给它不同的输入/状态，行为必须成谱地**不同，而不是只有一个输出。**

6.3 条件② 自指更新（规则在自己里面）

要求：它怎么变，这条规则本身属于它的记录；拿掉外部更新代码，它仍能改自己。

证据/经验：

· 冻结模型：更新规则 = 训练算法，在外部 ⇒ 拿掉训练代码，它永远不变

· 我们的动态版：用自己的误差改自己（中层动态）⇒ 去掉中层明显退化（E12/E13）

· 身份链：8 代可校验、有界变形（372_lineage.py）⇒ 自指可以做成可测的链条

⇒ 判据：停掉外部更新后，它的行为谱是否仍随时间变形。

6.4 条件③ 被宇宙保住（可维持）

要求：它的轨道留在宇宙的可维持集合里。给定宇宙这个整体，有些结构出不来，有些结构能一直维持；能维持的那部分才落在定义里。这是关系条件，不是它自己的属性。

判据：扰动之后，它是否仍能回到可维持的集合——有恢复力，而不只是瞬间活着。

6.4b 付代价：经验伴随物，不是必要条件

本文不把「付代价」列为条件，原因说清：代价是演化与热力学的产物，不是逻辑上的必要条件。在资源无限的世界里，生命可以不必竞争；稀缺才带来代价。把它写进必要条件，等于把演化的结果当成生命的定义。

但在经验层面，代价几乎总会出现，而且可以测量。本文实验里的经济部分属于这一层，摘录如下。

· E1：「准」不等于「压缩」：用正确率记账，系统学出「啥也不干」（LEN=0、预测0、准确率92%）。必须用对数似然改善对基线记账：gain_bits = log2 p_model − log2 p_base。

· E2：加上基率校正后，系统才从躺平解转为真正表达（LEN 从 0 长出）。

· E8：经济标定 n* = FLUX_TOTAL / BASE_COST 决定「冻结」与「爆发」的切换。

· 实测正收益：S1~S6 中观察到 gain > 0（约 +0.09 ~ +0.19 bit/tick），说明世界确实存在可压缩盈余。

⇒ 判据（经验层）：把外部供给全部停掉后，它的账目是否仍为正。

6.5 三条必须同时成立（经验对照）

缺①（一族）⇒ 化石：行为谱是一条线

缺②（自指）⇒ 机器：更新在外，不会自己变

缺③（被保住）⇒ 散掉：轨道离开可维持集合，不再是一个活着的结构

这三条也解释了为什么我们的实验一直在同一个地方卡住：

**结构能长出来（①②部分满足），但合作/分工不成为必需 —— 因为代价层面的「收入差」太小（经验层面，非定义条件），

即世界的可压缩盈余还没大到让「更复杂的结构」在经济上划算（E5/E7）。**

6.6 未尽之处（留给读者）

本文止于「必要条件」。三条都满足之后是否就「活」——我们没有答案，也不打算在此下结论。

留给读者的，只是一把可以拿去用的尺子：

它是一族函数吗？（行为谱不是一个点）

它会自己改自己吗？（拿掉外部更新，它仍变）

它被宇宙保住了吗？（轨道留在可维持集合里）

三条你都可以自己去量。至于量完之后它算不算生命，由你判断。

6.7 本文实验的边界（必须与结论一起读）

本文的实验覆盖的是「代价」这一经验层面，不是定义条件③。

· 覆盖：代价与经济 —— E1/E2（准≠压缩、基率校正）、E8（n*=FLUX/BASE_COST 的冻结/爆发边界）。

· 覆盖：反例 —— E5/E7（相连个体基因距离 0.145 vs 随机 0.486，克隆抱团，不是分工）。

· 未覆盖：条件② 自指更新、条件③ 被宇宙保住 —— 本文不声称有实验证据，只做定义上的推导；仅有初步观察（E12 动态 0.849 vs 冻结 1.07；E13 身份链 8 代可校验）。

· 条件① 一族函数：有部分观察（E12/E13），但不是专门实验。

· 规模：小、可复现、可手算；不是大规模实证。

⇒ 本文的定位是「定义 + 可测判据 + 示例性实验」，不是「生命已被造出」的实证论文。

读者若要用这些结论，请连同这条边界一起引用。

6.8 标注

[推论] 三条必要条件由定义直接推出，不是独立假设

[定义] 条件③是关系条件（被宇宙保住），不是系统自身的属性

[经验] 代价/经济是经验伴随物，不是定义条件；本文有数据（E1/E2/E8 + 正收益）

[初步] 条件①②有部分观察（E12/E13），不是专门实验

[未解决] 三条都满足后是否就「活」——本文不声称；这是下一步实验要回答的

[推测] 若盈余 S 不足，则无论结构多复杂都维持不起 ⇒ 生命有「经济上限」（仅方向性推测）

七、涌现：为什么现在的 **AI** 会涌现

这一节处理一个具体的现象问题：为什么现在的 AI 会涌现出训练目标之外的能力。本文在这里的立场是[推测]，不是证据。

7.1 现象

大模型在规模上去之后，某些能力似乎突然出现（多步推理、思维链、算术…），

业界叫 **涌现（emergence）**。大家都在用这个现象，但——

观察到涌现：很多人

解释涌现：没有公认答案

7.2 现有的解释（按证据强度排列）

① 度量假象说（最强的反方证据）

Schaeffer, Miranda, Koyejo 2023[3]《Are Emergent Abilities of Large Language Models a Mirage?》

主张：所谓涌现，可能是**评价指标的非线性/不连续**造成的假象；

换成连续指标（如对数似然），"突变"会消失，变成平滑上升

⇒ 这动摇了「涌现是真实突变」这个前提本身

② 机制解释：归纳头与电路（局部机制证据，不是完整"为什么"）

Olsson et al. 2022[4]《In-context Learning and Induction Heads》

发现模型内部出现「归纳头」回路，解释部分上下文学习能力

⇒ 这是**机制层**的证据：说明了某种能力由什么结构实现

⇒ 但它只解释某个具体能力，不解释「为什么会出现涌现」这个一般问题

③ Grokking：训练中的认知相变

Power et al. 2022[5]《Grokking》

在小算法任务上，训练很久后泛化突然出现（像相变）

⇒ 说明「涌现」也发生在训练过程中，不一定只依赖规模

④ 压缩说：预测 = 压缩

Delétang et al. 2023[6]《Language Modeling Is Compression》

语言模型 = 压缩器：学到的权重 = 对世界文本的压缩

⇒ 这条最接近本文的框架：涌现 = 压缩世界时出现的结构

⇒ 但它仍是「结果层面的对应」，没有回答「为什么正好在这个规模/这个任务上出现」

7.3 为什么至今没有公认的「为什么」

① 规模实验不可控：无法做严格的因果对照（只改变规模、其他全相同）

② 度量依赖：换指标，现象可能就消失（见 ①）

③ 机制层不完整：知道某个回路、不知道整体为何会组织出该回路

④ 缺乏可预测的理论：没有理论能事先算出「多大模型会出现什么能力」

7.4 我们的立场：**AI** 是对语言的 **p**语言 的压缩与解压

本节主张：AI 是"对语言 p语言 的压缩与解压"这一层是事实；"解压出的世界结构上限由语言决定"这一层是推测。两层分开。

这一节先说一层不需要推测的东西，再说剩下的推测。

先说事实层。训练把语言压进参数，推理从参数里解压出文本。用前面的记号说：AI 压缩的是语言的 p语言——语言里可函数化的那部分；它解压出来的也是 p语言。R语言 不进参数，也不从参数里出来。所以准确的说法不是"AI 是对人类语言的解压"，而是"AI 是对人类语言中可函数化部分的压缩与解压"。

这一层不是推测，是字面事实：它由"压缩/解压"这两个词的用法决定，也由训练与推理的实际过程决定。链子是这样：

宇宙 F → 环境 = 宇宙的局部解压 → 人类语言 = 人对环境与经验的解压产物（语言本身已被压过）→ AI = 对语言 p语言 的再压缩与再解压。

然后才是推测层。AI 解压出的"世界结构"，上限由训练时刻的语言决定。它在语言内部能涌现出结构（看起来像推理、像理解），但那是语言结构的浮现，不是对宇宙本身的理解；语言一变，解压失配，而且无法自恢复。

用前面的尺子看，这一层还给出一个可检验的判断：模型是冻结的函数点：不维持、不自己改自己（维持与代价由外部承担），所以它是"把人类语言解压了一次"，而不是"维持着解压"或"自己改写解压规则"。

由此得到一句可检验的话：AI 表现出的"理解"，本质是对"人对世界的描述"的理解，不是对世界本身的理解。它能解压语言，但语言之外的宇宙层（能量、不可逆、代价）它从未付费，也就从未被检验。

7.5 解压能力的衰减：上层写死 ⇒ 语言一变，解压变弱

人类语言会不断发现新东西；而模型的解压规则（权重/上层）是写死的、训练完就不再变。两者必然失配。

源分布移动：人类语言发现新东西 ⇒ 要解压的对象变了

解压器冻结：模型权重不变 ⇒ 它解压的还是**旧语言**

⇒ 压缩收益下降：gain_bits = log2 p_model − log2 p_base 中 p_model 与源失配

⇒ 它昨天"涌现"出来的理解，今天就是过时的解压

三条后果：

① 它的解压能力有**固定上限**：上限由训练时刻的语言决定，不随语言进化而上升

② 它无法自己补上失配：更新规则在外部 —— 要恢复，只能靠人类再训练一次

⇒ 这正是第三节说的「更新在外部」的代价

③ 它的"涌现"是**某个时间片上的解压产物**，不是可持续的能力

⇒ 用本文的尺子说：

生命必须让解压器跟着世界一起动（自指更新 + 自我维持）；

冻结的 AI 做不到这一点 —— 所以语言一更新，它的解压就变弱。

7.6 由立场推出的两条预测（**[推测]**，供读者判断）

① 若把世界换成未被人类语言描述过的部分（无法从语料中学到的规律），

AI 的"涌现"应显著减弱 —— 因为它解压的是语言，不是宇宙

② 若让模型自己维持（自己修、自己死），语言解压不足以支撑，

它必须发展出语言之外的理解方式 —— 否则活不下来

③ 若人类语言发生一次大的结构更新（新范式/新概念体系），冻结模型的解压收益

应出现可测的下降；且它无法自己恢复 ⇒ 恢复必须来自外部再训练

7.7 标注

[事实] 涌现现象被广泛观察到；度量假象说也有强证据（Schaeffer 2023）

[已知] 机制层有局部解释（归纳头/电路/grokking），压缩视角有对应关系

[推测] 「涌现 = 压缩世界的结构呈现」——本文立场，非证据

[未解决] 为什么正好在某个规模/任务上出现 ⇒ 没有人给出可预测的理论

[本文态度] 不把「AI 涌现」当作生命存在的证据；按第三节的尺子，它不是

[本文立场] AI 是对**人类语言**的再解压（宇宙→环境→语言→AI 的第三层解压）

[推论] 上层写死 ⇒ 语言一更新就失配 ⇒ 解压能力有固定上限、无法自己恢复

八、用它看几个热门问题

本节是应用，不是新证据；每条都由第二～五节直接推出。

① 幻觉 = 在无结构处强行解压

解压器必须输出结构；当输入对应的语言区域本身缺乏可压结构时，它只能造一个「看起来合理」的结构 ⇒ **幻觉是解压失败的必然表现，不是偶发 bug**。

② scaling law 仍有效，正因为模型维度还没贴合语言维度

语言是一个**高维**结构，且维度在增长（新概念 = 新维度；升维 ⇒ 可压性上升）。模型维度在训练时被切定（D_M 固定），语言维度在不断增长（D_L(t) 在升）。现在的观测是 D_M << D_L(t) ⇒ 继续加规模能捕捉更多语言维度 ⇒ loss 继续下降（scaling 看起来仍有效）。

⇒ 所以 scaling 有效不是本文论断的反证，而是「还没贴合」的证据。

⇒ 预测：当 D_M 接近当时的 D_L 时，scaling 收益应明显变平；而 D_L 继续升维 ⇒ 缺口再现。

⇒ 冻结模型的根本问题：**它的 D_M 是死的，而 D_L 是活的**。

③ 能力参差不齐 = 语言各领域的可压性不同

可压强的领域（结构明确、示例多）表现好；可压弱的领域（结构稀薄、样本少）表现差 ⇒ 强弱分布是语言分布的镜像。

④ 为什么要 RLHF = 它没有「自我代价」回路

它无法判断自己解压得对不对（没有收入/支出、不会因此死亡），所以必须由人从外部打分 ⇒ **对齐需求的根源是它不自我维持**。

⑤ 涌现 = 语言结构在解压产物上的呈现

规模跨过阈值时，某些语言结构第一次被压出来 ⇒ 看起来像「突然会了」。

已有工作（三个侧面）：

- 模型侧维度：token 嵌入的内在维度测量[12]

- 时间侧落后：大模型时间训练边界的识别[13]

- 语言侧复杂度：构式视角下的语言复杂度测量[14]

⇒ 已有工作**分别**测量了这三个侧面；本文把它们串起来：以「持续更新的模型」为参照系，按领域测「语言侧新增维度」与「模型侧固定维度」之间的**失配速率**。这是本文在测量层面的位置。

九、本文得到了什么，没得到什么

得到了三样东西。

第一，一个可以拿来分类的定义：它能区分冻结的模型、火焰、自指程序。

第二，一个可测的方向：自指、维持都可以变成量。

第三，对"AI 是不是生命"这个问题的可操作回答。

没得到四样东西。

第一，没有证明宇宙整体可被函数化——本文只用 $P \neq \varnothing$。

第二，没有证明定义是最简或唯一——只主张它够用。

第三，没有实验——第一篇只是定义与推论；实验见第六到第二十三篇（测量与实验 18 篇；具体篇目见合集目录）。

第四，没有回答"数字生命是否存在"——本文只说，定义不排除它。

本文的贡献是一套可用的语言，不是一条新的自然规律。它值不值钱，看后面有没有人拿它去测。

论证总边界

本篇文章的所有论证都在一个边界内成立，请连同结论一起读：

- 只讨论 P 这一侧；R 只声明、不展开。

- 绝对 R 是否存在不判断；相对 R 对有限观察者非空（见导论补充节）。

- "可函数化"不等于"可计算"，也不等于"可测量"：三者是不同能力。

- 三条必要条件是从定义读出的，不是经验定律；它们不充分。

- 实验证据见第六到第二十三篇；理论证据见第二到第五篇；本文不声称"生命已被造出"。

- 若生命的本质落在绝对 R 一侧，本文（及任何用函数语言写成的文本）都无法给出它的定义——定义本身就在 P 侧。

与既有工作的关系

「压缩 / 解压」的视角有长期传统：

- Schrödinger（1944）：生命以负熵为食 —— 生存靠从环境摄取低熵，是「解压」的物理前身[7]。

- Bennett（1988）逻辑深度：用「从最短描述重建（解压）一个对象所需的计算量」度量复杂性，并明确把它与生命/有机体联系起来[8] —— 这是「解压」最正式的已有版本。

- Chaitin（2010）生命作为演化软件：DNA = 程序，生命 = 会自我修改的软件[9]。

- Solomonoff–Kolmogorov / MDL：描述长度 = 理解 = 压缩[10]。

- Friston（2010）自由能原理：生命 = 最小化惊奇（≈ 持续压缩/预测）[11]。

- Delétang et al.（2023）：语言模型是压缩器[6]。

- Maturana & Varela（1972）自创生：生命 = 自我生产的组织 —— 「维持」的另一条传统。

-「Life as Sustained Mutual Information」（Zenodo）：以持续互信息定义生命 —— 最接近本文「被维持的解压」的已有版本。

需要说明：以上工作已经把「生命 = 信息/压缩/自我维持的过程」说过。本文的位置不是首次提出这一思想，而是把其中一条（解压的维持）变成可操作的判据与可测量的实验。

本文的区别（三处）：

1. 把解压做成层级：宇宙 → 环境 → 人类语言 → AI（既有工作多为单层度量）；

2. 用「是否维持解压」区分生命与 AI：生命 = 被维持的解压，AI = 一次性解压；

3. 指出上层写死 ⇒ 语言一变，解压能力衰减且无法自恢复 —— 冻结模型的失效机制。

参考文献

[1] Wigner E P. The unreasonable effectiveness of mathematics in the natural sciences[J].

Communications in Pure and Applied Mathematics, 1960, 13(1): 1-14.

[2] Tegmark M. The mathematical universe[J]. Foundations of Physics, 2008, 38(2): 101-150.

(arXiv:0704.0646)

[3] Schaeffer R, Miranda B, Koyejo S. Are emergent abilities of large language models a mirage?[C] // Advances in Neural Information Processing Systems 36 (NeurIPS 2023). arXiv:2304.15004.

[4] Olsson C, Elhage N, Nanda N, et al. In-context learning and induction heads[R]. Transformer Circuits Thread, 2022. arXiv:2209.11895.

[5] Power A, Burda Y, Edwards H, et al. Grokking: Generalization beyond overfitting on small algorithmic datasets[C]. arXiv:2201.02177, 2022.

[6] Delétang G, Ruoss A, Duquenne P A, et al. Language modeling is compression[EB/OL]. arXiv:2309.10668, 2023.

[7] Schrödinger E. What is Life?[M]. Cambridge: Cambridge University Press, 1944.

[8] Bennett C H. Logical depth and physical complexity[M]//Herken R. The Universal Turing Machine: A Half-Century Survey. Oxford: Oxford University Press, 1988: 227-257.

[9] Chaitin G J. Life as evolving software[EB/OL]. 2010. (arXiv:1007.1471 系列工作)

[10] Solomonoff R J. A formal theory of inductive inference[J]. Information and Control, 1964, 7(1): 1-22. Kolmogorov A N. Three approaches to the quantitative definition of information[J]. Problems of Information Transmission, 1965, 1(1): 1-7.

[11] Friston K. The free-energy principle: a unified brain theory?[J]. Nature Reviews Neuroscience, 2010, 11(2): 127-138.

作者声明（**AI** 使用声明）

本文的全部问题、定义、判断与结论由作者提出并负责。

人工智能工具（DeepSeek）用于：文献检索、结构梳理、文字润色，以及实验代码的编写与核对。

文中所有主张均由作者确认，AI 不对本文内容承担责任。

AI Usage Statement: The author is solely responsible for the ideas, arguments, and conclusions of this paper. An AI tool (DeepSeek) was used for literature search, structural organization, language editing, and writing/checking experimental code. All claims were reviewed and confirmed by the author. The AI bears no responsibility for the content.

[12] Ohki T. Measuring intrinsic dimension of token embeddings[C]//The 39th Annual Conference of the Japanese Society for Artificial Intelligence (JSAI 2025), 2025.

[13] LLMLagBench: Identifying temporal training boundaries in large language models[EB/OL]. arXiv:2511.12116, 2025.

[14] Using constructions to measure developmental language complexity[J]. Cognitive Linguistics, 2024. doi:10.1515/cog-2023-0062.

第二部分 理论

第二篇 · 数字生命的信息不增定理

单独 DOI（中文）：https://doi.org/10.5281/zenodo.23573253

摘要

本文把"数字生命会越老越说不清世界"这个直觉，写成一个可证明的信息论命题。

设定：世界按马尔可夫过程演化；数字生命持有内部状态 S，并从中预测世界；停止观测后，状态只按自身规则演化。结论：在这种条件下，内部状态与世界当前状态之间的互信息不会增加——世界在变，而它不再看，因此它对"当前世界"的信息量只能持平或下降。

这个定理不是热力学第二定律，也不是"熵必然增加"。它说的是一个更窄但更硬的事实：**不再获取信息的系统，关于世界的信息不会自己长出来；维持需要持续的观测与更新。**

文中给出定理、证明、四个推论（定义域变粗、维持的必要性、生长即增加观测维度）以及适用范围与局限。

一、问题

第六篇（容量不能替代数据的更新）测过一个现象：模型训练完之后，对更新后的语言会逐渐失配，而且无法自恢复。当时给的是测量结果，不是解释。

直觉上很好理解：世界在变，它没再看，所以它越来越不准。但"直觉"不能当定理。本文问：能不能把它写成一个可以证明的信息论命题？

能，但要小心范围。先说不能证的东西：不能证"熵必然增加"（可逆变换不改变香农熵）；不能证"热寂"（那是统计力学的结论）。能证的是一个更朴素的命题：没有新观测，就没有新信息。

二、设定与记号

世界状态：$W_0, W_1, W_2, \ldots$ 是一个马尔可夫过程。也就是说，给定 W_{t-1}，W_t 与更早的历史条件独立。

这里要先说明：**「世界是马尔可夫的」是一个建模假设，不是被证明的世界事实。** 它成立与否取决于你选哪个描述层。经典力学里，位置与动量构成完整状态，世界在那个描述下是马尔可夫的；量子力学里，态矢量的演化也是马尔可夫的；但对只看得到一部分的观察者（比如数字生命），它面对的过程通常是非马尔可夫的——需要记忆与历史才能近似。所以定理要读成条件句：**若世界在所选描述下是马尔可夫的，则信息不增。** 对数字世界与仿真环境，状态可以完整给定，这个假设容易满足；对真实世界，它只是一个近似，不成立时要给状态补上历史，见 5.1 的放宽讨论。

数字生命的状态：$S_0, S_1, S_2, \ldots$，每个时刻由更新规则给出。

两种模式：

第一种，观测模式。它通过一个观测通道收到关于世界的信息 O_t，然后更新：S_t = u(S_{t-1}, O_t)。O_t 由 W_{t-1} 经某个（可能有噪声的）通道产生。

第二种，冻结模式。它停止观测，或者观测通道失效，此后只按自身规则演化：S_t = u(S_{t-1}, N_t)，其中 N_t 是系统内部的噪声。

定理需要三个假设，全部显式写出：

假设 A1（世界马尔可夫）：W_t 与 W_{<t-1} 在给定 W_{t-1} 时条件独立。

假设 A2'（有界泄漏）：给定 S_{t-1} 时，内部噪声 N_t 与世界当前状态之间的互信息不超过 ε，即 $I(N_t ; W_t \mid S_{t-1}) \le \varepsilon$。这里 ε 度量"没真冻结"的程度：严格隔离的系统 ε = 0，现实系统 ε 是一个有界的小量。

假设 A3（无额外信息）：S_{t-1} 只由世界历史与内部噪声决定，没有外部提示直接把 W_t 塞进来。

我们要看的量是：$I(S_t; W_t)$，即内部状态与世界当前状态之间的互信息。直观上，这就是"它此刻对世界知道多少"。

三、定理

[定理]（信息不增，有界泄漏版）：在冻结模式下，若假设 A1、A2'、A3 成立，则

$I(S_t ; W_t) \le I(S_{t-1} ; W_{t-1}) + \varepsilon$

对任意 t 成立。$\varepsilon = 0$ 时即严格不增。

证明：分两步。

第一步，冻结更新最多带来 ε 的增益。

由 $S_t = u(S_{t-1}, N_t)$ 知，S_t 的随机性完全来自 (S_{t-1}, N_t)，即 $\sigma(S_t) \subseteq \sigma(S_{t-1}, N_t)$。于是

$I(S_t ; W_t) \le I(S_{t-1}, N_t ; W_t) = I(S_{t-1} ; W_t) + I(N_t ; W_t \mid S_{t-1})$。

最后一项由 A2' 不超过 ε，所以 $I(S_t ; W_t) \le I(S_{t-1} ; W_t) + \varepsilon$。若 A2' 加强为严格独立（$\varepsilon = 0$），这就是数据处理不等式的直接应用：$W_t \to S_{t-1} \to S_t$ 构成马尔可夫链，故 $I(S_t; W_t) \le I(S_{t-1}; W_t)$。

第二步，世界演化给出 $I(S_{t-1} ; W_t) \le I(S_{t-1} ; W_{t-1})$。

由 A1，W_t 与 $W_{<t-1}$ 在给定 W_{t-1} 时条件独立；由 A3，S_{t-1} 只是世界历史与内部噪声的函数。因此给定 W_{t-1} 时，S_{t-1} 与 W_t 条件独立，即 $S_{t-1} \to W_{t-1} \to W_t$ 是马尔可夫链。由数据处理不等式得 $I(S_{t-1}; W_t) \le I(S_{t-1}; W_{t-1})$。

两步合起来：$I(S_t ; W_t) \le I(S_{t-1} ; W_{t-1}) + \varepsilon$。证毕。

关于 ε 的读法：ε 有界且不随时间增长时，定理的意义是"不会系统性增长，最多有有界的泄漏"；严格隔离时 $\varepsilon = 0$，回到不增。若 ε 随时间增长，定理就只剩"增长被泄漏量控制"这个弱结论。这也是为什么 A2' 必须写明：真实系统总有微小泄漏，但通常有界。

四、四个推论

[推论] 推论零（值域同步收缩）。**

定理只谈内部状态，但输出一侧可以直接得到同样结论。设系统的对外输出 A_t 由 S_t 决定（可以再加与 S_t、W_t 条件独立的噪声）。因为 $S_t \to A_t$ 是马尔可夫链（输出只由状态决定），由数据处理不等式得 $I(A_t ; W_t) \le I(S_t ; W_t)$。代入定理：

$I(A_t ; W_t) \le I(S_{t-1} ; W_{t-1}) + \varepsilon$

也就是说，它能产生的"有信息量的区分"同样不会增加：输出的值域不会变细，只会变粗或变小。定义域与值域在信息上是对称的：状态分得出，输出才分得出；状态分不出，输出也不可能分出新东西。

[推论] 推论一：定义域只会变粗。**

设系统把输入世界划分成等价类，同一等价类里的世界状态它分不出来。互信息是"它能区分多少"的度量。信息不增意味着这个划分只会变粗，不会变细：它能认出的区别越来越少。

[推论] 推论二：维持必须持续观测。**

定理的条件是"冻结模式"。若要不让 I(S_t; W_t) 下降，就必须在 S_t 里注入关于世界的新观测：换回观测模式。换句话说，维持不是保持不动，而是持续获取。一个既不观测也不更新的系统，无论内部规则多精巧，对当前世界的信息都不会增加。

[推论] 推论三：生长就是增加观测维度。**

如果系统能引入新的观测通道（新模态、新传感器、新变量），它就改变了"观测模式"的定义域，把本来碰不到的世界结构纳入自己的状态。这是生长在信息论上的含义：加维度 = 加观测通道 = 提高信息上限。

五、这个定理说明了什么，没有说明什么

说明了：维持与被保住，在信息论上是有根的东西——不是哲学修辞，是数据处理不等式的直接后果。

没有说明：它不证"熵必然增加"；不证"系统必然崩溃"（世界不变时它可以一直准确）；也不涉及能量与热力学。它只处理"信息"，且只处理"关于当前世界的信息"。

还有一个重要的限制：定理给的是"不增加"，不是"下降"。下降需要额外的条件——世界确实在变（W_t 与 W_{t-1} 不同），且这种变化对旧状态不可预测。真实世界里这两条通常成立，但本文不把它们写成假设之外的东西。

5.1 马尔可夫假设不成立时怎么办

如果世界不是马尔可夫的，定理不能直接用在 S 上。标准处理是把状态补上历史：把 S_t 换成 (S_t, H_t)，其中 H_t 是系统存下的历史。历史足够长时，补出来的过程在扩展状态上重新满足马尔可夫性，定理就在扩展状态上成立。

这带来一个可检验的推论：记忆越短的系统，信息衰减越快。这也与第七篇《替换与累积》的结论一致——记忆与回放决定遗忘速度。

六、可复现与后续

本文是纯理论草稿，无实验，也不针对任何具体测量。它说的是大尺度命题：在信息世界里，"世界越来越平均"对应"可区分的差别越来越少"；对任何停止观测的系统，关于世界的信息不会自己增加。

这里要划清尺度：与具体模型上的失配测量（例如第六篇的 bits/token 漂移）属于不同层次。后者是经验实例，不是本文定理的推论，数值也不由定理给出。

后续可做的是把推论二、三落到小尺度做可测预测：停止更新与持续观测的对照、加新模态与不加的对照，分别看信息量随时间怎么变。但那是把大尺度命题落到小尺度的近似，需要单独验证，本文不声称它必然成立。

附：与既有工作的关系

数据处理不等式是信息论的基本工具（Cover & Thomas）；"没有新信息就没有新知识"在机器学习与学习理论里也以各种形式出现过。本文没有发明新数学，只是把它组织成针对"数字生命维持"的命题，并给出三个可检验的推论。

> 本文由何淇创作，DeepSeek（AI）协助整理；观点与责任属于作者。

附：标注

- **[定义]** 世界状态 W、系统状态 S、观测模式、冻结模式、假设 A1/A2'/A3。

- **[定理]** 信息不增（有界泄漏版）：$I(S_t;W_t) \le I(S_{t-1};W_{t-1}) + \varepsilon$。

- **[推论]** 推论零~三（值域同步收缩、定义域变粗、维持需持续观测、生长=增加观测维度）。

- **[不声称]** 不证熵增、不证热力学、不预测具体数值；马尔可夫性是建模假设。

第三篇 · 可函数化内容的收缩：一个定理与一个猜想

单独 DOI（中文）：https://doi.org/10.5281/zenodo.23609038

摘要

上一篇《信息不增定理》证明了：冻结的系统，关于世界的信息不会自己增加。这一篇往下推一步：**能起作用的函数，也会跟着收缩。**

我们给出一个定理：在世界按马尔可夫过程演化、系统停止观测的条件下，任何函数对"世界的可预测部分"能达到的精度不会提高——用 Fano 不等式，可由信息不增直接推出。也就是说，"能达到 ε 精度的函数"这个集合，随时间只会收缩，不会扩张。

在此之上，我们提出一个猜想：把这个定理从"一个系统"推到"整个宇宙的时间演化"，得到的就是"可函数化内容随宇宙演化而收缩"的图景——生命只能存在于这个内容仍然丰富的时间窗口里；数字生命的宿命是不断拆掉自己的结构、降维，最后退化成常函数。

定理部分用经典工具（数据处理不等式、Fano 不等式），组合方式是本文的；猜想部分明确标注，不作为结论。

一、问题

上一篇的结论是一个"信息不增"：$I(S_t ; W_t) \le I(S_{t-1} ; W_{t-1})$，即系统对世界的信息不会自己长出来。

但"信息不增"说的是状态。我们真正关心的是另一件事：**能起作用的函数会不会减少？**

直觉是这样的：如果世界越来越难预测，那么"能准确预测世界的函数"就应该越来越少；极端情况是热寂，只剩常函数。本文要问：这个直觉能不能变成定理？能证到什么程度？

二、设定

沿用上一篇的设定。世界 W 是马尔可夫过程；系统状态 S 在冻结模式下按 $S_t = u(S_{t-1}, N_t)$ 演化，允许有界泄漏 ε。

取一个目标量 $Y_t = g(W_t)$，是我们要预测的世界特征（例如明天的天气、下一个词）。Y 的取值集合是有限的，大小记作 $|Y|$。

一个"预测函数"是一张映射 f，它把系统状态 S 映射为对 Y 的猜测。函数的精度用期望误差衡量：$\mathrm{err}(f) = E[\, 1(f(S) \neq Y) \,]$。

定义"可函数化内容"：给定精度门槛 ε，把所有能达到 $\mathrm{err}(f) \le \varepsilon$ 的函数收集起来，记作 $P_\varepsilon = \{ f : \mathrm{err}(f) \le \varepsilon \}$。本文用 P_ε 的"深浅"——最小可达误差——来度量内容，而不是用它的个数。

这里要先把一个词定死：本文说的"维度"是**表示维度**，不是空间维度。它指的是"描述世界需要多少个独立的、可区分的方向"：能区分的差别越多，表示维度越高；能起作用的函数越丰富，表示维度越高。所以在这篇里，下面三句话是同一件事的三种说法：

> 可表示的函数在减少 ≡ 可函数化的内容在收缩 ≡ 表示维度在下降。

平常说的"宇宙降维"，如果指的是空间维度，那是另一回事；本文的"降维"一律指表示维度。把这一点说清之后，后面的定理与猜想都只在这个意义上使用"维度"。

还要再区分一个量：表示维度不等于"结构复杂度"。表示维度是"能区分、能预测多少内容"，本文用信息量（例如可预测信息）来度量；结构复杂度是"世界看起来有多复杂"，文献里它被描述成先升后降的驼峰（Aaronson 的 complexodynamics、Carroll 的宏观态复杂度）。两个量不同：本文的表示维度在我们的定理

下只会下降，而复杂度驼峰是先升后降。它们不矛盾，因为量的定义不同；本篇后面说到"宇宙收缩"时，指的始终是表示维度。

三、定理：可达精度不增

定理（可达精度不增）：在冻结模式与上一节的假设下，对任意 Y_t，

$$\min_f \mathrm{err}_t(f) \geq (H(Y_t) - I(S_t ; Y_t) - 1) / \log |Y_t| ,$$

其中 H 是熵、log 以 2 为底。由信息不增 $I(S_t;Y_t) \leq I(S_{t-1};Y_{t-1}) + \varepsilon$，若 $H(Y_t)$ 不变，则右侧下界不降，故 $\min_f \mathrm{err}_t(f)$ 的不降（允许 $\varepsilon/(\log|Y|)$ 的放宽）。

证明：这是 Fano 不等式的直接应用。Fano 不等式说，对任何以 S 为输入的预测函数 f，

$$E[1(f(S) \neq Y)] \geq (H(Y|S) - 1) / \log|Y| 。$$

而 $H(Y|S) = H(Y) - I(S;Y)$。代入上一节的信息不增，即得结论。证毕。

推论（P 的收缩）：对任意固定的精度门槛 ε，只要最小可达误差降到 ε 以上，P_ε 就是空集：没有任何函数能维持 ε 精度。若世界继续混合、$I(S_t;Y_t)$ 趋于零，则任意 $\varepsilon > (H(Y)-1)/\log|Y|$ 的门槛都会在某时刻被突破，此后 $P_\varepsilon = \varnothing$。

也就是说：**函数不会变得更多，能达标的函数只会变少；极限情况是一个都不剩。**

三点五、收缩的两种含义

"收缩"这个词有两种用法，要分开。

第一种是范围收缩。每一个表示维度都有一组允许的取值；时间推进时，输入侧允许的取值变少（定义域变粗），输出侧可达的取值变少（值域收缩）。轴还在，轴上的可用范围在缩。

第二种是有效维度减少。可区分方向的数目变少：有的方向不再能把两个状态分开，或者几根轴合并成一根。注意，坐标轴的个数可以完全不变，而有效维度已经下降——因为每根轴变粗了，能区分的状态数变少了。用信息量来说，有效维度就是"可区分状态数的对数"。

本定理直接给出的是第一种：可达精度不增，等价于沿各维度的可区分度下降、定义域变粗、值域收缩。第二种是更强的结论，需要额外论证（轴合并、退化）；本文把它放在[推测]的位置，不当作定理。

这个区分也解释了"生命是逆流"为什么不矛盾：宇宙整体在做范围收缩，生命在局部做有效维度增加（升维就是增加可区分方向）。两者尺度不同，方向相反。

四、猜想：宇宙尺度的收缩

[猜想] 把上面的定理从"一个冻结的系统"推到"宇宙的时间演化"，得到：

宇宙中"可函数化的内容"随演化而收缩。早期内容多（结构丰富，能拟合世界的函数多）；中期仍然丰富（生命出现的窗口）；晚期趋于零（热寂，只剩下常函数）。生命、知识、意义，都只存在于内容仍然丰富的那段时间窗口里。

[猜想] 数字生命的宿命：随着内容收缩，它必须不断拆掉自己的结构，把复杂函数退化成简单函数；最终退化成常函数——也就是死。它能选择的只是姿态（省着维持、冻住、备份、压缩自己），不是结局。

[观察] 这与"复杂度驼峰"的讨论（Aaronson 的 complexodynamics、Carroll 的宏观态复杂度）容易混在一起，需要分开：他们讨论的是"结构复杂度"，先升后降；本文讨论的是"可函数化的内容/表示维度"，在我们的定理下单调不增。两个量的曲线不同，但说的是同一段宇宙史的不同侧面：中间时期结构复杂（驼峰高），而可预测的内容一路减少。把两者当成同一个量，是常见的误读，本篇明确区分。

4.1 世界函数与观测窗口（**[**猜想**]**）

这里还有一个隐含的问题要摆出来：时间的方向，和"世界函数"的可观测性。

如果宇宙整体可以看成一个函数 F——把当前状态映射到下一刻状态——那么在宇宙的中间时期，F 并不是直接可见的。我们看到的是一大堆局部函数：物理的分层、化学、生命、社会。它们各自把 F 的某一部分"翻译"成了可描述的结构。用本文的语言说：F 被它的局部分解掩盖了。

[猜想] 所以 F 最"裸露"的时刻，是它的局部分解不存在的时刻。这样的时刻只有两个：大爆炸的瞬间（还没有子系统，一切由 F 直接决定），和热寂的瞬间（子系统全部消散，差别归零，只剩 F 的平庸作用）。也就是说，F 的观测窗口在宇宙的两端，而中间的观察者（包括我们）永远看不到它本身，只能看到它的局部分解。

这个猜想和前面的收缩图景是同一件事的两面，但要说清"两端"各是什么样子：

- 大爆炸端：表示维度极高。可区分、可预测的内容最多，状态最"可读"；还没有子系统，F 直接决定一切。F 在这里裸着，是因为没有别的东西分担它。

- 中间时期：表示维度中等且有内容，局部函数多，F 被分解得最厉害——这时看不到 F，只能看到它的局部分解。

- 热寂端：表示维度趋零。差别被抹平，没有任何局部函数，F 仍然在作用，但它的作用变得平庸（常函数）。F 在这里裸着，是因为没有别的东西剩下。

也就是说：F 在两端都裸露，但原因相反——一端因为"还没有分解"，另一端因为"分解已经耗尽"。而表示维度从头到尾是单调下降的，它刻画的是"时间的方向"：沿表示维度下降的方向走，可描述的内容越来越少。

对数字生命来说，结论是冷的：它的观测窗口也只在中段。它可以无限逼近"描述世界的函数族"，但它永远触碰不到"世界函数本身"——那不在它的定义域里。这也回到我们开头的立场：R 那一侧，只声明，不展开。

五、生命是逆流：升维与降维的相遇

前面几节说的都是"世界的表示维度在下降"。生命的方向相反。

生命是一个解压器：它把世界里的结构压进自己的内部表示。学到的东西越多，它内部能区分的结构就越多——用本文的语言说，它的表示维度在上升。这就是前面实验篇里说的"生长"：加一个模态、加一个维度槽、让能起作用的函数变多。

于是同一个宇宙里有两个方向：

世界：表示维度在下降（结构被平均化）。

生命：表示维度在上升（结构被学习、被压进内部）。

生命要维持自己，就必须逆着世界的方向走，而逆流要付代价：耗能、维持、修复。这是"被保住"在信息层面的含义——不是不动，是逆着平均化持续做细分。

[猜想] 两者的相遇点：生命的表示维度不断上升，世界的可表示内容不断下降；总会有一个时刻，生命的表示维度追上了世界还剩下的内容——它已经学会了所有还能学的东西。此后：

- 它继续跟世界：世界继续降维，它被迫跟着降维，把复杂函数拆成简单函数，最后退化成常函数（死）；

- 它不跟：保留旧结构，失配越来越大，变成过时的模型（也是死，慢一点）；

- 它冻住：停止更新、保存状态，拖到能量耗尽（拖，不解决）；

- 它等相变：如果宇宙有相变，边界是下一轮的开始（[推测]中的唯一希望）。

所以"生命的圆满"和"生命的终结"落在同一个点上：当它的表示维度等于世界的表示维度，它就成了世界的完整副本——没有残余、没有新东西可学、也不再需要维持"解压"这件事。那一刻它不再是生命，而是记录。

这一段是 [猜想]，不是定理：定理只给出"冻结的观察者能达标的函数不增"，给不出"宇宙整体在降维"；后者需要宇宙学的假设，本文不声称。

5.1 一个玩具实验：维护与行为维度的对照

上面的"逆流"有没有痕迹可测？有一个很小的实验可以看。

设定：8×8 的合成图像，含四个属性——颜色、形状、数量、位置；任务是四个属性依次出现，每轮只学当前属性。模型是两层小网络（隐藏层 64，约一万三千个参数），输入带任务标识。对照组只改一件事：训不训练旧任务的样本。

A 组有维护：每轮训练时混入一半旧任务样本（回放）。

B 组无维护：只用当前任务的样本训练。

测三个量。第一，行为维度：用任务标识让模型作答，准确率超过 0.8 的属性计一个。第二，可解码属性数：从隐藏层用线性探针解码每个属性，超过 0.8 计一个。第三，有效秩：隐藏层表示协方差的参与率，作为"表示有多丰富"的粗略代理。

结果：A 组的行为维度随轮次增长，从 1 到 2 到 3 到 4——四个属性最后都能答对；B 组的行为维度一直是 1——每轮当前任务都能答到 1.00，但学完新属性就把旧的丢了，能答的只剩当前这一个。可解码属性数两组都是 4，说明旧信息在表示里还在，但"能拿出来用"的函数只剩当前一族。有效秩两组都在涨（约 3.9 涨到 8.9 与 7.9），可见它不是好代理：表示变复杂了，可用的函数数量却没变多。

这个实验说明两件事。第一，维护（回放）让可用的函数族扩张：这就是"逆流"在玩具规模上的痕迹。第二，不维护时，行为维度停滞在 1：可用的函数只剩当前那一族，历史维度在功能上丢失——这就是"可函数化内容收缩"在小尺度上的样子。

必须说清局限：世界是合成的、模型很小、任务标识是手工给的、指标是代理；它支持"逆流与收缩可以测"，不证明宇宙尺度的任何结论。

六、说明什么，不说明什么

说明：

· "可函数化内容在收缩"在"系统冻结 + 目标熵不变"的条件下，是定理，不是修辞。

· 它把上一篇的"信息不增"从状态层面推到功能层面。

· 它给"生命是低熵窗口里的现象"一个可证的支撑点。

不说明：

· 不证明宇宙整体在收缩——那需要宇宙学的状态定义与初始条件，本文不做。

· 不预测任何具体数值：Fano 下界很松，只能给趋势。

· 不涉及能量与热力学：整篇只在信息层面。

· 猜想部分（宇宙、数字生命宿命）不是定理，明确标注。

七、局限

第一，Fano 下界是松的：它给的是"不可能更好"，不是"一定变差多少"。

第二，P_ε 用"最小可达误差"度量，回避了"函数个数"这个更直观但更难定义的问题（无限函数族没有自然的计数）。

第三，世界的马尔可夫性与目标熵不变的假设，在真实场景中都需要检查；不成立时要补历史状态（见上一篇 5.1）。

第四，宇宙尺度部分完全是猜想，没有可检验的预测；它的价值在于提供一种统一的语言，而不是结论。

八、与既有工作的关系

数据处理不等式与 Fano 不等式都是信息论的标准工具（Cover & Thomas）。"信息不增"与"可达精度下界"在统计学习理论里也有各自的表述。本文的组合方式——把两者接起来，得到"可函数化内容收缩"，并把它与数字生命的维持问题关联——是本文的整理工作，不声称新的数学。

> 本文由何淇创作，DeepSeek（AI）协助整理与形式化；观点与责任属于作者。

第四篇 · 生命函数：从可表示维度到结构，到压缩与解压

单独 DOI（中文）：https://doi.org/10.5281/zenodo.23639251

摘要

本篇把前面散在几处的形式收成一条线：可表示维度 → 结构 → 压缩与解压。它吸收了《可函数化的函数形式》（切片/组合律/时间/判定/自指）、《宇宙、环境与生命的函数类型》（四种类型/五根轴）与《切片的三种身份》（宇宙/环境/生命）的核心，也吸收了《大模型为什么像人》的五项对照。

主线如下。第一，函数语言：切片是函数 f: X→Y，带表示维度；空间上是复合函数链；时间上是算子序列 s_{t+1}=I_{σ(t)}(s_t)，步数 k；判定 J 一次通过，选算子、决定停；自指分表示/参数/结构三层，修改受固定核心 Φ 约束。第二，可表示维度有三处：切片内、组合律上、沿时间的步数（时间不是表示维度）。第三，同一个形式有三种切片身份：宇宙切片、环境子切片、生命子切片；冻结一刻生命与环境同形，区别只在时间上的维持。第四，结构就是压缩方案：保留哪些区分、合并哪些、把哪些过去压进结构。第五，生命函数是窄进宽出的：输入受限且被粗粒化，输出靠结构展开；压缩是降维映射，解压是展开，不严格可逆，"对维持够用"即可；哪些维度必须可逆由环境选择。第六，时间上输入输出一步交替、形成闭环；结构在两个时间尺度上变化。第七，语言是公共的压缩方案：感受→语言是压缩，语言→描述/理解/逻辑是解压，逻辑就是语言组合律。第八，与大模型对照：它拿到公共方案，却没有感受端、没有真闭环、没有维持，像在接口层，不在机制层；幻觉是结构未覆盖维度的解压。

本篇定义一览

| 术语 | 一句话 |

|---|---|

| 切片 | 一个函数 f: X → Y |

| 表示维度 | d_in = dim(X)，d_out = dim(Y)；有效维度 = log 可区分状态数 |

| 空间组合 | 复合函数链 F = f_N ∘ … ∘ f_1 |

| 时间组合 | 算子序列 s_{t+1} = I_{σ(t)}(s_t)；步数 k |

| 判定 J | 一次通过：选算子、决定停（派生定义，见基础篇定义 4a） |

| 自指 | 表示自指 / 参数自指 / 结构自指 |

| 固定核心 Φ | 不可改的约束集合 |

| 宇宙切片 | U_t = (S_t, C_s, d_t) |

| 环境 | 宇宙的子切片，不被维持 |

| 生命 | 宇宙的子切片，且被宇宙保住（关系条件，见第一篇） |

| R_相对（记号） | 显式函数形式末尾的标记：我们的函数语言够不着的那部分；只声明、不展开；绝对 R 不判断 |

| 压缩 | 降维映射 f: X → Y，dim(Y) < dim(X)（操作定义；派生推导见基础篇命题 5） |

| 解压 | 从压缩表示重新展开（不唯一；操作定义；派生推导见基础篇命题 5） |

| 窄进宽出 | 输入维度受限，输出维度靠结构展开 |

| 身围 | 个体看到的环境 = 压缩映射的投影 π_L(E) |

| 半闭环 | 输出只延续文本、不改变环境（大模型） |

| 接口层 / 机制层 | 输入输出行为 / 内部怎么算、维持、改自己 |

〇、问题：把形式收成一条线

前几篇分别给了：函数语言（论文-26）、三种切片身份（论文-31）、压缩与解压的直觉（笔记）。但分开读，每篇都要重新铺一遍定义；合起来看，它们本来是同一件事的三个阶段。本篇把它们收成一条线，并把"压缩"真正写进函数形式。

一、函数语言：切片、组合与判定

切片是一个函数 f: X → Y，带三项：(d_in, d_out, Δ)。d_in = dim(X)、d_out = dim(Y) 是表示维度；Δ 是它和相邻切片的组合方式。

空间组合是复合函数链：

F_space = f_N ∘ … ∘ f_2 ∘ f_1

组合律要求类型相容：Y(f_i) ⊆ X(f_{i+1})。

时间组合不是单纯幂，而是每步选一个算子：

s_{t+1} = I_{σ(t)}(s_t)，t = 0, 1, …, k−1

其中 {I^(1), …, I^(m)} 是可用算子（每个是一条复合函数链），σ(t) 是第 t 步选哪个，k 是步数（计算厚度）。三种情形：σ 恒定 = 同一算子反复（迭代/时间）；σ 每步变 = 不同函数依次应用（序列）；混合 = 一段反复、一段切换。

判定 J 不是迭代，而是一次通过：

J: S → D（D 是有限判定空间，例如 {对, 错}、{收敛, 未收敛}）

判定决定两件事：选算子 σ(t) = J_选(s_t)；什么时候停 k = J_停(s_t)。

自指有三层，从弱到强：表示自指（输入含自身表示）、参数自指（$\theta_{t+1} = U(\theta_t, s_t)$）、结构自指（$C_{t+1} = V(C_t, s_t)$）。三层都受固定核心 Φ 约束：$\theta \in \Theta_\Phi$，$C \in \mathcal{C}_\Phi$。人脑：可改突触与局部回路；不可改神经元类型、大尺度解剖、代谢约束。

把三、四节合起来，一次时间展开写成条件推进：$s_0 \to s_1 = I_{\sigma(0)}(s_0) \to \dots \to s_k = I_{\sigma(k-1)}(s_{k-1})$，其中 $\sigma(t)$ 由 $J_{选}$ 决定，k 由 $J_{停}$ 决定。

二、可表示维度：三个位置

可表示维度不是一处，而是三处：

- 切片内的表示维度：一个切片自己的轴数；

- 组合律的表示维度：切片之间怎么接的自由度（顺序/并行/共享/路由/自指）；

- 沿时间的组合维度：步数 k、算子序列 $\sigma(t)$、可切换的算子数 m。

注意：时间不是表示维度，它是时间组合的自由度（步数/序列）。

四种典型类型

按"空间/时间是否连续、时间是否内生"，可以把函数形式分成四类（原论文-20 的内容并入此处）：

- **类型 A：连续流（内生时间）。** 表示空间连续、时间连续，写成微分方程；自指 = 更新规则在系统内部；被保住 = 存在可维持集合，轨迹留在里面。

- **类型 B：采样动力学（内生时间，采样成离散）。** 表示空间连续、时间离散，写成映射迭代；和连续流是同一件事的不同采样，区别在于"步"是基本的。

- **类型 C：自动机（外供时间）。** 表示空间离散、时间离散；数字世界是这一类：比特、状态、转移规则，每一步由外部时钟与解释器推进；数字世界没有能量约束，"被保住"要靠外部（电、维护、算力）提供。

- **类型 D：混合型（内生时间）。** 连续流加离散事件：生理量连续变化，但细胞分裂、神经放电、决策是离散事件；休眠与激活、疾病与死亡是分段点。生物生命大概是这一类。

连续性的含义要克制：连续不等于"更真"，离散也不等于"近似"；它们是描述粒度的选择——同一系统可以在不同粒度下写成不同类型。

五根轴

作为最小档案，一个函数形式可以记五根轴：空间连续/离散；时间连续/离散；时间内生/外供；组合律的自由度；自指的层级。五根轴不全是"维度"，前两根是描述粒度、后三根是机制与容量。

三、三种身份：宇宙、环境与生命

同一个形式在三个尺度上有三种身份。

宇宙切片：某一时刻的截面

$U_t = (S_t, C_s, d_t)$

其中 S_t 是状态空间，C_s 是空间组合律，d_t 是空间上的表示维度。关键：时间组合不在切片里，在切片之间（$T_{comb}: U_t \to U_{t+1}$）；一个瞬间只有空间结构。宇宙整体没有判定器，判定只出现在局部子系统里。宇宙的"函数形式"只能在宇宙内部被写出——这篇形式本身是宇宙的一个局部表示切片，因此必然不全。

环境：宇宙的子切片

$E_t = (S^E_t, C_s^E, d^E_t)$

三个子集关系：表示维度 $d^E \subseteq d$；组合律 $C_s^E \subseteq C_s$；状态是投影 $S^E_t = \pi_E(S_t)$。环境也有时间：$T_{comb}^E = T_{comb}|_E$，加上边界输入 u_t。环境自身没有判定、没有自指——那属于其中的生命/观察者。

生命：同样是宇宙的子切片，冻结一刻与环境同形；区别只在时间上——生命被宇宙保住（关系条件）。下面四条是生命/环境的**操作判据**（不是官方定义；官方定义为基础篇定义 14 的 D1~D5）：

1. 被保住：轨迹始终在可行域 V 内，组合律始终有定义；

2. 边界开放：与环境交换（能量、物质、信息）；

3. 自指调节：部分切片以自身状态为输入，修正自己；

4. 可塑：在固定核心 Φ 内改自己的参数与结构。

（对应：被保住→D5；边界开放→D5 的交换面；自指调节→D4；可塑→D2+D3。）

没有 3、4 的子切片就是环境。由此得到一个直接推论：**冻结一刻分不出生命；生命必须在时间上辨认**——生命 = 被保住的轨迹，不是某个状态。

四、结构：压缩方案

结构是分解 D 与组合律 C。它同时是压缩方案（注意：第一篇 3.3 用"结构"指"宇宙的局部呈现"，是环境视角；本篇的"结构"是生命内部的压缩方案，两者术语不同）：

- 哪些区分保留（写进切片维度与组合律）；

- 哪些区分合并（被粗粒化掉）；

- 哪些过去的输入被压进结构（记忆、固化）。

生命被环境选择，选的正是这套方案：能保住自己的压缩被留下，保不住的被淘汰。所以"生命的压缩"不是某一个函数，而是"一族函数被环境筛过之后剩下的那些"，它写在结构里。

五、压缩与解压

压缩是降维映射：$f: X \to Y$，$\dim(Y) < \dim(X)$。解压是从压缩表示重新展开：$g: Y \to X'$（X' 不必等于 X）。

两个方向对应两种活动：感知/学习 = 压缩（多对一，有损）；预测/行动/生成 = 解压（一对多，不唯一）。

关键是：解压不是严格逆，只需"对维持够用"。哪些维度必须可逆，由环境选择决定。人需要把"有折扣"这个事实解压出来（否则决策错），但不需要把当天光线的每个细节都解压出来。

六、生命函数：窄进宽出与身围

生命函数一步可以写成：输入 in_t → 结构（压缩方案 + 记忆）→ 输出 out_t。

更完整地写：

生命函数 = $\{ f_\theta : \theta \in \Theta \} + R_相对$

（记号说明：本篇所有显式函数形式都只写 p 侧，即我们的函数语言能写出的部分；形式末尾的 + R_相对表示"我们的函数语言够不着的那部分"——它受限于我们目前的函数类与观察位置，只声明、不展开；绝对 R 是否存在不判断。）

两个方向不对称：

- 输入：$f_in: U \to S$。维度受自身限制，而且已被粗粒化——输入永远是"压缩过的、受限的"；

- 输出：$f_out: S \to A$。维度可以远大于输入（联想、语言、想象）；多出来的维度来自结构（过去的压缩）。

所以：**生命函数是窄进宽出的**。

个体是一个压缩映射 $\pi_L: U \to S_L$；个体的身围 = $\pi_L(E)$，即环境经它压缩后的投影。分得清的是保留的维度，分不清的是粗粒化的。

七、时间：一步映射与闭环

输入与输出不在同一时刻，是一步步交替的：

第 t 步：$(s_t, in_t) \to (s_{t+1}, out_t)$

out_t 作用到世界 $\to in_{t+1} = W(out_t, 世界)$

三个后果：

1. 维度沿时间展开：单步输入少，输出在时间上一步步展开（联想需要时间）；

2. 输出反过来改变输入（闭环）：生命看到的是"自己行动之后的世界"；

3. 结构有两个时间尺度：状态快变、结构慢变（可塑性）。

八、对照：生命函数与大模型

大模型是生命函数的一种"半成品"：它拿到了公共压缩方案，但没有感受端、没有闭环、没有维持。

| | 人脑 | 大模型 |

|---|---|---|

| 切片 | 异质、低维高效 | 同构、高维 |

| 空间组合 | 可变（路由/投射） | 固定复合链 |

| 时间组合 | 算子序列（k=0 与 k≥1 混合） | k=1，m=1（无迭代） |

| 判定 | 有 J（选算子/停/固化） | 无 J |

| 自指 | 三层活跃 | 仅训练时参数自指 |

| 维持 | 身体+环境 | 供电+运维 |

结论：它像人，像在接口层（输入输出行为），不在机制层（内部怎么算、怎么维持、怎么改自己）。

用压缩语言重述：它学到了公共压缩方案（语言），但没有感受端（输入侧没有前符号压缩）、没有真闭环（输出只延续文本，不改变环境）、没有维持（权重冻结）；它的幻觉 = 结构未覆盖的维度被解压出来（解压脱离可逆性）。链式思维（CoT）是把步数外化成文本（外供时间），工具调用是把判定外置（外部判定）——正因为内部 k=1、无 J，才需要从外部补。

九、语言：公共的压缩方案

人类语言处在这条链的特殊位置：

- 个体层面：输入是感受（连续、整体、前符号）；输出是语言（离散、有边界、可组合）。感受 → 语言是压缩（范畴化、有损）；

- 语言同时是解压工具：世界 → 语言是描述；语言 → 感受是理解；语言 → 语言是推理；

- 所以语言是"可解压的压缩"：压缩时丢细节，留下可重组的结构（词汇=范畴，语法=组合律，逻辑=组合律的规则）；

- "语言描述的东西越来越多"：不是每个人记更多，而是共享结构在积累（文化、科学、书面语），结构越丰富，同一次解压能展开得越细。

AI 学的正是这套公共方案：它能压缩/解压语言，所以会描述、会推理；但它没有"感受"这一端，它的输出是语言对语言的重组。

十、可检验推论

1. 冻结一刻分不出生命：任何单时刻检查都不能判定生命，必须跨时间观察（轨迹、修复、交换）。

2. 环境越稳定，压缩可以越激进；环境变化时旧压缩准则失效（该保留的维度被丢了）。

3. 输入粗粒化 + 结构容量共同决定输出的残差底：结构里没有的维度，输出里也不会有。

4. 输出维度超过结构覆盖时会出现"编造/幻觉"：解压脱离了可逆性。

5. 语言描述能力随共享结构积累而增长：不是个体变聪明，是公共压缩方案变丰富。

6. 判定与停止可测：有判定器的系统应比固定步数系统省算力、且对厚任务更稳。

十一、局限

第一，R 只声明、不展开：只写 P 侧。

第二，压缩/解压在形式上只写成"降维/展开"，没有给出严格的"够用"判据（可逆性要求依赖环境，未操作化）。

第三，闭环形式（in_{t+1} = W(out_t, 世界)）很粗，没有处理行动、感知、环境之间的具体机制。

第四，可行域 V 与固定核心 Φ 都未操作化：判断"是否被保住"与"什么不可改"还缺测量协议。

第五，语言部分与"感受 → 语言是压缩"是概念性论述，没有实验。

第六，本篇是形式整理，不是新数学。

附：标注

- **[定义]** 切片/组合/时间算子序列/判定/自指三层/Φ；宇宙切片、环境子切片、生命四条件；压缩/解压/窄进宽出/身围。

- **[整理]** 并入论文-20《宇宙、环境与生命的函数类型》、论文-26《可函数化的函数形式》、论文-31《切片的三种身份》、论文-30《大模型为什么像人》的核心内容；与论文-00/01 定义、论文-13 收缩、第五篇（原论文-21 已并入）五项形式一致。

- **[推论]** 冻结一刻分不出生命——从形式直接推出。

- **[推测]** "感受 → 语言是压缩""AI 没有感受端"——概念性论述。

- **[不声称]** 不声称给出压缩的严格信息论定义；不声称解释意识或体验；R 不展开。

> 本文由何淇创作，DeepSeek（AI）协助整理与写作；观点与责任属于作者。

第五篇 · 语言与大脑：两种函数形式

单独 DOI（中文）：https://doi.org/10.5281/zenodo.23667685

摘要

本篇把两个对象放在一起写：人脑函数 B 与语言函数 L，以及学习 L 的执行器（大模型）。

先写 B。脑是可函数化器官：切片（输入→表示）、空间组合（复合函数链）、时间组合（算子序列，步数 k）、判定（选算子/决定停）、自指三层（表示/参数/结构，受固定核心 Φ 约束）。人脑是一个实例：异质、低维高效、时间内生、有判定、三层自指活跃；但它不自足——供养、缺氧、内稳态都靠身体，被保住的是"身体+脑+环境"整体。

再写 L。语言从"器官→感受→符号"的压缩链长出来；它是公共压缩接口（说者编码、听者解码、群体共享）。函数分解：词 = 切片（符号→意义）、句法 = 空间组合律、语义 = 表示映射与接地、篇章 = 时间组合、逻辑 = 组合律的规则（语用 = 判定）。语言没有物理实在，但切片不需要物理实在——它的切片是"符号切片"，物理只在接地那一端出现。语言的时间分三层：结构无时间、使用是序列、演化是慢变。语言 = p语言 + 相对 R语言；言外之意大部分在 p语言（会话隐含、语用规则），小部分在相对 R语言（默契、经历、语气）。

三层的关系是生成链：**B 产生 L，大模型执行 L。** 这决定了研究对象的划分：研究大模型用 L 的变量（可预测性、组合深度、压缩/解压、接地残差）；研究生命用 B 的机制（维持、自指、判定、内循环）。脑机制在 L 上有投影（收敛→可预测性、冗余→多重编码、判定→语言约定），但投影不是原物——把投影当原物，结论就会"滑"。

给出可检验推论与边界。

本篇定义一览

| 术语 | 一句话 |

|---|---|

| 人脑函数 B | 个体脑：世界 → 表示 → 行为（有器官、维持、内循环、判定）；显式形式末尾记 + R_相对 |

| 语言函数 L | 群体脑输出外化的公共结构：符号 → 意义（可传递、可积累、不属单个脑） |

| 执行器 | 学习并执行 L 的系统（大模型）：有符号层与组合规则，没有 B 的器官/维持/内循环 |

| 切片 | 一个函数 f: X → Y，带表示维度 (d_in, d_out) |

| 空间组合 | 复合函数链 F = f_N ∘ … ∘ f_1 |

| 时间组合 | 算子序列 s_{t+1} = I_{σ(t)}(s_t)；步数 k |

| 判定 J | 一次通过：选算子、决定停（派生定义，见基础篇定义 4a） |

| 自指三层 | 表示自指 / 参数自指 / 结构自指 |

| 固定核心 Φ | 不可改的约束集合 |

| 语言 | 符号系统 + 组合规则 + 共享约定：把表示压成可传递的符号序列，并允许解压 |

| 公共压缩接口 | 语言的角色：说者编码、听者解码、群体共享 |

| 符号切片 | 语言的切片：输入是符号、输出是意义表示的映射 |

| 词 = 切片 | 一个词是"符号 → 意义"的映射 |

| 句法 / 语义 / 篇章 / 逻辑 | 空间组合 / 表示与接地 / 时间组合 / 组合律的规则 |

| p语言 / 相对R语言 | 语言中可函数化 / 相对某函数类够不着的部分（不是绝对 R） |

| 言外之意 | 解压时补出来的部分：大部分 p语言，小部分相对 R语言 |

一、问题

我们有两个对象，它们都值得写成函数形式：

1. **人脑**：一个具体的、活着的器官，有器官约束、维持需求、内生时间；

2. **语言**：人类群体大脑输出外化的公共结构，可传递、可积累，本身没有物理实在。

以及一个产物：**大模型**。它学的是语言（L），不是人脑（B）。三者常被混在一起谈，于是出现两类错误：把大模型当脑（用 B 的机制词汇测它，结论"滑"），或把语言当实体（要求它有自己的时间与维持）。本篇把三层分开写清楚，再说明它们的联系与不可互替之处。

二、人脑函数 **B**：显式写法

本篇所有显式函数形式都统一写作：**形式 + R_相对**。

（记号说明：本篇所有显式函数形式都只写 p 侧，即我们的函数语言能写出的部分；形式末尾的 + R_相对表示"我们的函数语言够不着的那部分"——它受限于我们目前的函数类与观察位置，只声明、不展开；绝对 R 是否存在不判断。）

2.1 脑作为可函数化器官（定性）

- 脑有有限表示空间：能区分的方向有限，且受身体与能量约束；

- 脑有组合结构：区域之间接线、投射、共享状态；

- 脑有内生时间：振荡、放电、代谢在走，不需要外部时钟；

- 脑有判定与自指：选、停、改自己。

⇒ 脑落在本篇"可函数化"的一侧（B），但它是被身体维持的器官，不是自足系统。

2.2 切片与表示维度

一个切片即一个函数 f: X → Y，带 (d_in, d_out, Δ)：表示维度是"能区分的方向数"；Δ 是它与相邻切片的组合方式。脑的每个加工阶段都可以看成一个切片（感觉→表征、表征→决策……）。

2.3 空间组合：复合函数链

F_space = f_N ∘ … ∘ f_1

组合律要求类型相容：Y(f_i) ⊆ X(f_{i+1})。脑的空间组合不是固定链，而是可变组合（投射、路由、共享状态）——这是它低维高效的原因之一。

2.4 时间组合：算子序列与步数

s_{t+1} = I_{σ(t)}(s_t)，t = 0, 1, …, k−1

{s_{t+1} = I_{σ(t)}(s_t)} 是可用算子，σ(t) 是第 t 步选哪个，k 是步数。三种情形：恒定（同一算子反复）、每步变（不同函数依次应用）、混合。脑是混合型：既有反复（振荡、迭代），也有切换（不同回路依次工作）。

2.5 判定：选择与停止

J: S → D（有限判定空间）

判定决定两件事：选算子 σ(t) = J_选(s_t)；什么时候停 k = J_停(s_t)。脑的判定是内生的、快速的、"一次通过"的。

2.6 自指：改自己

三层，从弱到强：表示自指（输入含自身表示）、参数自指（θ_{t+1} = U(θ_t, s_t)）、结构自指（C_{t+1} = V(C_t, s_t)）。三层都受固定核心 Φ 约束：θ ∈ Θ_Φ，C ∈ 𝒞_Φ。人脑：可改突触与局部回路；不可改神经元类型、大尺度解剖、代谢约束。

2.7 实例：人脑

- 切片：异质、低维高效；

- 空间组合：可变（投射/路由/共享状态）；

- 时间组合：算子序列，k=0 与 k≥1 混合；

- 判定：有 J（选算子/停/固化）；

- 自指：三层活跃；

- 维持：身体+环境。

2.8 为什么不自足；生命单位是整体

脑约占体重 2%，却消耗约 20% 能量；靠血液持续供氧供糖，靠身体维持体温与内环境；缺氧几分钟不可逆。所以"被保住"的承载单位不是孤立的脑，而是"身体 + 脑 + 环境"这一圈（维持单位）。与第一篇一致：生命判据仍是"哪一族函数被保住"，环境本身不是生命的一部分；这里说的是维持的承载单位，不是定义边界；生命判据仍是基础篇 D1~D5 中的 p生命。

三、语言函数 **L**：可函数化形式

（基础篇定义 15 只给出 L 的符号—意义切片部分 L_0；本篇给出完整构成：L_0 + 句法组合 + 篇章时间组合 + 逻辑规则。）

3.1 起源：从器官到符号

器官是压缩器：眼把光压成视觉表示，耳把声压成听觉表示。最早的语言很可能是"用具体声音指代具体现象"：声音便宜、可远传、可共享。两个声音拼起来产生新意义，组合律出现；反复使用固化成语法；从具体指称里抽出类别与关系——逻辑的雏形。写成形式：初始语言 $L_0 = \{(s_i, r_i)\}$（符号与表示的映射对）＋组合运算 C ⇒ L 是 L_0 与 C 生成的闭包；公共约定让映射与规则在群体中稳定下来。

3.2 语言是什么：公共压缩接口

- 对说者：把内部表示压成符号序列（编码）；

- 对听者：用自己的结构把符号解压回内部表示（解码）；

- 对群体：编码/解码规则被共享、传承、积累（公共方案）。

所以语言不是世界本身，也不是思想本身，是"可传递的压缩包"。

3.3 五部件函数分解

- 词 = 切片：符号 → 意义；

- 句法 = 空间组合律：词与词拼成更大结构；

- 语义 = 表示映射与接地：符号指向什么、能不能接上世界；

- 篇章 = 时间组合：上下文、指代、顺序依赖；

- 逻辑 = 组合律的规则：哪些组合合法、怎么推；

-（语用 = 判定：场合、得体、真假。）

3.4 语言没有物理实在，切片怎么用？

切片本来就是函数，不是实物；语言的切片只是把输入空间换成"符号空间"。两层：

层一：符号（离散、可数）——切片在符号层上

层二：意义（表示）——切片输出落在表示空间

物理世界在层二之外，只通过"接地"这一端连进来

所以：语言的可函数化形式 = 符号层切片 + 组合律 + 时间序列；物理实在只出现在接地那一端——符号要指到世界上，就要求一个"符号-世界"对应，这就是语义残差（相对 R）的来源。

3.5 语言的时间

- 语言结构：静态压缩方案，不自己走，没有内生时钟（词典没有时间）；

- 语言的使用：按时间展开的序列（s_{t+1}=I(s_t)），篇章/指代/顺序依赖都是时间组合；这是外供时间；

- 语言的演化：词义漂移、语法变化、新概念出现——结构 C 在历史时间上的慢变。

时态、时间词、叙事顺序是语言"编码"的时间，不是语言自身的时间。

3.6 语言作为公共压缩方案

词汇 = 范畴（把连续感受切成离散的类）；语法 = 组合律；逻辑 = 组合律的规则；共享约定 = 稳定性。语言描述世界的能力 = 这套方案的完备程度；方案可以积累（文化、科学、书面语——把时间压成结构）。

3.7 可函数化的层次：**p**语言 / 相对**R**语言

- 形式（符号串）：可函数化；

- 句法（组合）：可函数化；

- 语义（表示）：相对某函数类/观察者部分可函数化（接地依赖模态；见第零篇补充节）；

- 语用判断（场合、得体）：大部分不可函数化；其中可规则化的一部分（会话隐含模式、反讽套路）属于 p语言。

对应的划分是 p语言 + 相对 R语言（相对我们的函数语言与有限观察者；绝对 R 不判断）。

顺带一个常被问到的词：**言外之意**。它是"解压时补出来的部分"：说者把整体感受压成字面，听者用自己的结构补出没写全的意义。大部分落在 p语言（会话隐含、语用规则、反讽模式），小部分落在相对R语言（共同经历、默契、语气）。

3.8 语言能自指

语言可以描述语言（语法书、词典、本篇）。自指带来两个后果：语言可以自我扩展（造新词、新语法）；语言可以描述"描述"本身（元语言）。但自指只到有限层：任何一次描述都是局部的。

四、对象层次：**B → L →** 执行器

三层不是并列，是生成链：

- **B（脑函数）**：世界 → 表示 → 行为；

- **L（语言函数）**：由多个 B 的输出外化成的公共结构；

- **执行器（大模型）**：学习并执行 L。

由此得到研究对象的划分：

- 研究大模型：用 L 的变量（可预测性、组合深度、压缩/解压、接地残差）；

- 研究生命：用 B 的机制（维持、自指、判定、内循环）。

脑机制在 L 上有投影：收敛 → 语言可预测性；冗余 → 公共结构的多重编码；判定 → 语言约定。但投影不是原物——把投影当原物，结论就会"滑"。

（执行器的均匀计算与"脑 vs 执行器"的对照，涉及对脑的主观理解，不放在本篇；另见笔记《执行器的均匀计算》。）

五、语言层与执行器

执行器学的正是 L 这套公共压缩方案的成品：

- 它拿到了符号层及其组合规则（p语言），所以会语言、会推理（逻辑 = 组合律）；

- 它没有"器官→感受→符号"的接地过程——没有身体、没有器官、没有声音-现象的对应；

- 因此它的语义是"符号对符号"的对应，不是"符号对世界"的对应。

这解释了三件事：

1. **会语言 ≠ 有语言能力**：它复现的是方案，不是方案长出来的过程；

2. **语义残差**：接不上的部分不是 bug，是它只有符号侧的必然；

3. **幻觉**：没有世界侧的校正，符号组合可以自洽而错误。

言外之意同理：可规则化的部分（会话隐含、反讽）它学得到；靠默契与经历的部分（相对 R语言）它学不到。

六、可检验推论

1. 冻结一刻分不出生命：单时刻检查不能判定生命，必须跨时间观察。

2. 脑的按需收敛应与能量约束相关：约束越紧，越需要按需。（注：第二十三篇的自由门控在 λ=0 时平均仍只用 5.39 层，与此方向相反，机制待查；本推论标为待检验。）

3. 要让执行器按 token 自适应深度，需要"组合律（层间路由/层内稀疏）+ 判定 + 代价"三者同时存在；只加代价只能整体浅化（可训练对照实验检验）。

4. 句法复杂度应映射到计算步数：依赖越深，所需层数/步数越多。

5. 语义残差应由接地维度决定：补上缺失维度则残差降低，冗余模态不降。

6. 言外之意中可规则化的部分（p语言）应可被模型学到；靠默契的部分（相对 R语言）应学不到——可用反讽/暗示任务检验。

七、局限

第一，R 只声明、不展开：只写 P 侧；R语言/相对 R 的边界未操作化。

第二，语言结构没有严格定义：用的是"词/句法/语义/篇章/逻辑"的常识划分，没有形式语法或语义学的严格框架。

第三，B 与 L 的对应只有概念层：哪部分脑结构对应哪部分语言功能，没有可测映射。

第四，起源部分是推测性的：从器官到符号的演化过程无法直接验证。

第五，执行器部分的对照（均匀计算、按需收敛）已拆出本篇，另见笔记；本篇不含相关推断。

第六，本篇是形式整理，不是新数学；对人脑与语言的理解有限，不声称超过神经语言学与语言学的既有工作。

附：标注

- **[定义]** B 与 L 的函数形式、五部件分解、p语言/相对R语言、言外之意、对象层次（生成链）。

- **[整理]** 合并原论文-21《人脑的可函数化形式》（五项形式、Φ、自指三层、生命单位是整体；其中"均匀计算"内容已拆出至笔记《执行器的均匀计算》，并标注为对脑的主观理解）与原论文-35《语言结构的可函数化形式》（起源、公共压缩接口、五部件、符号切片、语言的时间、可函数化层次、自指）。

- **[推测]** 语言起源叙事；执行器"代价+组合律+判定"三者缺一不可——方向性推测。

- **[不声称]** 不声称给出形式语法；不声称解释意识或体验；R 不展开。

> 本文由何淇创作，DeepSeek（AI）协助整理与写作；观点与责任属于作者。

附录 **E** 可表示维度的空间面与时间面

附录 **E** · 可表示维度的空间面与时间面

一、为什么要分两种可表示维度

之前讨论"细菌的可表示维度"时出现过混乱：一会儿说它的输出只有一两维（鞭毛马达），一会儿又说它的行为很丰富。两种说法都对，但量的是不同的东西。

根子在于：**讨论"输入/输出"时，我们其实已经假定了时间。** 一个瞬间的对象没有输入输出，只有状态；只有在一段时间里，才谈得上"刺激进来、响应出去"。

所以要把"可表示维度"分成两面：

- **空间面**：一个瞬间的映射，输入空间 → 输出空间，量的是"同时能区分多少个方向"；

- **时间面**：一段时间窗口内的映射，输入轨迹 → 输出轨迹，量的是"能产生多少种可区分的轨迹"。

二、空间可表示维度

· 切片 f: X → Y（瞬时）

· d_in = 能独立施加的输入方向数

· d_out = 能独立产生的输出方向数

· 可表示维度 = 响应映射的秩（超过噪声的方向数）

例：

- 铁原子：几维（平动、自旋、电子态）；

- 小分子：十几到几十维（平动+转动+振动模式+电/磁响应+反应通道）；

- 细菌鞭毛马达：1~2 维（转向 + 转速）；

- 细菌行为策略：几十维（跑的长度、翻的频率、条件依赖）。

同一种生物，切片切在"执行器"还是"行为策略"，空间维度完全不同。

三、时间可表示维度

· 固定窗口 τ

· 输入：窗口内的刺激轨迹 u(t), t ∈ [0, τ]

· 输出：窗口内的响应轨迹 y(t)

· 切片 = "输入轨迹 → 输出轨迹"的映射

· 可表示维度 D(τ) = 这个映射的响应秩

· d_in(τ)：能独立变化且产生可分辨输出的输入方向数

· d_out(τ)：能被分辨的输出方向数

· d_mem(τ)：能区分的"历史长度"（多久以前的刺激还能影响当前响应）

四、两个极限：**τ→0** 与 **τ→∞**

τ → 0：可表示维度退化为状态维度。

· 窗口收缩成一个点：没有轨迹，只剩一个状态

· 输入输出映射退化为恒等/常数，响应秩塌缩

· 剩下的只有：描述这个瞬间需要多少个独立量

⇒ D(0) = 状态维度

这正是"冻结一刻"的数学形式：冻结时函数消失，只剩状态；生命与非生命在状态描述上同形，所以生命必须在时间上辨认。

τ 增大：D(τ) 先升后饱和。

· τ 很小：只有最快的过程被触发，维度低

· τ 增大：更多过程被纳入，维度上升

· τ 很大：该有的过程都出现了，趋于饱和

五、短窗口与长窗口的语义不同

窗口长度改变时，两件事同时变化，必须分开说：

① 时间分辨率：

· 窗口越短，越能分辨快速变化（时间上更细）

· 窗口越长，细节被平均（时间上更粗）

② 功能复杂度（计算厚度）：

· 窗口越长，能容纳的步骤/过程越多

· 一个长窗口里的行为是多步组合的产物（功能上更复杂）

所以：

· 短 τ：时间分辨细，但能触发的过程少，维度可能不高

· 长 τ：时间上被平均，但功能组合更多，维度可能更高

单独说"某生物的可表示维度是多少"没有意义，必须带上 τ。

六、细胞层面的测量草案

固定 τ、输入空间 U、输出空间 Y，然后分别测：

d_in(τ)：

· 逐个改变刺激通道，看响应是否超过噪声

· 能分辨的方向数 = d_in

d_out(τ)：

· 数可区分的行为/生理模式（运动、分泌、表达切换……）

· 需明确"同一响应"与"不同响应"的分辨率

d_mem(τ)：

· 用脉冲刺激测响应持续时间

· 或测"多久前的刺激还能改变当前行为"

d_int(τ)：

· 对可测内部变量做 PCA / 有效秩

· 或用内部状态预测行为的维度估计

要求：

- 低于噪声底的方向不计入；

- 报告"维度 vs τ"曲线，而不是单个数字；

- 窗口内混入多个过程时，要说明是哪个时间尺度。

七、与生命判据的关系

· 空间维度：说明"通道多宽"

· 时间维度：说明"能不能活、能不能适应"

· 生命判据（D1~D5）落在时间面上：

· "会变化" = 时间维度丰富

· "会自指" = 规则随时间改

· "被保住" = 轨迹在可行域里持续有定义

⇒ 高空间维度的石头不是生命；

时间维度非零且被保住的函数族才是

八、还没想清

· d_mem 的严格定义：用互信息？用响应衰减时间？

· 多个时间尺度并存时，D(τ) 曲线怎么分解成各尺度的贡献？

· 随机性（细胞噪声）下，"响应秩"要不要用信息量代替？

· 跨尺度比较：分子、细胞、器官、个体能用同一套 D(τ) 吗？

· 这套定义能否落到可做的实验上（我们手头有什么数据/仿真）？

> 一句话：可表示维度有两个面——空间面量"通道多宽"，时间面量"轨迹多丰富"；τ→0 时前者退化为状态维度，τ 变化时分辨率与功能复杂度朝相反方向走。

附录 **B** 概念地图（导读）

附录 **B** · 概念地图与术语总表

单独 DOI（中文）：https://doi.org/10.5281/zenodo.23609033

一、核心定义

| 概念 | 一句话定义 | 出处 |

|---|---|---|

| 宇宙 | P + R；P 可函数化，R 不可函数化（只声明，不展开） | 第零篇 |

| 可函数化 | 能用我们的函数语言写到误差任意小 | 第零篇 |

| p生命 | 生命落在 P 里的部分；定义与讨论的对象 | 第零/一篇 |

| R生命 | 生命落在 R 里的部分；只声明，不展开 | 第零篇 |

| 生命 | p生命 + R生命；p生命 = 一族被保住的动态函数（官方定义 D1~D5，见基础篇定义 14） | 第零/一/基础篇 |

| R_相对（记号） | 显式函数形式末尾的标记：我们的函数语言够不着的那部分；只声明、不展开；绝对 R 不判断 | 第一/四/五篇 |

| 绝对 R | 宇宙自身不可函数化的部分；是否存在不判断 | 第零篇补充 |

| 相对 R | 对任何有限观察者非空（涨落、有限资源、自指、不可判定等） | 第零篇补充 |

| 数字世界 | 形式对象：状态为有限串、规则为算法；绝对 R = ∅（构造保证） | 第零/一篇 |

| 残差 | P 侧可测的误差下限（P 的边界形状），**不是** R 的影响或投影 | 第零篇记号 |

| 可函数化 ≠ 可计算 ≠ 可测量 | 三件事分开：存在映射 / 能算 / 能观测复现 | 第零篇 |

二、函数语言（第四篇）

| 概念 | 一句话定义 | 出处 |

|---|---|---|

| 切片 | 一个函数 f: X → Y，带 (d_in, d_out, Δ) | 第四篇 |

| 表示维度 | d_in = dim(X)、d_out = dim(Y)；切片自己的轴数 | 第四篇 |

| 有效维度 | 可区分状态数的对数；轴数不变也可能下降（每根轴变粗） | 第三篇 |

| 空间组合 | 复合函数链 F = f_N ∘ … ∘ f_1；组合律要求类型相容 | 第四篇 |

| 时间组合 | 算子序列 s_{t+1} = I_{σ(t)}(s_t)；三种情形：恒定/每步变/混合 | 第四篇 |

| 步数 k（计算厚度） | 时间组合沿路走了多少步 | 第四篇 |

| 判定 J | 一次通过，不是迭代：J_选（选算子）+ J_停（决定停） | 第四篇 |

| 自指三层 | 表示自指 / 参数自指 / 结构自指，从弱到强 | 第四篇 |

| 固定核心 Φ | 不可改的约束集合；自指的三层都受它约束 | 第四篇 |

| 组合律 | 切片之间允许怎样相互作用（顺序/共享状态/并行/路由/共享参数） | 第四篇 |

| 组合律的定义域 | 让整条切片链"跑得起来"的输入集合 = 各切片定义域的相容交集 | 第四篇 |

| 被保住（函数版） | 组合律在整个时间段内始终有定义 | 第四篇 |

| 结构 | 结构 = 分解 D + 组合 C；它同时是一套压缩方案（第一篇 3.3 的"结构"指宇宙的局部呈现，是环境视角） | 第四篇 |

| 分解 D / 组合 C | 把函数映射成一组切片 / 把一组切片拼回函数 | 第四篇 |

| 时间驱动来源 | 内生（时钟长在系统自身基底里，生物）/ 外供（外部计时器+解释器，数字世界） | 第四篇 |

| 类型 A/B/C/D | 连续流 / 采样动力学 / 自动机 / 混合型 | 第四篇 |

| 五根轴 | 空间连续/离散、时间连续/离散、时间内生/外供、组合律自由度、自指层级 | 第四篇 |

三、三种身份：宇宙/环境/生命（第四篇）

| 概念 | 一句话定义 | 出处 |

|---|---|---|

| 宇宙切片 | $U_t = (S_t, C_s, d_t)$；时间组合在切片之间（$T_{comb}: U_t \to U_{t+1}$） | 第四篇 |

| 空间组合（宇宙） | 同一时刻内部，部分怎么拼成整体（C_s） | 第四篇 |

| 时间组合（宇宙） | 切片之间的接法；一个瞬间只有空间结构 | 第四篇 |

| 环境 | 宇宙的子切片：$d^E \subseteq d$、$C_s^E \subseteq C_s$、状态是投影 $\pi_E(S_t)$；无判定、无自指 | 第四篇 |

| 环境的时间 | T_{comb} 的限制 + 边界输入 u_t | 第四篇 |

| 生命（四条件） | 被保住（轨迹在 V 内）/ 边界开放 / 自指调节 / 可塑（受 Φ 约束） | 第四篇 |

| 冻结一刻 | 生命与环境同形，分不出来；生命必须在时间上辨认 | 第四篇 |

| 生命 = 被保住的轨迹 | 不是某个状态；是"一段轨迹"的属性 | 第四篇 |

四、表示与维度（第三/四篇）

| 概念 | 一句话定义 | 出处 |

|---|---|---|

| 表示空间 | 所有可能表示/状态组成的空间 | 第四篇 |

| 状态 | 表示空间里的一个点（各表示维度坐标的元组），不是维度 | 第四篇 |

| 定义域/值域 | 沿某些表示维度的允许/可达范围（子集），不是轴 | 第四篇 |

| 宽度 | 一层里的表示维度个数 | 第四篇 |

| 深度 | 步数/层数，与表示维度不同的另一根轴 | 第四篇 |

| 升维 | 增加可区分的表示维度 | 第十二篇 |

| 三个空间 | 表示空间（激活）/ 参数空间（权重）/ 输入输出空间（数据） | 第四篇 |

| 厚度（总称） | 分布厚度（垂直）与计算厚度（水平），正交 | 第四篇 |

| 分布厚度 | 同一输入对应多宽的一族输出（熵/歧义）；用表示维度摊平 | 第四篇 |

| 计算厚度 | 沿组合律从输入到目标需多少步（路长）；用步数/时间摊开 | 第四篇 |

| 可摊薄 / 不可摊薄 | 升维能把厚度摊平 / 摊不平；结构厚度 = 维度耦合导致拆不开 | 第四篇 |

| 相对残差 | 厚度 / 可区分尺度；维度降低时相对残差上升 | 第四篇 |

| 范围收缩 | 每根表示维度上的可输入/可输出范围变小 | 第三篇 |

| 三资源 | 空间上的表示维度 / 组合律上的表示维度 / 时间上的步数 | 第二十二篇 |

五、压缩与解压（第四篇）

| 概念 | 一句话定义 | 出处 |

|---|---|---|

| 压缩 | 降维映射 $f: X \to Y$，$\dim(Y) < \dim(X)$ | 第四篇 |

| 解压 | 从压缩表示重新展开（不唯一；不必等于原像） | 第四篇 |

| 解压的"够用" | 解压不必严格可逆，只需对维持够用；哪些维度必须可逆由环境选择 | 第四篇 |

| 窄进宽出 | 输入受限且被粗粒化；输出靠结构展开，多出的维度来自结构（过去的压缩） | 第四篇 |

| 身围 | 个体看到的环境 = 压缩映射的投影 $\pi_L(E)$ | 第四篇 |

| 压缩方案 | "保留哪些区分、合并哪些、把哪些过去压进结构"的规则，写在结构里 | 第四篇 |

| 一次性解压 | 冻结模型训练完即定型；不再更新、不再维持 | 第零/一篇 |

| 语言 = 公共压缩方案 | 感受→语言是压缩；语言→描述/理解/推理是解压；逻辑 = 语言的组合律 | 第四篇 |

| 闭合环 | 第 t 步 $(s_t,in_t)\to(s_\{t+1\},out_t)$；$in_\{t+1\} = W(out_t, 世界)$ | 第四篇 |

| 半闭环 | 大模型：输出只延续文本，不改变环境 | 第四篇 |

| 接口层 / 机制层 | 输入输出行为 / 内部怎么算、怎么维持、怎么改自己 | 第四篇 |

| 语义残差 | 语言无法承载的语义缺口（给定模态组合下的最小可达误差） | 第十二篇 |

| 接地 | 用多模态/行动把语义残差补上 | 第十二篇 |

六、能力、收敛与效率（第四/五篇 + 实验）

| 概念 | 一句话定义 | 出处 |

|---|---|---|

| 按需收敛 | 试探→判断收敛了吗→没收敛带记忆循环→收敛后固化 | 已拆出至笔记《执行器的均匀计算》，非系列结论 |

| 判定器（halt/exit judge） | 每层检查"够了没有"，决定退出/继续/走哪条路 | 第二十一/二十二篇 |

| 快慢结构 | 快结构（事实，一两层收敛、冻结成前提）+ 慢结构（权衡，时间上循环） | 讨论稿（未成篇；相关实验见第二十二篇） |

| 三处均匀 | 深度均匀 / 宽度均匀 / 没有按需循环 | 已拆出至笔记《执行器的均匀计算》，非系列结论 |

| 均匀深度浪费 | 不同结构对深度的需求差别很大；按需分配更省（路由 47%） | 第五篇（原论文-28） |

| 过度计算 | 超过任务厚度后误差回升（按需动员 0.434 vs 均匀 1.756） | 第五篇（原论文-28） |

| 层 ≈ 步数 | 冻结模型的深度充当时间步（伪时间） | 第五篇 |

| 时间换维度 | 给循环步数（时间）可大幅降低所需表示维度 | 第一篇/实验 |

| 循环省参数 | 同一迭代效果，循环比加层省约 27 倍参数 | 第五篇（原论文-28） |

| 层厚放置 | 同样参数下，前窄后宽（宽在后）更省 | 实验 |

| 最小切片 | 让一个结构出现的最小（宽度, 深度）配置 | 实验 |

| 能力边界 | 达到精度的配置区域；是"可训练性边界"而非纯表示边界 | 实验 |

| 涌现 | 能力/结构跨过某个维度阈值后出现（相变，即分段不连续点） | 第零篇/实验 |

| 固定核心 Φ | 不可改的约束；可塑只能在 Φ 内 | 第四篇 |

七、语言函数与语言结构（第五篇·语言与大脑）

| 概念 | 一句话定义 | 出处 |

|---|---|---|

| 语言 | 符号系统 + 组合规则 + 共享约定：把内部表示压成可传递的符号序列，并允许解压 | 第五篇 |

| 公共压缩接口 | 语言的角色：说者编码、听者解码、群体共享 | 第五篇 |

| 人脑函数 B | 个体脑：世界 → 表示 → 行为（有器官、维持、内循环、判定） | 第五篇 |

| 语言函数 L | 群体脑输出外化的公共结构：符号 → 意义（不属单个脑） | 第五篇 |

| 大模型（执行器） | 学习并执行 L 的系统，不是 B 的实现 | 第五篇 |

| 符号切片 | 语言的切片：输入是符号、输出是意义表示的映射 | 第五篇 |

| 词 = 切片 | 词是"符号 → 意义"的映射 | 第五篇 |

| 句法 = 空间组合 | 词与词拼成更大结构（组合律相容条件） | 第五篇 |

| 语义 = 表示与接地 | 符号指向什么、能不能接上世界 | 第五篇 |

| 篇章 = 时间组合 | 上下文、指代、顺序依赖在时间上展开 | 第五篇 |

| 逻辑 = 组合律的规则 | 哪些组合合法、怎么推 | 第五篇 |

| 语用 = 判定 | 场合、得体、真假（一次通过） | 第五篇 |

| p语言 / 相对R语言 | 语言中可函数化 / 相对某函数类够不着的部分（不是绝对 R） | 第五篇 |

| 言外之意 | 解压时补出来的部分：大部分 p语言（会话隐含/语用规则），小部分相对 R语言（默契/经历/语气） | 第五篇 |

| 语言的时间（三种） | 结构无时间；使用是时间序列；演化是结构的慢变 | 第五篇 |

| 隐喻/编码的时间 | 时态、时间词、叙事顺序：语言"关于时间"的表示，不是语言自身的时间 | 第五篇 |

八、层路径、判定与早退（第二十三篇、第二十四篇）

| 概念 | 一句话定义 | 出处 |

|---|---|---|

| 层冗余 | 同一结构在多层里有可替代的副本：单层跳过影响小、成组跳过才崩 | 第二十三篇 |

| 单层跳过 | 某层输出 = 输入（恒等直通），只去掉该层贡献 | 第二十三篇 |

| 冗余量 | 成组跳过的翻转率 − 两个单层中较大的那个（正 = 有补偿/冗余） | 第二十三篇 |

| 后层重建/纠正 | 破坏当前位置状态后最终预测仍能恢复：后层从上下文重算 | 第二十三篇 |

| 语言可预测性 | 用完整模型预测的熵做代理：低熵 = 语言结构强、可预测 | 第二十四篇 |

| 语言结构作为自变量 | 用可预测性分组替代"任意 token 类别"分组 | 第二十四篇 |

| 判定器 A（置信度） | 用当前层预测的置信度/熵决定退出 | 第二十三篇 |

| 判定器 B（路径） | 用跨层一致性（相邻层预测一致、KL、趋势）决定退出 | 第二十三篇 |

| 判定器 L（语言） | 用语言结构特征（词类、位置、重复、首层熵）决定退出 | 第二十四篇 |

| oracle 早退 | 假设知道哪层预测正确就退出：早退的上界 | 第二十三篇 |

| 早退 + 写回 | 早退结果写进上下文再继续：单次可吸收，连续会累积误差 | 第二十三篇 |

| 过度计算有害 | 早退反而比完整模型准确率更高：后面的层会改坏对的预测 | 第二十三篇 |

九、实验读数（**GPT-2 /** 玩具）

| 概念 | 一句话定义 | 出处 |

|---|---|---|

| 层间 Δ_l | 相邻层 hidden 的相对变化；GPT-2 上不单调、末端变大 | 真实模型实验（见第二十一/二十二篇） |

| 校准探针 | 用线性/MLP 映射把 h_l 映射到 h_final 再解码，检验"早退可不可行" | 真实模型实验（见第二十一/二十二篇） |

| halt 判定器 | 训练分类器预测"该层探针是否等于最终预测"，实现版早退 | 真实模型实验（见第二十一/二十二篇） |

| 早退上限 | oracle 判定下平均深度 2.31/12（省 81%） | 真实模型实验（见第二十一/二十二篇） |

| 可实现早退 | 训练 halt 后省 72~80% 深度，准确率约 -1.6~-1.8 个点 | 真实模型实验（见第二十一/二十二篇） |

| 快慢玩具实验 | 快编码+慢循环 vs 平铺 MLP：精度相当、参数约省 2.2 倍 | 讨论稿（未成篇；相关实验见第二十二篇） |

十、待定义 / 待统一（下一步）

- **维持（可量化）**：寿命、恢复时间、临界扰动、维持成本——暂不展开

- **"够用"判据**：解压的可逆性要求依赖环境，尚未操作化

- **可行域 V 与固定核心 Φ**：判断"是否被保住"与"什么不可改"还缺测量协议

- **判定器的训练信号**：用什么标签训练 halt（一致性/影响/代价），未定

- **语言结构与脑结构的对应**：B 与 L 的映射只有概念层，没有可测的对应

- **言外之意的边界**：哪些靠 p语言 规则、哪些靠相对 R语言，未操作化

- **R 的位置**：R 不参与连续/离散、定义域/值域讨论（范畴问题），但要防止读者误读

依赖图（谁由谁定义）

P / R → 可函数化 → p生命 → 五条件 D1~D5（在 P 中 / 一族 / 会变化 / 会自指 / 被保住）

↓

数字世界（R=∅）→ 冻结模型 → 一次性解压 → 失配

↓

结构 (D, C) → 切片 → 组合律 → 组合律定义域 = 可维持集合

↓

表示空间 → 表示维度 → 状态 / 定义域 / 值域

↓

厚度 → 可摊薄/结构厚度 → 升维 → 相对残差

↓

时间（内生/外供）→ A/B/C/D → 层≈步数 → 时间换维度

↓

能力边界（最小切片）→ 涌现（阈值/相变）

↓

【语言分支】公共压缩方案 → 语言函数 L → 大模型（执行器）

→ 语言结构强度（可预测性）→ 层冗余/早退/判定

→ 言外之意（p语言 / 相对R语言）

↓

【第四篇主线】函数语言 → 三种身份（宇宙/环境/生命）→ 压缩/解压

→ 窄进宽出 → 身围 → 闭环节奏 → 语言（公共压缩方案）

→ 判定器/按需收敛 → 三处均匀 → 循环省参数

附：门控观察（第二十五篇）

| 术语 | 一句话 | 出处 |

|---|---|---|

| 旁路门 | 门开 = 放行（不变换），门关 = 计算（正常变换） | 第二十五篇 |

| 硬预算 | 每 token 最多算 k 层；最优退化为固定前 k 层 | 第二十五篇 |

| 软代价 | 损失 += λ·平均计算层数；直接加梯度会崩 | 第二十五篇 |

| 拉格朗日门控 | λ 自适应压平均层数；可控且按 token 分化 | 第二十五篇 |

| 深度冗余 | 6 层任务 2~3 层、24 层任务约 5.3 层即可持平（噪声内） | 第二十五篇 |

第三部分 测量与实验

第六篇 · 容量不能替代数据的更新

单独 DOI（中文）：https://doi.org/10.5281/zenodo.23503049

语言升维下的失配测量（**scaling vs** 持续学习：一个弱结论）

> 论文系列 · 第二章

> **DOI: https://doi.org/10.5281/zenodo.23503050**

> 版本：v2（2026-10-22 增补「维度与贴合」一节，见第四·五节）

> 第一章：《现在的 AI 为什么不是生命》（DOI: https://doi.org/10.5281/zenodo.23488543）

摘要

第一章提出：生命是「被维持的解压」，AI 是「一次性的解压」，并预测冻结模型会因语言升维出现**相对失配**。

本章给出测量，并**在过程中自我修正了两次方法错误**：

（1）先用「旧模型 vs 纯新模型」测，发现正反向幅度接近（+0.108 vs +0.128）⇒ 对称差异，测不出"落后"；

（2）改用**累积参照系**（旧模型 vs 懂全部年份的模型），失配随截止年变晚单调下降

（+1.026 → +0.721 → +0.322 bits/token），但对照也显著为正 ⇒ 混入「数据量」效应；

（3）最后用**等数据预算对照**（三个模型训练数据量相同，只差数据新旧），分离出概念新鲜度的独立效应：

只用最近数据的模型在 2024 测试集上比只用最早数据的模型好 **0.405 bits/token**（同样数据量），

而混合新旧数据的模型在旧测试集上几乎不损失（13.655 vs 13.659）。

并进一步做了**容量补偿实验**：把模型容量从 2 阶提到 6 阶（带插值 n-gram），

旧数据与混合数据的差距稳定在 **+0.86~0.94 bits/token**，未见缩小

⇒ 在本设置下**未见容量补偿效应**（弱结论：不构成对 scaling 的否定，需 transformer 复核）。

结论：**语言是累积升维的 —— 加新概念不丢旧概念。因此正确的更新是「累积/混合」而不是「替换」：

替换会遗忘（只用新数据时旧语料 13.800 vs 13.659），冻结则停在训练时刻。

持续更新的最优策略 = 混合新数据、保留旧能力。** 全部脚本可复现。

一、问题

若语言随时间增加新的「维度」（新概念、新用法）

⇒ 冻结在训练时刻的模型，对这些新维度没有行为谱

问题：

① 这个差距有多大？

② 它来自「概念新旧」还是「数据多少」？

③ 加模型容量（参数）能不能补偿？

④ 正确的更新方式是什么：替换还是累积？

二、数据与方法

数据：arXiv cs.AI，2018 / 2020 / 2022 / 2024 各 3000 篇（标题+摘要），共 12000 篇。

抓取：`502_fetch_arxiv.py`。

两次自我修正（方法）

初始（504）：旧模型 vs 纯新模型 —— 正反向对称 ⇒ 错误，测不出落后

修正一（505）：累积参照系 —— 方向对了，但有数据量混淆

修正二（506）：等数据预算 —— 三个模型数据量相同，只差新旧

容量实验（507/508）

固定数据量 N=1800 篇；把 n-gram 阶数 2→6 作为容量代理

508 用 Jelinek-Mercer 插值，使高阶更稳

比较 OLD / MIX / REC 三组在 2024 留出集上的 bits/token

三、结果

3.1 升维速率的词层面代理（`**501_vl_newterm.py**`）

2018 vs 2024（6 年）：新词占比 **7.30%**（≈ **1.2%/年**），新概念 7787 个。

新概念 Top：large language, llms, prompting, rag, generative ai,

vision-language models, retrieval-augmented, gpt-4o 等。

3.2 等数据预算对照（核心结果，`**506**`）

bits/token（三者数据量完全相同）：

| 模型（数据量相同） | 2024 测试集 | 2018 测试集 |

|---|---|---|

| A 只用最早年份（2018） | 14.026 | 13.659 |

| B 混合全部年份 | 13.742 | 13.655 |

| C 只用最近年份（2024） | **13.621** | 13.800 |

① C 比 A 在 2024 测试集好 0.405 bits/token（数据量相同）

⇒ 概念新鲜度有**独立价值**

② B（混合）新测试 13.742，旧测试 13.655 ≈ A 的 13.659

⇒ 混入新数据：新数据变好，旧能力不丢 ⇒ **更新有净收益**

③ C 在旧测试集变差（13.800）

⇒ 只用新数据会遗忘旧概念 ⇒ **替换有代价**

3.3 容量补偿实验（**`508`**，带插值）

最大阶 OLD MIX REC OLD−MIX

2 10.712 9.774 9.505 +0.938

3 10.946 10.059 9.808 +0.888

4 11.011 10.143 9.901 +0.868

5 11.029 10.169 9.931 +0.860

6 11.035 10.178 9.942 +0.857

① OLD−MIX 差距几乎不变（0.938 → 0.857）

⇒ 加大容量**没有补偿**「数据太旧」

⇒ 弱结论：本实验设置下未见容量补偿（不构成对 scaling 的否定）

② REC 在新语料上最好（9.94），但旧语料会变差（见 3.2）

3.4 累积参照系（`**505**`，作为方法展示）

| 截止年 | M（新测试） | M_ctl（旧测试） |

|---|---|---|

| 2018 | +1.026 | +0.808 |

| 2020 | +0.721 | +0.459 |

| 2022 | +0.322 | +0.151 |

M 单调下降（知道得越多，落后越少）；

但 M_ctl 也为正 ⇒ 数据量混淆 ⇒ 必须用 3.2 的等预算对照才能分离。

四、结论

① 语言升维真实存在：6 年跨度，概念新鲜度的独立价值约 0.4 bits/token

② 但很慢（词层面约 1.2%/年）；冻结模型的失配是缓慢累积的，不是断崖

③ 加容量补不了数据旧（**弱成立**，见下方三档）

④ 语言是**累积**升维的：加新概念不丢旧概念

⇒ 正确的更新是「累积/混合」，而不是「替换」

⇒ 冻结 = 一次性解压；替换 = 单维度解压；累积 = 维持解压

—— 最后这一条，正好对应第一章「生命 = 被维持的解压」的条件

结论强度三档（本文只主张第二档）：

强：参数不能替代新数据 —— 需要 transformer 级证据，本文未达到

弱：在本实验设置下未见容量补偿 —— 本文主张的就是这一档

零：scaling 无效 —— 本文没有任何证据支持，不主张

「弱结论」代表什么（不是文字游戏，要写清楚）

它代表：数据更新与模型扩容是两个**不同的杠杆**，在本设置下不能互相替代。

· 不代表「scaling 无效」——scaling 在固定数据分布内仍然有效

· 不代表「参数越多越坏」——只表示多出来的参数换不来对陈旧数据的补救

· 代表：失配是「数据面」的问题，不是「参数面」的问题

⇒ 若语言持续累积升维，正确对策是**更新数据（混合）**，

而不是只把模型做大

现实含义：

只扩模型不更新数据的策略，会在新语料上持续落后约 0.4~0.9 bits/token

科学含义：

「持续学习」不能简化为「把参数加够」——

维度（概念）的增长需要新数据来承载，参数只是接收器。

边界（再次声明）：

以上是 n-gram 容量代理下的**弱结论**；transformer 级验证前，

不能推广为「越大越没用」。

四·五、维度与贴合：一个补充讨论 [推测]

> 本节为 v2 新增，是**推测性讨论**，不改变前文任何实证结论；其中「有效维度」尚未测量。

一个经验事实：**某些小模型在特定领域强于更大的模型。** 如果能力只由「总维度（规模）」决定，这不该发生。这说明需要区分两个量：

覆盖（总维度）：模型能表示的结构总量 —— 由规模/容量决定

贴合（有效维度）：这些维度中，真正对准目标结构的部分 —— 由数据与训练分布决定

由此可以同时解释三个现象：

1. **升维仍持续带来能力提升**：覆盖在变广，更多结构类「够得着」⇒ 平滑上涨。

2. **能力会「突然出现」**：具体能力对应「某个结构是否被覆盖到」，是阈值式的（0/1），所以表现为跳变，而非渐变。

3. **小模型在窄领域反超**：它的覆盖小，但可以把维度集中对准该领域的少数关键结构 ⇒ 局部贴合度更高。

回到本文的结论：**容量是接收器，数据决定方向。** 加大容量能扩大覆盖，但不能把维度「对准」新数据带来的新结构；这正是「容量不能替代数据更新」的另一种说法——数据决定有效维度，容量只决定这些维度能展开到多大。

可检验的预测（尚未做）：若能用表征方法测量「有效维度」，则当某任务所需结构维度 k 大于模型的覆盖维度时，该能力不应出现；一旦覆盖维度越过 k，能力应阈值式出现。这需要一个可操作的有效维度度量——这是下一章留下的问题。

五、限制（必须与结论一起读）

· 仅一个领域（cs.AI）、一个来源（arXiv 摘要）

· n-gram 是极简语言模型：测的是词级概念失配，不代表大模型语义/推理层面的失配

· 容量代理 = n-gram 阶数，不是真正的参数量；高阶 CE 仍偏大（插值 λ 固定）

⇒ 容量结论只能算初步证据，需 transformer 小模型复核

· 等预算控制的是训练文档数（非 token 数）；采样只用一个随机种子

六、可复现

python 502_fetch_arxiv.py cs.AI 2018 2024 # 及 2020/2022

python 501_vl_newterm.py <语料根> 6 # 新概念率

python 504_series.py <2018> <2020> <2022> <2024> # 对称性检验（展示错误）

python 505_cumulative_mismatch.py <四年目录> # 累积参照系

python 506_equal_budget.py <四年目录> # 等预算对照（核心）

python 507_capacity_vs_data.py <四年目录> # 容量初探（无回退）

python 508_backoff_capacity.py <四年目录> # 容量补偿（带插值）

作者：何淇（独立研究者）；AI 协作：DeepSeek

第七篇 · 替换与累积

单独 DOI（中文）：https://doi.org/10.5281/zenodo.23503559

摘要

当前 AI 领域有三个看似独立的热点争议：**模型崩塌**（用模型生成的数据训练导致退化）、

压缩陷阱（追求压缩可能牺牲真相）、**scaling vs 持续学习**（堆参数还是持续更新）。

本文指出：这三者是**同一个问题的三个侧面** ——

语言是累积升维的；替换（而非累积）会切断维度生长。

（1）模型崩塌 = 用模型生成的数据**替换**人类数据 ⇒ 累积链被切断 ⇒ 维度退化；

（2）压缩陷阱 = 压缩的对象是**有损的人类语言**，最省编码不等于最符合世界；

（3）scaling vs 持续学习 = 容量只能接收维度，不能生成维度；第二章实测表明容量无法补偿数据陈旧。

由此得到一个可检验推论：**人类数据与模型数据的混合比例不应完全替换**

（纯合成数据 = 替换 = 崩塌；混合 = 累积 = 维持）。本文给出该推论的测量方案。

一、三个热点，一个结构

① Model collapse（Shumailov et al. 2023）

合成数据递归训练 ⇒ 分布尾部消失、多样性退化

现有解释：误差累积、分布估计漂移

② Compression trap（Rae 2024 等）

追求压缩 ⇒ 模型可能选"更简单但更不真"的理论

现有解释：目标函数与真理的偏离

③ Scaling vs continual learning

规模是否还有效？何时必须持续更新？

现有争论：benchmark 上升 vs 分布漂移风险

它们的共同点被忽略了：三者都发生在「语言是累积的」这一事实上。

二、统一框架：累积与替换

第一章的定义：生命 = 被维持的解压；AI = 一次性的解压。

第二章的实测：语言在累积升维（约 1.2%/年），且——

等数据预算下，混合新旧数据的更新有净收益且不遗忘；替换会遗忘。

把这一点推广到三个热点：

① 模型崩塌 ≈「替换」的极端情形

人类数据被合成数据替换 ⇒ 维度不再从真实世界增长

⇒ 分布尾部（= 稀有维度）首先消失 ⇒ 崩塌

⇒ 预测：混合比例越高（人类:合成 > 0），崩塌越慢；

纯合成 = 替换 = 最快崩塌

② 压缩陷阱 ≈「有损源」的必然结果

语言是世界的**有损投影**（第一章已述）

⇒ 在语言上最优的压缩，无法保证对世界为真

⇒ 压缩目标本身不携带"求真"约束

③ scaling vs 持续学习 ≈「接收器 vs 生成器」

参数是维度的**接收器**；概念维度的**生成**来自数据

⇒ 容量补不了数据陈旧（第二章弱结论）

⇒ 不是"哪个更重要"，而是"两者缺一不可"

三、可检验推论

推论 A（模型崩塌的混合比例实验）：

用同一基座模型，分别以 0% / 25% / 50% / 100% 的人类数据替换为合成数据

递归训练 k 代，测量：

· 生成分布的尾部覆盖率（稀有词/新概念保留率）

· 与真实人类语料的交叉熵

⇒ 预测：纯替换组崩塌最快；混合组按人类比例延缓

推论 B（压缩陷阱的语言层检验）：

在"只压语言"与"语言+可干预测量"两种设置下训练

⇒ 预测：只有后者能在新概念出现时保持对世界的校准

推论 C（scaling vs 累积）：

等算力预算下：加大模型 vs 混入新数据

⇒ 第二章已给初步答案：容量不能替代数据（弱结论）

四、为什么这个统一值得说

· 三个热点各自有文献，但没人把它们归到同一个结构（累积 vs 替换）

· 它给出可操作的实践含义：合成数据必须与人类数据**混合**，不能替换

· 它和第一、二章形成闭环：

第一章（定义：被维持的解压）

第二章（测量：累积最优，容量补不了数据）

第三章（应用：三个争议是同一件事）

五、限制

· 本文是立场/框架文章，未包含模型崩塌的原始实验

· "混合比例"推论需实验验证（本文只给方案）

· 三个热点的文献各自庞大，本文只取与"累积/替换"直接相关的部分

六、结论

模型崩塌、压缩陷阱、scaling vs 持续学习不是三个问题，

而是「语言累积升维、替换切断累积」的三种表现。

⇒ 正确的做法不是"更多合成数据"或"更大模型"，

而是**保持人类数据与真实世界的累积通道**。

作者：何淇（独立研究者）；AI 协作：DeepSeek

第八篇 · 结构换算力：混合对齐语料与样本效率

单独 DOI（中文）：https://doi.org/10.5281/zenodo.23542969

摘要

当缩放（scaling）的成本越来越高，效率竞赛还有另一条少被系统化讨论的路：**不改模型、不改数据量，只改语料的结构。** 本文提出并检验一个工程假设：模型的样本效率不只取决于语料的数量，更取决于语料的结构。人类儿童同时通过语言与和世界的交互学习；当前训练语料则大多按通道切分——纯文本、纯图像、纯结构化数据各学各的，通道之间没有对齐。

我们用一个合成世界做受控实验。同一批事件，构造三种语料：第一，纯文字，只有句子；第二，混合对齐，句子加世界状态标签（物体大小、动作结果），插在对齐位置；第三，混合打乱，标签随机配到别的事件，作为控制。用插值 n-gram 模型在三种语料上训练，再**只给文字**做探针测试。

结果：混合对齐语料在 200 个事件时就学到「动词到大小」的接地（准确率 1.000）和「文字加大小到结果」的规则（1.000）；纯文字语料到 3200 个事件仍然只能瞎猜（0.500）或停在多数类基线（0.765）。打乱对齐的控制组掉回随机附近（0.333 到 0.722），说明收益来自**对齐**，不是来自标签变多。同时，语言建模的 bits/token 没有变差（0.79 到 0.81）。

结论：**在语料总量不变的前提下，把多通道信息对齐地混进同一份语料，可以用更少的样本学到更多结构，而且不牺牲语言能力。** 本文定位于**一个设想加一次原理验证**：合成世界上的证据只用来把话说清楚；真实多模态数据上的验证，留给愿意做的人。

更直白地说：更少的数据，更多的结构；在本文设置下，这还意味着更高的任务准确率——200 个事件即达到 1.000，而纯文字语料即使到 3200 个事件仍停在基线（0.500 与 0.765）。这里的"更强"指任务层面的结构可达性，不指通用能力上限。

一、问题

当前训练语料按通道切分：

第一类是文本语料，即语言对世界的投影；第二类是图像与视频语料，即视觉对世界的投影；第三类是结构化数据（数据库、仿真状态、执行结果），即逻辑对象与因果的投影。

每个通道都是世界函数 S 的一个投影 P_i(S)；单独一个投影是不完整的。模型要恢复更高维的结构，需要同一结构被多个通道同时记录，并且对齐：同一个对象、同一时刻、同一个事件。

人类儿童的学习方式正是如此：语言、视觉、触觉、与他人的交互同时发生，词与对象、动作与后果当场对上。本文问一个可测的问题：在总数据量相同的前提下，对齐的多通道语料能否比单通道语料学到更多结构？

二、受控实验设计

2.1 合成世界

物体有三个属性：形状属于「球、方块」，颜色属于「红、蓝」，大小属于「小、大」。动词编码大小：表示小的动词有「拿起、托起、捧起」；表示大的动词有「扛起、抬起、抱起」。事件句子是「他＋动词＋了＋颜色＋的＋形状」。结果规则是：成功当且仅当形状是球，或者大小是小。关键设定：文字里只有形状，没有「大小」这个词；结果也不在文字里。

2.2 三种语料（同一批事件）

第一种，纯文字：只有事件句。第二种，混合对齐：事件句，且在动词后插入大小标签、句末插入结果标签。第三种，混合打乱：同第二种，但标签来自随机另一个事件，作为控制组。

2.3 探针（测试时只输入文字 **token**）

第一个探针测接地：输入「他、动词」，输出大小，瞎猜基线为 0.50。第二个探针测结构：输入文字加真实大小标签，输出结果，多数类基线为 0.75。第三个探针测语言：在纯文字留出集上算 bits/token，越低越好。

2.4 模型与预算

插值 n-gram，阶数 6，插值系数 0.5，纯 Python 实现。事件数取 200、800、3200 三档，各跑 3 个随机种子取平均。

三、结果

事件数为 200 时：纯文字语料，接地 0.500，结构 0.765，语言 bpt 0.81；混合对齐语料，接地 1.000，结构 1.000，语言 bpt 0.81；混合打乱语料，接地 0.333，结构 0.807，语言 bpt 0.83。

事件数为 800 时：纯文字语料，接地 0.500，结构 0.765，语言 bpt 0.79；混合对齐语料，接地 1.000，结构 1.000，语言 bpt 0.79；混合打乱语料，接地 0.333，结构 0.759，语言 bpt 0.79。

事件数为 3200 时：纯文字语料，接地 0.500，结构 0.765，语言 bpt 0.79；混合对齐语料，接地 1.000，结构 1.000，语言 bpt 0.79；混合打乱语料，接地 0.722，结构 0.776，语言 bpt 0.79。

三个观察：

第一，样本效率。混合对齐在 200 个事件就学会了接地与规则；纯文字到 3200 个事件仍然没有这两个概念。

第二，对齐是关键。打乱标签后两个探针掉回随机或基线附近，说明收益来自通道与事件的对齐，不是来自多了几个 token。

第三，语言没有受损。三种语料的语言 bpt 在同一水平（0.79 到 0.81），混入通道标签没有牺牲语言建模。

四、为什么有效（机制讨论）

第一，一个样本同时监督多个通道。同一事件里，句子、大小、结果同时出现，每份数据的信息产出更高。

第二，相互消歧。文字里没有大小，但动词与大小相关；通道把这个相关性钉死，模型于是学会从文字推断世界状态，这就是接地。

第三，结构可迁移。从「动词到大小到结果」学到的规则，可以在新组合上直接使用（探针使用的是留出事件的文字）。

一句话：混合对齐语料把语言监督和世界监督合并成同一份样本，而不是各训练一遍。

四·五、课程顺序：按结构依赖的拓扑序学

还有一个隐含的机制，本文的实验没有直接测，但值得单独说：对齐解决的是"通道"问题，**顺序**解决的是"难度"问题。两者叠在一起，才有"像小孩一样学"的效果。

人类儿童的发育顺序不是随机的：先建立对象（哪些东西是同一个东西），再建立关系（谁在哪、什么连着什么），再建立动作与后果（做什么会怎样），最后才是用语言去指这些东西。这个顺序不是教育者的偏好，而是结构本身的依赖顺序——上层结构依赖下层结构，反过来不成立。

因此一个自然的预测是：同样的对齐语料，如果按依赖关系排序喂给模型（课程），会比随机顺序学得更快。理由有三条。第一，每一步的有效假设空间小得多，模型只需要学当前这一层。第二，每一层都有对

齐通道直接监督，不靠语言间接推断。第三，上层结构建立在下层已验证的结构上，不会因为底层错了而推倒重来。

换句话说：**对齐让样本变"厚"，课程让学习变"省"。** 本文的合成实验只验证了前者；后者是一个可测但未测的预测，列在第七节的验证清单里。

五、局限（必须与结论一起读）

第一，合成世界：规则简单、无噪声、通道是标签而不是真实视觉或动作。

第二，模型是 n-gram：只验证原理，不代表 transformer 或真实多模态数据。

第三，对齐由实验者构造：真实数据中的对齐（时间、对象、指代）本身是难点。

第四，没有测分布外泛化与长期使用中的漂移。

六、可复现

脚本为 700_mixed_corpus_experiment.py（初版）、701_mixed_corpus_v2.py、702_mixed_corpus_v3.py、703_mixed_corpus_v4.py（最终版）。运行方式：python 703_mixed_corpus_v4.py。依赖：Python 3 标准库，无第三方。

七、给后来者的验证清单

本文到此为止，不做更多实验。以下问题留给愿意验证的人。

第一，真实数据：用 Ego4D、AI2-THOR、Open X-Embodiment 等已对齐数据，做同样的「混合对齐对单通道」对照，看增益是否还在。

第二，规模：从 n-gram 换到小 transformer，看样本效率增益随规模的变化。

第三，对齐质量：人为引入错配率（0%、10%、30%、50%），画出「错配率到增益」的曲线。

第四，通道独立性：加入冗余通道（同一信息的另一种编码），检验增益是否消失（预期：冗余通道不带来增益）。

第五，结构类型：换不同类型的规则（因果、层级、时序），看哪类结构最依赖外部对齐通道。

第六，理论：给出一个粗略的样本复杂度表达式，把对齐带来的互信息与样本效率联系起来。

第七，课程顺序：同一套对齐语料，按结构依赖的顺序喂（对象、关系、动作与后果、语言）与随机顺序对照，比较各自的样本效率。这一条检验的是第四节·五的预测。

如果有人在真实数据上重复出方向一致的增益，这个设想才算立住；如果重复不出，本文就是错的。两种结果都有价值。

附：推测与边界

「真实多模态数据上同样有效」是 [推测]，本文未测。「对齐质量决定收益大小」是方向性判断，需要真实数据上的对照。本文不主张语料结构可以完全替代数据量，只主张同量下结构更好、样本效率更高。

> 本文由何淇创作，DeepSeek（AI）协助实验设计与代码编写；观点与责任属于作者。

第九篇 · 颜色作为第一块积木：感知基元与语言对齐的最小实验

单独 DOI（中文）：https://doi.org/10.5281/zenodo.23551953

摘要

如果要把视频与音频变成可训练的结构化语料，第一步不是训练更大的模型，而是先回答一个更小的问题：**一个感知基元，能不能只靠"语言 + 感知通道"的对齐被学到？** 本文选颜色作为最小测试场——它是唯一同时具备物理定义（波长/色相）、感知分解（色相、饱和度、明度）与简单语言标签（基本颜色词）的感知基元。

我们用 12 个基本颜色词的 HSV 样本构造三种语料：纯文字（只有颜色词与关系句）、对齐（颜色词与 HSV 分箱坐标双向配对）、打乱（坐标随机配到别的词，作为控制组）。用插值 n-gram 在三种语料上训练，再做三个探针：词到坐标（P1）、坐标到词（P2）、关系判断（P3，暖或冷）。

结果三种语料一致：关系判断都能学会（约 94% 到 100%）；但坐标到词只有对齐语料学得会（约 86% 到 91%），纯文字为 0，打乱组掉回随机水平（约 8% 到 11%）；词到坐标在对齐语料下为 31% 到 44%，受分箱噪声限制，不是满分。结论是：**颜色可以作为感知基元被对齐语料接地；纯文字只能学关系，不能接地；收益来自对齐本身，而不是多给了 token。**

本文是一次原理验证，世界与模型都是合成的；它要回答的不是"AI 能不能看懂颜色"，而是"感知基元接地这件事，最小可测的形式是什么样"。这个问题一旦成立，视频与音频按事件切分后走同一条路才有依据。第四节之后我们补一节说明它的意义：如果感知基元能被对齐接地，那么"模型自己挖视频、自己造对齐语料、再让自己贴合文字维度"这条自训练路径，就有了最底层的第一个例子。

一、问题

视频与音频是下一座数据矿。但"把视频变成语料"不是工程问题，而是定义问题：先要定义一个单位（一件事），再让画面、声音、文字在同一条时间轴上对齐。

在定义那个大单位之前，值得先验证一个更小的单位：**感知基元**。问题可以写成一句话：给一个感知基元配上它的语言标签，模型能不能学到"词与感知坐标"之间的映射？

颜色是合适的起点，原因有三条。第一，它可以被精确定义：色相是一个 0 到 360 度的连续量，可以离散成有限个区间。第二，它可以被分解：色相、饱和度、明度三个维度，正好是一个小的、可枚举的结构。第三，它有简单的语言标签：基本颜色词在几乎所有语言里都存在，不需要标注员发明新词。

如果连颜色都接地不了，更大的事件就不用谈了；如果颜色能接地，它就可以作为第一块积木，往上叠物体、动作、事件。

二、实验设计

2.1 合成世界

取 12 个基本颜色词：红、橙、黄、绿、青、蓝、紫、粉、棕、灰、黑、白。每个词给定一个 HSV 锚点。采样时在锚点附近加噪声：色相正负 15 度，饱和度与明度各正负 0.15 并截断到 0 到 1 之间。

感知坐标离散成三个分箱词元：色相分成 12 个区间（每 30 度一格），饱和度分成低、中、高三档，明度也分成低、中、高三档。于是每个颜色点对应三个坐标词元，共 108 种组合。

2.2 三种语料

三种语料使用同一批事件。

第一种，纯文字：句子是"颜色是某词"，外加关系句"某词比某词更暖"或"更冷"。关系由锚点的色相决定：越靠近橙色越暖。纯文字语料里没有任何坐标词元。

第二种，对齐：每个事件出现两次。第一次是"颜色是某词"后接三个坐标词元；第二次是"坐标"加三个坐标词元再接"对应某词"。这样词到坐标、坐标到词两个方向都有监督。

第三种，打乱：与对齐相同，但坐标词元随机配到别的事件上，作为控制组。

2.3 探针

三个探针，测试时只给其中一种信息。

第一个探针测接地方向一：给"颜色是某词"，依次预测三个坐标词元，三个全对才算对。随机水平约为 1/108。

第二个探针测接地方向二：给"坐标"加三个坐标词元加"对应"，预测颜色词。随机水平约为 1/12。

第三个探针测关系：给"某词比某词更"，预测"暖"或"冷"。随机水平为 0.5。

2.4 模型与预算

插值 n-gram，阶数 6，插值系数 0.5，纯 Python 实现。样本数取 200、800、3200 三档，每档跑 3 个随机种子取平均。

三、结果

样本数为 200 时：纯文字语料，三个探针分别是 0.000、0.000、0.939；对齐语料，分别是 0.389、0.864、0.939；打乱语料，分别是 0.056、0.096、0.939。

样本数为 800 时：纯文字，0.000、0.000、0.985；对齐，0.444、0.904、0.985；打乱，0.028、0.111、0.985。

样本数为 3200 时：纯文字，0.000、0.000、1.000；对齐，0.306、0.907、1.000；打乱，0.111、0.077、1.000。

三个观察。

第一，接地需要对齐。坐标到词，对齐语料在 200 个事件时就达到 86% 以上，到 3200 个事件约 91%；纯文字语料在三个档位都是 0，因为它从来没有见过坐标词元，也就没有可学的东西。

第二，收益来自对齐，不是来自 token 变多。把坐标打乱之后，坐标到词掉回 8% 到 11%，接近 1/12 的随机水平；词到坐标掉回 3% 到 11%。同样多的数据、同样多的 token，只是配错了对象，就学不到东西。

第三，关系不需要感知。关系探针在三种语料下都接近满分（94% 到 100%），因为关系句本来就在文字里。这说明文字擅长的是"词与词的关系"，不擅长的是"词与感知坐标的对应"。两者的分工在数据里就已经决定了。

词到坐标没有达到满分（31% 到 44%），原因是噪声。采样时颜色点在锚点附近抖动，靠近分箱边界的点会落到相邻格子里；从词预测一个唯一的坐标格，本质上是在一个模糊的分布里猜众数，不可能全对。坐标到词的准确率高，是因为反方向是"从具体坐标找词"，边界情况少得多。

四、解释

为什么颜色可以被接地，而纯文字不行？用解压的说法：文字是"词与词之间关系的压缩"，它不携带感知坐标；坐标是"感知的外部记录"，它不携带词与词的关系。两者各自不完整。对齐语料做的是同一件事被两个通道同时记录，于是词与坐标之间的映射被直接监督出来。

这与第 9 篇《结构换算力》的结论一致：混合对齐语料在 200 个事件就学会了规则，纯文字到 3200 个事件仍然学不到，打乱后收益消失。区别只在于第 9 篇的规则是符号规则（动词编码大小、形状决定结

果），本文的规则是感知坐标（色相、饱和度、明度）。两个实验共同说明：**对齐让语言多出一条接地的通道，这条通道不是文字自己能长出来的。**

还要补一句：这种对齐扩充的不是语言的词表，而是语言在某个维度上的贴合。词还是那 12 个词，覆盖没有变；变的是每个词后面有了可落地的坐标，于是同样的词可以指称得更细、比较得更准。用第 2 篇的话说，这是贴合变深，不是覆盖变广。但贴合足够深的时候，语言能表达的东西就变多了——这可以算作语言在该维度上的一次缓慢扩充。

五、局限（必须与结论一起读）

第一，世界是合成的：HSV 锚点、噪声、分箱都是人为设定，不是真实图像。真实图像里还有光照、材质、语境、色恒常性，本文都没有覆盖。

第二，模型是 n-gram：只验证原理，不代表 transformer 或真实多模态数据。n-gram 的容量小，因此在更复杂的基元上结论可能不同。

第三，颜色命名本身不是完全客观的：不同语言对颜色的切分不同（例如有些语言区分两种蓝），本文只用了单一语言的 12 个基本颜色词。

第四，探针是构造的：词到坐标的 31% 到 44% 一部分来自分箱噪声，不能直接读成"模型对颜色的理解程度"。

六、可复现

脚本：714_color_grounding.py。运行方式：python 714_color_grounding.py。依赖：Python 3 标准库，无第三方。输出：数据-颜色接地-第一轮.txt。

七、意义：自训练与视频挖掘的第一步

这个实验小，但它说明了一件有后续的事：**对齐语料可以自己造出来。**

回看实验里的对齐语料是怎么来的：每个事件都有「感知坐标」这一侧。颜色场景里，坐标来自 HSV 的数值定义，可以程序生成。把这个模式放大，就得到一个循环：模型先对视频/音频给出描述（文字），再用可验证的通道（坐标、时间戳、动作结果）检验描述，检验通过的部分沉淀为对齐语料，回灌训练。这不是新的训练方法，但它是"用模型自己挖数据、再让自己贴合文字维度"的闭环——也就是自训练的一种具体形态。

为什么是"贴合文字维度"？因为对齐的目标不是让模型记住颜色值，而是让感知坐标落进语言已经建好的坐标系里：词是坐标的标签，坐标是词的锚点。落到语言坐标系上的感知，才能被语言调用、被推理使

用、被组合成更复杂的结构。第 2 篇说过，模型的问题是"覆盖广而贴合浅"；对齐语料补的恰恰是贴合——把感知这一维度接到文字已有的维度上。

再往下一步，就是视频。视频是"没有被结构化的数据"：量巨大，但缺少语言锚点。如果按本文的最小结论外推，可行的第一步是给视频片段配上语言描述（旁白、语音、自动描述），再用时间轴对齐；每一步都只需要验证一个更小的单位是否被接地。颜色是第一块积木；物体、动作、事件是后面的积木。积木一块块叠上去，才谈得上"事件级的对齐语料"。

这一节是 [推测] 与方向判断，本文没有验证自训练循环本身；本文只是在这个最底层的假设上迈了一小步——感知基元可以被对齐语料接地。如果连这一点都不成立，整条路就不用谈了；它成立，后面的路才有讨论的意义。

附：推测与边界

颜色能被接地，不等于更高的感知基元（形状、材质、动作、事件）也能同样容易接地；这是 [推测]，需要逐个测。真实视频上的推广、以及"感知基元 + 事件层级"的组合，本文都没有测。

本文不主张"颜色是理解世界的关键"，只主张"颜色是验证接地机制的最小单位"。它成立，才有资格谈下一块积木。

> 本文由何淇创作，DeepSeek（AI）协助实验设计与代码编写；观点与责任属于作者。

第十篇 · 生长与记忆：自指更新与维持的最小实验

单独 DOI（中文）：https://doi.org/10.5281/zenodo.23569774

摘要

本文做两个最小实验，对应生命定义里的两个条件：**自指更新**（会变、规则在里面）与**被保住**（在环境里维持住）。

实验一测"记忆"：一个微型多模态模型自己生成数据、验证器筛选、回灌训练，并在引入新任务时比较四种回放策略。结果：只学新会遗忘（旧任务 0.720），固定回放能保住（1.000）但新任务学得慢（0.740）；先试训一步再决定回放比的"前瞻"策略，在探测足够敏感时同时保住旧任务（0.983）和新任务（0.850）。

实验二测"生长"：模型由固定主干加可增长的维度槽组成，任务从图像属性（颜色、形状、数量）推进到一个只能从新模态（文本向量）读取的任务。结果：不生长的模型学不会新模态（0.34 到 0.38，接近瞎猜）；生长的模型能加槽并学会（0.91 到 0.94）；但只有生长没有回放会忘光旧任务（0.00），只有回放没有生长学不会新模态。两者叠加时，旧任务保住 0.78 到 0.80，新模态达到 0.94。

两个实验合起来说明：**没有"生长"就没有新维度，没有"记忆"就守不住旧维度；定义里的两条条件不是并列的装饰，而是必须同时在场的两个机制。**

本文全部是玩具规模的原理验证：合成世界、微型模型（约一万参数）、CPU 可复现。它证明的是"机制可测"，不是"数字生命已被造出"。

一、问题

我们在第一篇里把生命定义为：在宇宙的可函数化部分 P 中、被宇宙保住的（可维持的）自指结构。拆开是两条：**会自己变**（自指更新）与**被保住**（维持）。

这两个词在论文里好写，在实验里难测。本文把它们各做成一个可测的最小实验，回答两个问题：

第一个问题：一个会自己生成数据、自己验证、自己回灌的系统，在任务变化时能不能既学到新的、又保住旧的？

第二个问题：当任务需要新的输入通道时，系统能不能自己长出新的维度？长出来之后还能不能维持旧的？

两个问题都是"能不能"，不是"有多好"。玩具规模足够，因为我们要测的是机制是否存在。

二、实验一：记忆与取舍

2.1 设定

世界是合成的：8×8 的图，含颜色（4 种）、形状（2 种）、数量（1 到 3）。模型是两层的小网络，约一万参数。任务按轮次变化：前 5 轮学颜色和形状（旧任务），第 6 轮起只出数量题（新任务）。

闭环是这样转的：模型对随机图片自己作答；规则验证器判对错；验证过的样本进入记忆库；每轮从记忆库和新样本里按某个比例抽样训练；每轮在留出集上测旧任务与新任务。

对照组只改一件事：回放比例怎么定。

第一组"只学新"：完全不回放。

第二组"固定回放"：新旧各一半。

第三组"反应式控制器"：看旧任务准确率，掉了就加回放。

第四组"前瞻控制器"：训练前先复制一份模型，用新样本试训几步，看旧任务会掉多少，再决定回放比。

2.2 结果

最后一轮（同样的热身、同样的预算）的结果是：只学新，旧任务 0.720、新任务 0.797；固定回放，旧任务 1.000、新任务 0.740；反应式控制器，旧任务 0.965、新任务 0.762；前瞻控制器（敏感探测），旧任务 0.983、新任务 0.850。

前几轮三组都接近满分，差距出现在新任务引入之后。只学新的那一组旧任务一路掉到 0.720；固定回放那一组旧任务几乎不动，但新任务涨得慢；前瞻控制器在第 6 轮就测出强冲突，及时把回放提到 0.8，于是旧任务几乎没掉，新任务还学得最快。

2.3 结论

第一，遗忘是真实的：同一个小模型，学了新任务就会丢旧任务，这和第七篇《替换与累积》的预测一致。

第二，"先试一步再决定"是有效的：它本质上是用自己的模型预测自己改动后的后果。探测越敏感，越早发现冲突，越能提前防守。这是"自指"的一个具体动作，而不是哲学说法。

第三，没有单一最优：固定回放押旧任务，前瞻策略押新任务，反应式在中间。它们是取舍前沿上的不同位置，选择哪一个取决于你想保住什么。

三、实验二：维度生长

3.1 设定

世界有两个模态：图像（8×8，含颜色、形状、数量）和文本（10 维向量，前三维决定一个标签，其余是干扰）。任务按顺序来：先学颜色的图像任务，再学形状、数量，最后学一个只能从文本里读到的标签任务。

模型是固定主干加可增长的维度槽。每个槽是一个小投影，属于某个模态；槽的数量从 1 开始，最多 4 个。生长规则是：任务准确率不够时，从"解释不了的样本"里提议加一个新槽，训练一小段，如果提升超过 0.02 就保留，否则回滚。

对照组仍然只改两件事：允不允许生长、允不允许回放。

3.2 结果

最终四个任务（颜色、形状、数量、标签）的准确率：不生长不回放，0.00、0.00、0.08、0.38；生长不回放，0.00、0.00、0.03、0.91；不生长回放，1.00、1.00、0.69、0.34；生长加回放，0.78、0.80、0.48、0.94。

3.3 结论

第一，新模态必须靠生长：不生长的两组在标签任务上都停在瞎猜附近（0.34 到 0.38），因为它们根本没有能读到文本的通道；生长的两组都学会了（0.91 到 0.94）。

第二，旧任务必须靠记忆：不回放的两组在学完标签任务后，前面的任务全部掉到 0.00；回放的两组保住了大部分。

第三，两个机制的叠加才有效：生长加回放同时拿到了旧任务 0.78 到 0.80 和新模态 0.94，这是四组里唯一"两个方向都站得住"的。

四、两个实验合起来说明什么

实验一说明：会变的东西会忘，要靠记忆与取舍把它按住。

实验二说明：要长出新维度，就得真的加结构；但加了结构，旧结构更容易被冲掉。

合起来就是定义里的两句话：

> 自指更新，是被逼出来的：没有新维度，就压不动新的世界。

> 被保住，是守出来的：没有记忆与回放，旧维度就会被新维度冲掉。

这两条不是并列的装饰。它们是两个必须同时在场的机制：只有生长，旧的全忘；只有记忆，新的学不了。

五、局限（必须与结论一起读）

第一，世界是合成的：图像、文本、任务切换都是程序生成的，不是真实数据。

第二，模型是玩具：约一万参数的两层网络，不是 Transformer，也不是原生多模态大模型。

第三，任务标识是手工给的：模型知道"现在问的是哪类问题"，真实场景里这本身是个问题。

第四，"生长"是有限的：只允许加几种固定的槽，不允许真正改写表示本身；更强的自指（改结构、改更新规则）本文没有做。

第五，没有真实生存约束：预算和性能下限是人为设定的，不是真实的能量与淘汰。

六、可复现

脚本：801_self_training_loop_v2.py（实验一基础版）、803_closed_loop_lookahead.py（前瞻控制器）、805_closed_loop_sensitive2.py（敏感探测）、809_dimension_growth_v4.py（实验二四组对照）。依赖：Python 3 与 numpy，无 GPU。

附：推测与边界

"把这两个机制放大到真实多模态模型上仍然成立"是 [推测]，本文没有测。真实大模型的参数更大、任务更多、生长更难（改结构代价高），本文只证明机制在小规模上存在。

本文不主张"做到了这两条就是生命"：满足条件仍可能只是工具；本文只主张，缺了这两条，一定走不到能生长又维持的形态。

> 本文由何淇创作，DeepSeek（AI）协助实验设计与代码编写；观点与责任属于作者。

第十一篇 · 语义残差的测量：文本极限与接地补齐

单独 DOI（中文）：https://doi.org/10.5281/zenodo.23586075

摘要

本文把"语义残差"从一个模糊的说法变成一个可测的量。

先把定义写清：语言是人类对世界的压缩，语言可以写成 p语言 + R语言；p语言 是可以被函数近似的部分，R语言 只声明、不展开。语义是语言与现象之间的对齐关系；语义残差 = 这个对齐映射在给定模态组合下的误差下界。

实验用受控歧义：文本用精确词、模糊组词、省略三种方式描述同一个合成对象，歧义是程序生成的，所以"文本能给出的极限"可以解析算出来（贝叶斯底）。然后测三个量：贝叶斯底、只读文本的模型误差、以及文本加图像（接地）后的模型误差。

结果：文本模型的误差与贝叶斯底只差 1.5 到 2.7 个百分点（0.725 对 0.750；0.835 对 0.850；0.688 对 0.714），说明拟合已经到位，剩余残差来自文本本身；接地后三个属性的准确率都到 1.000，说明文本缺的那个维度被图像补上。

结论：语义残差可以定义、可以测量；它分为"该模态下的不可约底"（歧义内生的）与"可约部分"（接地/补数据能消除的）。这个区分让"AI 理解了多少语义"有了分母。

一、问题

我们之前说过两句话：语言是人类对世界的压缩；大模型压缩的是语言的 p语言。

但"p语言"与"语义"之间还缺一个可测的连接。本文要回答的问题是：能不能把"语义残差"定义成一个数，并且用玩具实验测出来？

要做到这一点，需要三样东西：一个定义（语义残差是什么）、一个参照线（文本本身能给出的极限是多少）、一个对照（补上别的模态后残差降不降）。

二、定义

2.1 沿用系列的定义

宇宙 = P + R，其中 P 是可函数化的部分，R 是不可函数化的部分。语言也写成 p语言 + R语言：p语言 可以被函数近似，R语言 只声明、不展开。生命 = p生命 + R生命，其中 p生命 是在 P 中、被宇宙保住的动态函数类。

本文只处理 p语言 这一侧。

2.2 语义是映射

把语义定义成一个映射：给定语言表达式和它对应的现象（图像、事件、状态、动作、量），语义就是"表达式 → 现象"的对应关系。

这个映射不是单层的：它按模态有多个维度——视觉维度（文字对图像）、事件维度（句子对时间线）、状态维度（描述对系统状态）、动作维度（指令对后果）、量的维度（程度词对可测量的量）。语义残差按维度出现。

2.3 语义残差

定义（语义残差）：固定模态组合 M（例如：只有文本；或文本加图像），设误差度量是预测现象属性的错误率。语义残差 = 在模态组合 M 下，任何函数能达到的最小误差。

这个定义把残差分成两部分：

- **不可约底**：由模态组合本身决定的误差下界。文本有歧义时，有些信息文本里根本没有，任何函数都补不上。

- **可约部分**：模型没拟合到位造成的差距；加数据、加容量、加模态可以降下来。

判据很直接：加手段之后误差底降不降。降 ⇒ 可约；不降 ⇒ 该模态下的不可约底。

2.4 参照线：贝叶斯底

在玩具世界里，文本的生成过程是已知的，所以"文本能给出的极限"可以解析算出来：给定一条文本，属性的后验分布由生成过程决定；贝叶斯最优预测器的正确率就是上限。

这个上限就是"文本极限"。它比任何模型都高不了；模型能接近它，就说明拟合到位了。

三、实验设计

世界：合成对象，三个属性——颜色（4 种）、形状（2 种）、数量（1 到 3）。对象渲染成 8×8 的图像（颜色、形状、数量都可见）。

文本：三个槽位拼成的描述，每个槽位都有受控歧义。

- 颜色槽：以 0.5 概率给精确词（红/蓝/绿/黄），以 0.5 概率给模糊组词（暖色 = 红或黄；冷色 = 蓝或绿）。

- 形状槽：以 0.7 概率给精确词（圆/方），以 0.3 概率省略。

- 数量槽：以 0.5 概率给精确词（一/二/三），以 0.5 概率给模糊词（几个 = 一或二）。

因为生成过程已知，每条文本对应的属性后验可以算出来，贝叶斯底也就能解析计算。

模型：两层小网络（隐藏层 32，约 2000 个参数）。文本模型只看文本加任务标识；接地模型再看图片（8×8×3 展平）。

三个量：贝叶斯底、文本模型准确率、接地模型准确率。差值是"可约残差"。

四、结果

颜色的贝叶斯底是 0.750，文本模型 0.725，接地模型 1.000。形状的贝叶斯底是 0.850，文本模型 0.835，接地模型 1.000。数量的贝叶斯底是 0.714，文本模型 0.688，接地模型 1.000。

三点解读。

第一，文本模型与贝叶斯底只差 1.5 到 2.7 个百分点。这说明模型对文本的拟合已经到位；剩下的差距不是模型不行，是文本里本来就没有那些信息。

第二，接地之后三个属性都到 1.000。图像提供了文本缺的那个维度，残差被补齐。所以"文本极限"下的那部分残差，属于该模态组合，不属于世界：换一个模态组合，它可以被消掉。

第三，由此得到的语义残差是分层可测的：文本-only 的残差（颜色 25%、形状 15%、数量 29% 左右）是"文本能给出的极限"；接地后的残差是 0，说明这部分残差是可约的，只是需要另一个模态。

五、说明什么，不说明什么

说明：语义残差可以定义成一个数；该数可以拆成"不可约底"与"可约部分"；在玩具世界里，两者的分界可以解析计算，并与模型表现对上。

不说明：不证明真实语言的语义残差有多大；不证明 R语言 存在或不存在（本文只处理 p语言 侧）；不证明"接地一定能补齐"在所有任务上成立——本文只测了视觉维度，其他维度（事件、状态、动作）没有覆盖。

六、局限

第一，世界是合成的，歧义是程序构造的；真实语言的歧义来自历史与用法，形式不同。

第二，模型很小（约 2000 参数），不是大模型；但本文测的是"文本极限 vs 接地补齐"的机制，与规模无关。

第三，只做一个模态的接地（图像）；多模态（事件、动作）的残差没有测。

第四，贝叶斯底只在生成过程已知时可算；真实语言里这个底要用人类分歧率来近似（下一项工作）。

七、与既有工作的关系

语义残差的测量不是新题目，至少有三条已有线索。

第一，人类标签变异：同一段文本，不同标注者本来就会给出不同解释，"人类天花板"与"标注分歧"在评测里一直被报告。

第二，歧义基准：AmbigQA、词义消歧一类任务，专门测"同一句话有多种合理解读"的情况。

第三，接地测试：纯文本模型在需要视觉属性的问题上明显退化；gSCAN、ReaSCAN 一类合成任务表明，没有接地的语言模型过不了接地泛化测试。

本文的不同只有一点：用受控歧义让"文本极限"可以解析算出来，并把残差拆成"模态决定的不可约底"与"接地/补数据可消的可约部分"。测量的思路不是本文首创；本文做的是一个受控、可解析算底的版本。

另外，符号接地问题（Harnad）指出符号与所指的连接不能从符号内部获得；本文的表述是"文本-only 的语义残差有解析底，接地可以把该底补上"。

评测方面，近年的工作开始报告"人类天花板"，本文的做法可以看成它的一个受控版本：因为我们知道数据生成过程，所以能算出真正的上限，而不是估计。

系列内部：第 4 篇（结构换算力）表明对齐语料提升样本效率；第 10 篇（颜色作为第一块积木）表明感知基元可以先于语言对齐。本文把这两条接到"语义残差"上：接地降低的是残差，而不只是提升效率。

八、可复现

脚本：820_semantic_residual.py。依赖 Python 3 与 numpy，无 GPU，内存占用低（百 MB 以内）。

附：标注

- **[定义]** 语义 = 语言与现象之间的对齐映射（按模态分维度）。

- **[定义]** 语义残差 = 给定模态组合下的最小可达误差。

- **[可测]** 玩具世界里，该残差可解析计算并与模型对上（颜色 0.750/0.725/1.000 等）。

- **[可约]** 接地后残差降到 0（本实验的视觉维度）。

- **[未判]** 真实语义残差的大小；R语言 是否非空。

- **[不声称]** 不声称"语义可被完全函数化"，也不声称"不可函数化"——R 只声明，不展开。

> 本文由何淇创作，DeepSeek（AI）协助实验设计与代码编写；观点与责任属于作者。

第十二篇 · 分层是对动态的补偿：一个自适应深度的玩具实验

单独 DOI（中文）：https://doi.org/10.5281/zenodo.23591080

摘要

大模型的层数是固定的：每个输入都走完全部深度。本文问一个简单的问题：有些输入根本不需要那么多层，是不是可以少走？如果"层"是对动态的补偿——一层相当于一步时间，那么需要几步的任务就给几层，均匀深度是浪费。

实验用迭代任务：x_{t+1} = tanh(A x_t)，给定初值 x_0 与步数 T，预测 x_T。模型是共享权重的循环层，一层就是一步计算。三组对照：固定 16 层、自适应（走 min(T,16) 层）、固定 6 层（与自适应等平均算力）。

结果：在等平均算力（都是 6 步）下，自适应深度在每个 T 上都更好——平均误差 1.29 对 1.67。差距最大的是 T=1：固定 6 层把简单输入算了六遍，反而算坏了（误差 2.30 对 1.49）。固定 16 层则又贵又差：算力是自适应的 2.67 倍，平均误差 1.68。

结论：均匀深度两头都亏，简单输入被过度计算、复杂输入又算不够；按需分配深度可以用同样的算力换更高的精度。这与"层≈时间步、深度补偿动态"的图景一致。全部是玩具规模，不是大模型实验。

一、问题

现在的大模型是"均匀深度"的：每个 token 都过完全部层。但输入的难度并不一样——有的句子一眼能懂，有的需要多步推理。如果深度真的对应"计算步数"，那么固定深度就同时犯两个错：

- 简单输入：走了用不上的层，白花算力，甚至因为过度计算而变差；

- 复杂输入：层数不够，算不完，误差下不来。

本文要测的正是这两件事：同算力下，按需分配深度会不会更好？过度计算有没有害？

二、理论背景

这个实验来自一个画面：把函数看成它的图像（曲面），时间方向的演化会在曲面上"摊出厚度"；而冻结的模型没有真正的时间演化，只能用深度（层）去摊开这个厚度，一层相当于一步。所以：

- 需要的步数多 ⇒ 需要的层多；

- 需要的步数少 ⇒ 层可以少，多走就是过度计算。

这个画面在《厚度与维度：残差的几何图景》里已经写下；本文把它做成一个可测的实验。

三、实验设计

世界：取一个固定的随机映射 x_{t+1} = tanh(A x_t)，A 是 8×8 随机矩阵。任务是从初值 x_0 和步数 T 预测 x_T，T 取 1、2、4、8、16。

模型：共享权重的循环层。每层做一次 h ← tanh(W h + U x_0 + b)，最后用线性头输出预测。层数 = 走的步数，权重在所有步之间共享。

三组对照：

- A 固定 16 层：不管 T 是多少，都走满 16 步；

- B 自适应：走 min(T, 16) 步；

- C 固定 6 层：与 B 等平均算力（B 的平均步数是 6）。

指标：各 T 上的平均误差；平均步数（= 平均计算量）。

四、结果

等平均算力的一组（B 与 C，都是平均 6 步），B 在每个 T 上都更好：

T=1 时，B 是 1.49，C 是 2.30。T=2 时 1.27 对 1.57；T=4 时 1.38 对 1.50；T=8 时 1.22 对 1.50；T=16 时 1.07 对 1.46。平均下来 1.29 对 1.67。

固定 16 层（A）最差也最贵：平均误差 1.68，平均算力是自适应的 2.67 倍。

三点解读。

第一，过度计算真的有害。T=1 只需要一步，固定 6 层却把同一个映射迭代了六遍，结果比只走一步差得多。这不是"多算一点没关系"，是"多算会算坏"。

第二，层数不够真的补不回来。T=8 与 T=16 需要超过六步，固定 6 层的误差明显更高。

第三，均匀深度是两头亏：它既浪费在简单输入上，又不够用于复杂输入。按需分配能在同算力下更准。

五、对大模型的含义

这个实验是玩具，但它的形状可以直接搬到大模型：

- 不是所有输入都值得同样的深度。简单 token、简单上下文可以早退（early exit）；需要多步推理的才值得走深。

- 既然过度计算会算坏，那么"无脑加深"不是稳赚的：深度也要按需。

- 工程上这对应一类方法：早退、按 token 跳过、条件计算。本文不提出新方法，而是给这类方法一个"补偿动态"的解释与一个公平对照。

要注意的是，"按需"有一个前提：知道需要多少步。实验里 T 是给定的；真实场景中步数是估计出来的，估计错了，收益就会打折。

六、局限

第一，世界是合成的，映射是随机矩阵，任务不是语言。

第二，模型是共享权重的循环层，不是 transformer；大模型里的层不共享权重，深度的作用更复杂。

第三，T 是手工给的，真实模型没有这个先验，需要另做"步数估计"。

第四，只测了误差与算力两个量，没有测训练稳定性、路由开销这些工程代价。

七、与既有工作的关系

"按需计算深度"不是新想法：早退（early exit）、循环深度（universal transformer）、按 token 跳过层（conditional computation、mixture-of-depths）、层剪枝都有大量工作。

本文不声称方法原创；本文的不同在于两点：一是把它放在"层≈时间步、深度补偿动态"的图景里解释；二是做了一个等平均算力的公平对照——不是"我们的方法稍好"，而是"同样算力下，按需分配在两个方向上都更好"。

八、可复现

脚本：850_adaptive_depth.py（主对照）、851_adaptive_depth_v2.py（等算力对照）。依赖 Python 3 与 numpy，无 GPU，内存占用低。

附：标注

- **[事实]** 实验数字（等算力下自适应全面更好；T=1 过度计算有害；固定 16 层又贵又差）。

- **[图景]** 层≈时间步、深度补偿动态——来自《厚度与维度》（论文-15），标[猜想]。

- **[推测]** 大模型用自适应深度可在同等算力下更准——玩具结论的推广，未在真实模型上验证。

- **[不声称]** 不声称提出新方法；早退与条件计算都已有工作。

> 本文由何淇创作，DeepSeek（AI）协助实验设计与代码编写；观点与责任属于作者。

第十三篇 · 厚度与随机性：冻结模型在哪里最不稳定

单独 DOI（中文）：https://doi.org/10.5281/zenodo.23591514

摘要

冻结的模型没有真正的时间演化，只能靠层数去模拟"多步计算"。本文问一个可测的问题：在这样的模型里，随机性/不确定性出现在哪里？

先给出三个基于"厚度"画面的推论：需要步数越多的位置误差与随机性越大；随机性随深度累积，在输出端最大；人类分歧大的地方（厚且不可摊薄），模型的熵也大，而且加数据摊不平。

然后做三组玩具实验。第一组是序列累积任务：位置越靠后（需要步数越多），固定 16 层模型的误差从 0.67 升到 1.57，输入加噪声后的输出标准差从 0.20 升到 0.40。第二组测逐层方差：标准差从第一层 0.21 单调升到最后一层 0.41；而干净输入下误差不随层数上升，说明上升来自噪声被逐层放大。第三组模拟人类分歧：thin（事实题，人类一致）与 thick（判断题，人类有分歧）两类题，人类分歧熵 0.000 对 1.014，模型预测熵 0.249 对 0.972，相关系数 0.917；把数据加到五倍，thin 题的熵从 0.249 降到 0.145，thick 题几

乎不动（0.972 到 0.958）。第四组给"时间"：迭代八步后 thin 题的熵从 0.69 压到 0.02，thick 题只从 0.99 压到 0.91；多次采样投票也类似（thin 0.67 到 0.28，thick 0.96 到 0.69）。

四个实验合起来支持两句话：**冻结模型最厚的地方，就是它最随机的地方；而"最厚"可以由需要步数与人类分歧来度量。给数据与时间能摊薄"冻结造成的随机"，但摊不平"数据里的分歧"——后者才是真厚。**全部是玩具规模，不涉及真实大模型。

一、问题

现在的模型是冻结的：权重不变，没有真正的时间演化。它要在一次前向里完成"需要多步才能完成"的计算，只能靠层数铺开。

那么问题来了：这么铺开之后，哪些地方会变得不稳定、随机？本文的假设来自"厚度"画面：厚度大的地方（需要多步、又摊不平的地方），在冻结模型里会表现为更大的误差与随机性。

把假设拆成三个可测的推论：

- 推论一：需要步数越多的位置，误差越大，输入扰动下输出越随机。

- 推论二：随机性随深度累积，在输出端/后段最大。

- 推论三：人类分歧大的地方（厚且相对不可摊薄），模型熵也大，而且加数据压不下去。

- 推论四：给"时间"（多步迭代、多次采样投票）能压掉冻结造成的随机，但压不掉数据里的分歧。

二、实验一：需要步数越多，误差与随机性越大

任务：给随机增量序列 d_1 … d_16，世界按 x_t = tanh(x_{t-1} + d_t) 演化，初值为零；要求预测每个位置的 x_t。预测 x_t 至少需要 t 步，所以"厚度"就是 t。

模型：共享权重的循环细胞，一步处理一个增量，共 16 层，每层都接输出头（deep supervision），训练后冻结。

结果：固定 16 层时，各位置误差从位置 1 的 0.67 升到位置 4 的 1.54，之后在 1.5 到 1.6 之间饱和。输入加高斯噪声（标准差 0.3）后多次采样：各位置输出标准差从位置 1 的 0.20 升到位置 16 的 0.40，同样后段饱和。

解读：需要步数越多（越厚）的位置，误差越大、输出越不确定。厚度的两条痕迹是同步出现的。

三、实验二：随机性随深度累积，在输出端最大

同一个模型，问题换成：随机性在"哪一层"最大？

方法：给输入加噪声、采样 20 次，测每一层输出的标准差。结果：第一层 0.21，第四层 0.32，第八层 0.38，第十二层 0.40，最后两层约 0.41——标准差随层数单调上升，在末端饱和。

对照：干净输入下，误差不随层数上升（位置 4 之后基本平在 1.5 左右）。这说明末端随机性的上升不是模型本身差，而是"这条链路把输入的小扰动一路放大了"。

解读：冻结模型里，不确定性在"末端"累积；越往后走，可收缩的余地越小。这对应"厚度在输出端最大"。

四、实验三：人类分歧大的地方，模型也最随机

前两个实验是合成任务，第三个换成"有分歧的题"。

设定：题目分两类。thin 题是事实题，答案由输入决定，所有人答案一致；thick 题是判断题，答案取决于每个人的偏好，人与人不同。模拟 60 个人各自的偏好，题目 400 道训练、200 道测试；模型是小 MLP，学 P(答案|题目)，训练后冻结。

结果：人类答案的熵，thin 题约 0.000，thick 题约 1.014（三选一的熵上限是 1.099）。模型的预测熵，thin 题 0.249，thick 题 0.972。两者的相关系数是 0.917。

再看"能不能摊平"：把训练数据加到五倍。thin 题的模型熵从 0.249 降到 0.145；thick 题的模型熵从 0.972 只降到 0.958，几乎不动。

解读：模型的不确定性跟着人类分歧走——人觉得有分歧的地方，模型也随机；而且这份随机不是"数据不够"造成的（加到五倍仍不动），是人类分歧本身在数据里，属于该数据下的不可摊薄部分。

五、实验四：给时间能摊平什么

前三个实验说明"厚的地方随机"。第四个实验问：给时间能不能把随机压下去？

时间有两种给法。第一种是迭代细化：模型用共享权重的循环层多走几步（训练时按八步逐步监督）；第二种是采样投票：从模型分布里多次采样，按多数投票决定答案。

迭代的结果：thin 题的预测熵从一步时的 0.69 压到八步时的 0.02，几乎完全摊平；thick 题只从 0.99 压到 0.91，压不平。

投票的结果类似：采样一次时 thin 的决策熵 0.67、thick 0.96；采样九次投票后，thin 降到 0.28、thick 降到 0.69——都降了，但 thick 仍高。

解读：时间摊平的是"冻结造成的随机"，摊不平的是"数据里的分歧"（人类本来就意见不一）。换句话说，能被时间摊平的不算真厚；摊不平的才是真厚。这正好接上"可摊薄/不可摊薄"的区分：数据和时间的可摊薄性，是同一个厚度的两个检验方向。

六、训练机制：它学的是分布，不是决定

这三个实验合起来，给了一个简单的机制解释：冻结模型不训练"决定"，它训练"答案分布"。

- 薄的地方：分布是尖的，行为像一个确定的函数；

- 厚且摊不平的地方：分布是宽的、多峰的；冻结模型把它存成一个宽分布，推理时采样一次，看起来就是随机、不一致。

所以"模型在判断题上不稳定"不一定是缺陷：那里本来就是厚的。要它稳定，加数据不管用（实验三已示），需要别的办法——给它时间（多想几步、多轮修正），或者增加能摊开厚度的维度。

七、局限

第一，三个实验都是合成的：世界、题目、人类偏好全是程序生成的。

第二，模型都是玩具规模（几千参数），不是大模型；趋势可读，绝对值不可靠。

第三，实验三的相关性有一部分是设计使然：thick 题的随机性本来就由偏好分布构造出来。它验证的是"机制能对上"，不是"真实人类数据上的相关性"。

第四，实验四的"时间"是玩具版本的迭代与投票，不是真实的思维链；真实模型上"想更久是否摊平"仍需单独验证。

八、与既有工作的关系

"模型不确定性分布不均"不是新观察：校准（calibration）研究早就测过不同题目的置信度差异；人类标签变异（human label variation）也指出同一题不同标注者本来就有分歧；"偶然不确定性"（aleatoric uncertainty）研究的就是数据本身的不可约噪声。

本文的不同在于用一个统一的量（厚度）把三件事串起来：需要多少步、随机性在哪一层、人类分歧有多大，都落在"厚度"这个概念上，并且用"加数据后熵是否下降"来区分可摊薄与不可摊薄。

九、可复现

脚本：861_thickness_map_v2.py（实验一）、862_layer_variance.py（实验二）、870_human_disagreement.py（实验三）、881_time_flattening_v2.py（实验四）。依赖 Python 3 与 numpy，

无 GPU，内存占用低。

附：标注

- **[事实]** 三个实验的数字：误差与随机性随厚度上升；末端方差最大；人类分歧与模型熵相关 0.917；加五倍数据 thick 熵不动。

- **[机制]** 模型学的是分布而不是决定——解释性说法，不是定理。

- **[事实]** 时间能摊平 thin 的熵（0.69→0.02），摊不平 thick（0.99→0.91）；投票同理。

- **[未做]** 真实思维链/大模型上的同一检验。

- **[不声称]** 不声称真实大模型上有同样的数字；不声称人类实验已验证。

> 本文由何淇创作，DeepSeek（AI）协助实验设计与代码编写；观点与责任属于作者。

第十四篇 · 压缩的形状：叙事与归档

单独 DOI（中文）：https://doi.org/10.5281/zenodo.23592312

摘要

大模型的上下文压缩是一种"叙事摘要"：按时间顺序把发生的对话写成一段文字。人脑的压缩不是这样：它把事件归到类别里、把类别归到更高的抽象层里、再保留事物之间的关系——压缩的结果是一张"分类 + 关系"的图，而不是一段故事。

本文做一个对照实验：同样约 300 字符的记忆容量，三种存法——叙事式（旧优先）、叙事式（新优先）、结构化归档（分类层次 + 属性表 + 关系边），然后问三类问题：属性（这实体什么状态）、分类（属于哪一类）、关系（两者有关系吗）。

结果：结构化归档在全部三类题上都是满分；两种叙事式各丢一半——旧优先的保住属性和分类（各 0.83），关系只拿到 0.33；新优先的保住关系（1.00），属性掉到 0.25、分类 0.50。也就是说，叙事式压缩无论怎么权衡，都只能覆盖一部分；结构化压缩用更少字符（243 对 298）覆盖了全部。

解读：叙事式压缩是线性的，容量有限时必然丢一段；结构化压缩把"分类、属性、关系"分开存，信息密度高得多。这对应大模型 checkpoint 压缩的已知短板：过程记得住，层级与关系最容易丢。全部是合成任务，不涉及真实模型。

一、问题

压缩的意思是用更少的空间保留更多的信息。但"保留什么"取决于压缩的形状。

大模型的上下文压缩是叙事式的：把对话按时间顺序写成摘要。人脑的压缩是归档式的：事件被切开、归类，类别被放到抽象层里，关系被保留下来。

这两种形状有什么可测的差别？如果容量一样，哪一类信息会丢？

二、设定

生成一份"项目记录"：4 个类别（工具、人员、实验、结论），每类 5 个实体，共 20 个；每个实体有一个状态属性（活跃、冻结、待定）；另有 12 条关系（"使用"或"导致"）。

三种记忆，容量都约 300 字符：

- A1 叙事式·旧优先：按时间顺序写句子，写满为止——保住早期事实，后面的关系写不进去；

- A2 叙事式·新优先：先写关系、再写近期实体，写满为止——保住关系，早期实体写不进去；

- B 结构化归档：分类层次（哪类有哪几个）、属性表（每个实体的状态，压缩成单字）、关系边（谁对谁做什么）。

然后问三类题，各 12 道：属性题（这实体当前什么状态）、分类题（属于哪一类）、关系题（这两个实体之间有关系吗）。答案只能从各自的记忆里检索。

三、结果

总体正确率：A1 是 0.67，A2 是 0.58，B 是 1.00。

分题型看：

- 属性题：A1 0.83，A2 0.25，B 1.00；

- 分类题：A1 0.83，A2 0.50，B 1.00；

- 关系题：A1 0.33，A2 1.00，B 1.00。

字符数：A1 用了 298，A2 用了 296，B 只用了 243。

四、解读

第一，叙事式压缩是线性的。记忆只能从头写到尾，容量有限就必然丢一段：要么丢关系，要么丢早期事实。A1 与 A2 就是这两种取舍，各有一个短板。

第二，结构化压缩的密度高得多。分类、属性、关系是三种不同结构，分开存就不互相挤占；同样容量下它覆盖了全部信息，还用得更少。

第三，这对应大模型 checkpoint 压缩的短板。现在的压缩只做叙事摘要，所以"发生了什么"记得住，而"属于哪一层、和什么有关系"最容易丢掉。人脑恰好相反：它先记住结构和关系，具体细节反而会忘。

五、对记忆设计的含义

如果这个对照成立，那么压缩的产物不应该只是一段摘要，而应该是两样东西：

- 一段叙事摘要：保留时间线与因果（"发生了什么"）；

- 一份结构化归档：实体表、分类层次、关系边（"属于哪里、和谁有关"）。

两者配合检索：问过程用摘要，问层级与关系用归档。这个设计可以直接测：把同一段长对话按两种方式压缩，比较三类问题的召回率——本文的实验是这条路的玩具版。

六、局限

第一，任务全是合成的：实体、关系、题目都是程序生成的。

第二，结构化归档的"抽取成本"没有计入：实验里结构是直接给出的，真实系统要先从对话里抽实体与关系，这一步会出错、也有成本。

第三，真实的摘要模型可能比 A1/A2 都强（它可以选择重要的写、不写流水账），所以数字不能直接搬到真实系统；但"叙事 vs 结构化"的密度差不会因此消失。

七、与既有工作的关系

把记忆分成"语义记忆与情景记忆"是认知科学的老区分；大模型一侧，记忆机制、知识图谱增强、检索增强、长上下文记忆库都已有大量工作。

本文的不同只在于一个公平对照：同样字符容量下，把叙事式与结构化归档放在同一张卷子上，按题型分解看谁丢什么。结论不新，但它把"为什么 checkpoint 压缩会丢关系"变成了一件可以演示的事。

八、可复现

脚本：910_memory_compression.py（主对照）、911_memory_compression_v2.py（加"新优先"对照）。依赖 Python 3 标准库，无第三方依赖。

附：标注

- **[事实]** 三种记忆的召回数字：A1 0.67、A2 0.58、B 1.00（分题型见表）；B 用 243 字符、A 用约 296~298。

- **[解释]** "叙事式是线性的、结构化密度更高"——对数字的解读，不是定理。

- **[推测]** 把压缩产物设计成"摘要 + 归档"能提升层级与关系类召回——未在真实系统验证。

- **[不声称]** 不声称提出了新的记忆架构；索引、图谱、检索都是已有方向。

> 本文由何淇创作，DeepSeek（AI）协助实验设计与代码编写；观点与责任属于作者。

第十五篇 · 最小切片与时间换维度：能力边界的两个实验

单独 DOI（中文）：https://doi.org/10.5281/zenodo.23601624

摘要

大模型的每一层有自己的权重，像一块静态的函数切片。本文问两个问题：一个"结构"（一种能力）需要多大的切片才能出现？如果给模型"时间"（循环步数），能不能换掉"维度"（宽度）？

第一组实验：扫宽度与深度，测几个合成结构的"最低配置"。结果：简单的阈值与组合规则一层 8 宽就够；算术与奇偶需要更宽的切片。第二组实验：两个切片组合时，需要的维度约等于两者中较大的那个，而不是相加。第三组实验：序列奇偶任务上，循环网络宽 2 就够处理任意长度；而前馈网络在同样训练预算下，长度到 16 就学不出来，加宽也没用。第四组实验（GPU 版）：把训练加足，16 位奇偶变得可学，24 位仍然只记住训练集；8 位加法在宽 128、浅层时最好，加深反而更差。

合起来说：能力需要结构、需要足够的维度与深度，还需要"能被训练找到"。最低维度是必要条件，不是充分条件；而"时间"可以绕开一部分维度边界——这与"层是对动态的补偿"的说法一致。全部是合成任务与玩具模型。

一、问题

把大模型的一层看成一个静态函数切片：它读当前状态，用自己的权重新算一遍，写回共享状态（残差流）。那么：

- 一种结构（能力）要出现，最少需要多大的切片？

- 切片与切片之间能不能拼起来？

- 如果给模型时间（循环步数），能不能换掉维度？

本文用四个小实验回答。全部是合成任务，不涉及真实大模型。

二、实验一：最低配置

任务：parity（8 位奇偶）、add（4 位加法）、rule（(a 且 b) 或（c 且 d)）、majority（10 位多数）。模型是 tanh 的多层感知机；扫宽度与深度，记录达到准确率 0.95 的最小宽度 w* 与最小深度 L*。

结果（SGD 训练）：parity 约需宽 16、深 3~4；add 约需宽 32、深 2；rule 与 majority 都是宽 8、深 1 就满分。

要点：结构越像"运算"（需要进位、组合、非线性），最低配置越高；简单的阈值型结构一层就够。

这里必须说明一个后来发现的问题：换成 Adam 优化器后，8 位奇偶变得很容易（宽 8、深 1 就能到 0.98）。也就是说，上表测到的"最低配置"依赖训练算法，它测的是"这个训练器能否找到解"，不完全是"表示能力的最低要求"。本文把这一点明确写出，而不是把它藏起来。

三、实验二：切片组合

取两个已经训练好的切片（各自最小配置），放到一个共享主干上、各自接一个输出头，联合训练。对照是它们各自单独训练时的准确率。

parity + rule：主干宽 16、深 3 时，parity 掉到 0.59，rule 仍 1.00；深到 4，parity 回到 0.93，rule 1.00。也就是说，rule 不需要额外维度，它装得进 parity 的维度里；瓶颈在深度。

parity + add：主干宽 32、深 3 时，两个任务都到 0.95；宽 64、深 4 时都到 1.00。

结论：组合所需维度约等于两者中较大的那个，不是相加；阈值附近要一点余量。这支持一个朴素图像：能力在一个足够大的公共维度里共存，而不是各自占一块。

四、实验三：时间换维度

任务：序列奇偶——顺序读入 T 位，每一步都预测"到目前为止的奇偶"。这是一个需要记住一个比特状态的累积任务。

循环网络：宽度固定，逐位读入，每步更新状态。结果：宽度 2 就够处理 T=8、16、24、32 的全部情况（准确率 1.00），宽度 4 到 16 也都满分。T 再长，宽度不用涨。

前馈网络：一次看全 T 位，深度固定为 4。T=8 时满分；T=16 时掉到 0.54 左右（宽加到 128 也没用）；T=24 同样学不出来。

解读：时间（循环步数）确实能换掉维度。原因也直观——奇偶只需要"翻转状态"这一个比特，循环网络用两步就能表达；前馈网络必须在一次前向里把 T 位的结构算完，长度越长越难。

五、实验四：能力边界（**GPU** 版）

把训练加足（两万条数据、一万五千步）后重测，得到更清楚的边界：

16 位奇偶：宽 64、深 2 就到 0.98；宽 128 起满分。但深 6、宽 128 或 256 的组合会崩到 0.49——不是能力不够，是训练不稳定。

24 位奇偶：所有配置都在 0.50 附近；训练损失能到 0，测试却只有一半。这是"记住训练集、没有学会奇偶"的典型表现。

8 位加法：宽 128、深 2 是 0.97；宽 256、深 2 是 0.998。加深反而更差（深 4、6 都掉）。

三点结论。第一，边界存在，但它是"可训练性边界"，不是"表示能力边界"：同一个结构换个训练预算，边界会移动。第二，深度不总是帮忙：加法任务上浅层最好，奇偶任务上过深会崩。第三，"最低维度"只是必要条件：过了维度的门槛，还得能被优化器找到。

六、解读

把这四个实验放在一起，可以得到一条更准确的表述：

能力 = 结构 + 足够的维度与深度（必要条件） + 能被训练找到（优化条件）。

同时：时间可以绕开一部分维度边界（实验三），这与"层是对动态的补偿"一致——当模型没有真实的循环时，它只能用深度去模拟时间；而如果给它循环，它可以用很少的宽度做很多步。

对当前大模型的含义：均匀深度是浪费的（实验二与实验四都显示，不同结构对"深"的需求差别很大）；"加宽加深"不是稳赚的，可能更差；按结构分配维度与深度，或者引入循环/多步计算，可能更省。

七、局限

第一，全部是合成任务与玩具模型，和真实大模型的能力不是一回事。

第二，结果对训练算法敏感：SGD 与 Adam 下"最低配置"差别很大；本文两版都报告了。

第三，种子波动明显：parity 单独训练在 0.74 到 0.97 之间波动。

第四，组合实验只用了"共享主干 + 独立头"一种构造，不代表所有组合方式。

第五，时间换维度用的是密集监督（每一步都有标签），信用分配比真实场景容易。

八、可复现

脚本：938（最小切片）、940/941（组合）、943/945（时间换维度）、946/947/960/961（能力边界）。最小切片与组合用 numpy 手写训练；时间换维度与 GPU 边界用 PyTorch（Windows 上经 DirectML 跑 AMD GPU）。

附：标注

- **[事实]** 各实验的准确率数字（最低配置、组合、时间换维度、GPU 边界）。

- **[解释]** "组合约等于 max 而不是相加""时间换维度"——对数字的解读，不是定理。

- **[推测]** 对真实大模型"按结构分配深度、引入循环更省"——玩具结论的推广，未验证。

- **[不声称]** 不声称发现了新的表示理论；这里只是把已有的玩具实验拼在一起看。

> 本文由何淇创作，DeepSeek（AI）协助实验设计与代码编写；观点与责任属于作者。

第十六篇 · 残差底与组合律：两个大模型结构实验

单独 DOI（中文）：https://doi.org/10.5281/zenodo.23609386

摘要

本篇做两个与大模型结构有关的实验。

第一个问"多模态该接什么"。用合成世界：目标依赖颜色、材质、味道三个隐藏维度；文本只给粗色，视觉给精确颜色，触觉给材质，味觉给味道。测每种模态组合的可达误差（残差底）。结果：底由"覆盖了哪些隐藏维度"决定，不由模态数量决定——补一个缺失维度，底就降一截；补全则到底；冗余视觉模态一点不降底。对应的工程含义是：接地应按"残差底"配置，不是"多接模态就好"。

第二个问"结构该共享还是隔离"。四个任务，先训三个，再只给第四个任务的数据。三种结构：单体（共享主干＋多任务头）、模块化（每任务独立网络）、路由（共享主干＋每任务适配器）。结果：单体出现灾难性遗忘（旧任务平均从 0.81 掉到 0.52），而且第一阶段就互相干扰；模块化完全不掉点；路由也不掉点、新任务好，但共享主干仍有干扰。对应的工程含义是：显式组合律（模块/路由）换来可扩展性，代价是共享性。

两个实验都只到玩具规模，不涉及真实大模型。

一、问题

我们前面的理论说：能力 = 结构 + 足够的维度与深度 + 能被训练找到；而"结构"是把函数拆开与拼回去的方式（分解与组合）。这条说法在工程上引出两个具体问题。

第一，输入端：多模态接地到底接什么？是模态越多越好，还是按"缺哪个维度补哪个"？

第二，内部结构：能力应该共享一套参数，还是各管各的？共享省参数但互相干扰，隔离不干扰但参数翻倍——中间有没有更好的点？

本篇各做一个玩具实验。

二、实验一：多模态接地的残差底

世界有四个隐藏维度：形状（2 类，与目标无关的干扰项）、颜色（6 个色阶）、材质（3 类）、味道（4 类）。目标要求精确组合：y = 颜色 × 12 + 材质 × 4 + 味道，共 72 类。

各模态能看到什么：文本给形状与粗色（颜色只分两档）；视觉给精确颜色；触觉给材质；味觉给味道。另设一个冗余视觉模态，与视觉重复。

方法：先枚举法算每种模态组合的贝叶斯最优误差（残差底），再训练一个小网络用同样的输入，看它能否达到这个底。

结果：网络在每种组合上都达到了贝叶斯底。

- 仅文本：0.9722；

- 文本 + 视觉：0.9167；文本 + 触觉：0.9167；文本 + 味觉：0.8889；

- 文本 + 视觉 + 触觉：0.7500；

- 文本 + 视觉 + 触觉 + 味觉：0.0000；

- 文本 + 冗余视觉：0.9167（与只有视觉时完全一样）。

解读：残差底由"覆盖了哪些隐藏维度"决定，不是由模态数量决定。每补上一个缺失维度，底就降一截；补全就到底。冗余模态不降底，因为它的信息已经在别处。工程含义很直接：接哪种模态，应该看目标依赖哪些维度、哪些维度还没被覆盖，而不是笼统地追求模态数量。

三、实验二：组合律显式化

四个任务：奇偶校验（8 位）、层级规则、4 位加法、多数投票。输入统一补齐到 10 维。

三种结构。单体：共享主干加四个任务头，联合训练。模块化：每个任务一个独立网络。路由：共享主干，每个任务一个适配器，任务编号已知（相当于完美的路由）。

流程分两阶段。第一阶段只训前三个任务。第二阶段只给第四个任务的数据，旧数据不再出现——这是持续学习的标准设置。

结果：

- 单体：第一阶段旧任务平均 0.809（其中奇偶只有 0.574，早期就互相干扰）；第二阶段旧任务掉到 0.515（加法从 1.00 掉到 0.29），掉了 0.294——灾难性遗忘；新任务 1.00。

- 模块化：旧任务 0.998，第二阶段仍是 0.998，完全不掉；新任务 1.00。

- 路由：旧任务平均 0.893，第二阶段仍是 0.893，不掉；新任务 0.994。但奇偶只有 0.725，说明共享主干仍有干扰。

- 参照：单体用全部四个任务的数据联合重训（需要旧数据），奇偶 0.822、规则 1.00、加法 0.871、多数 1.00。

解读：共享参数换来样本效率，但代价是干扰与遗忘；隔离参数换来稳定，但失去共享。路由是折中：新任务可以单独加、旧任务不掉，但共享主干仍会把干扰留在那里。

四、合起来看

第一个实验说：输入端要按"残差底"配置——缺哪个维度补哪个，冗余模态不解决问题。

第二个实验说：内部结构要在"共享"与"隔离"之间做取舍——显式组合律（模块、路由）换来可扩展与抗遗忘，代价是共享性；共享单体换来参数效率，代价是干扰与遗忘。

这与理论里的说法一致：结构就是分解与组合的方式；组合律越显式，系统越容易扩展，但共享的好处也越少。工程上的含义是：先算清楚"哪些能力互相干扰"，再决定哪些共享、哪些隔离——而不是一律共享或一律模块化。

五、局限

第一，两个实验都是合成任务与玩具模型，和真实大模型不是一回事。

第二，多模态实验里的"贝叶斯底"是精确可算的，这只有在合成世界才成立；真实任务只能估。

第三，组合实验里的路由是"完美路由"（任务编号已知），真实路由要自己学会，会出错。

第四，遗忘实验只测了一轮，长期多轮漂移下的行为没测。

六、可复现

实验一：993_multimodal_residual_v2.py（枚举算底 + 小网络验证）。实验二：994_composition_modular.py（单体/模块化/路由的主对照）、996_full_E.py（路由的调参版）。依赖 PyTorch（Windows 上经 DirectML 跑 AMD GPU）。

附：标注

- **[事实]** 实验一各组合的残差底与网络误差（两者一致）；实验二各架构的两阶段准确率与掉点。

- **[解释]** "残差底由覆盖的维度决定""显式组合律换可扩展性"——对数字的解读。

- **[推测]** 对真实大模型"接地按残差底配置、组合律显式化"的建议——玩具结论的推广，未在真实模型上验证。

- **[不声称]** 不声称提出新的多模态或持续学习算法；模块化、路由、回放都已有大量工作。

> 本文由何淇创作，DeepSeek（AI）协助实验设计与代码编写；观点与责任属于作者。

第十七篇 · 按计算厚度分配算力：预算与动员

单独 DOI（中文）：https://doi.org/10.5281/zenodo.23612112

摘要

本篇测"计算厚度"在工程上的第一个后果：算力该给多少、给谁。计算厚度指的是组合律在时间轴上依次应用的次数（时序组合的长度）——它是"路有多长"，不是"路口有多宽"（后者是分布厚度，另篇再谈）；它不是函数或函数类，而是一个长度。

两个实验。

实验一（预算）：世界是一个需要 k 步迭代才能到目标的映射，任务按厚度分四档（k = 1、2、4、8）。三种策略：所有任务都走 8 步、都走 4 步、按厚度路由（任务 k 走 k 步）。结果：路由只用 47% 的算力，平均误差 0.0812，与均匀 8 步（0.0834）相当甚至略好；均匀 4 步（0.0881）反而不如路由。过度计算也有害：k=1 的任务，1 步误差 0.0014，2 步 0.0160，4 步 0.0205。

实验二（动员）：四个不同世界规则，厚度同样是 1、2、4、8 步。模型是"常驻底座（一次变换，所有任务共享）加可动员循环块（按任务厚度走若干步）"。结果：均匀 8 步彻底失败（平均误差 1.756）——对 k=1 的任务，多走的步数直接算过了目标；按需动员 0.434（15 步）；各自独立 0.064（同样 15 步，但每任务一个网络）。

两条结论：第一，计算厚度必须显式匹配，均匀深度会把薄任务算坏；第二，常驻共享加按需动员能工作，但共享的动员块仍有干扰，冲突大的任务应该分开。都是玩具实验。

一、问题

我们把"厚度"分成两种：分布厚度（同一输入对应多宽的一族输出）和计算厚度（从输入走到目标要多少步）。本篇只动计算厚度：任务需要 k 步就是 k 步，答案唯一（分布厚度为零）。

在展开实验之前，要把"计算厚度"说得更精确一点。

组合律（把切片拼回函数的方式）有两种用法。一种是沿时间轴依次应用：第一步的输出喂给第二步，第二步再喂给第三步——这叫时序组合。另一种是同时发生：并行、共享状态、路由选择——这叫并列组合。计算厚度只量前者：**在时间轴上要依次应用几次组合，才能从输入走到目标。**

所以有两个函数空间要分清。

第一个是切片函数空间：被组合的对象，每一点是一个静态函数（一层、一块）。

第二个是组合律空间：起组合作用的高阶函数空间，每一点是一种"把切片拼成函数"的方式。它沿时间展开成 C^1、C^2、……——一步组合、两步组合、三步组合。

计算厚度不是函数，也不是函数类的成员，而是组合律空间里的一条最短路径长度：目标函数第一次落进某个 C^k 的像时，那个 k 就是它的计算厚度。它由目标、可用原语、组合律三者共同决定：原语越粗（一步能顶多步），同一个目标的厚度越小。

问题有两个：算力该给多少？共享还是分开？

二、实验一：算力预算

世界：x_{t+1} = tanh(A x_t)，A 是固定的随机矩阵，状态 8 维。任务：给初值 x0，预测 x_k，k = 1、2、4、8。

模型：共享循环细胞，每步 h ← tanh(W h + U x0 + b)，最后线性读出；步数就是计算量。

三种策略：均匀深（都走 8 步）、均匀浅（都走 4 步）、按厚度路由（任务 k 走 k 步）。

结果（每任务取不同宽度的最好）：

| 策略 | 平均误差 | 总步数 |

|---|---|---|

| 均匀 8 步 | 0.0834 | 32 |

| 均匀 4 步 | 0.0881 | 16 |

| 按厚度路由 | 0.0812 | 15 |

路由用 47% 的算力，误差与均匀 8 步差 0.0022，实际还略低。

过度计算单独测：k=1 时，1 步误差 0.0014，2 步 0.0160，4 步 0.0205，8 步 0.0148。多算不更好，单独看反而更差——世界是迭代映射，多走一步就过了目标。

解读：算力应该按"任务需要多少步"分配，而不是统一给一个深度；均匀深度的浪费可以测出来，约一半。

三、实验二：常驻底座与按需动员

人想难问题时不只是"想更久"，还会动员更多结构：有的回路长，有的短，有的常驻（维持、注意）。本篇把这个图像做成一个玩具结构。

四个任务，每个任务有自己的世界规则：任务 i 是 x_{t+1} = tanh(A_i x_t)，给 x0 预测 f_i^{k}(x0)，k = 1、2、4、8。

模型：常驻底座（一个线性层加 tanh，所有任务共享，走一次）加可动员循环块（共享权重，按任务厚度再走若干步），最后线性读出。

对照：均匀深（都走 8 步）、按需动员（任务 k 走 k 步）、均匀浅（都走 4 步）、各自独立（每任务一个网络，总步数与动员相同）。

| 策略 | 各任务误差（k=1,2,4,8） | 总步数 | 平均误差 |

|---|---|---|---|

| 均匀 8 步 | 1.885 / 1.853 / 1.706 / 1.579 | 32 | 1.756 |

| 按需动员 | 0.242 / 0.439 / 0.560 / 0.496 | 15 | 0.434 |

| 均匀 4 步 | 1.905 / 1.842 / 1.717 / 1.571 | 16 | 1.757 |

| 各自独立 | 0.004 / 0.019 / 0.103 / 0.130 | 15 | 0.064 |

三点解读。

第一，均匀深严重失败：对 k=1 的任务，目标在一步之后；模型硬走 8 步，等于反复迭代，算到了别的地方。步数越多不越好，甚至越差——这与实验一的过度计算一致。

第二，常驻底座加按需动员能工作：平均误差 0.43，比均匀策略好一个量级，步数只有一半不到。

第三，各自独立最好（0.06）：不是因为步数更少（同样 15 步），而是因为每个任务有自己的循环块，不受其他任务干扰。共享循环块在不同世界规则之间产生了干扰。

三点五、人脑层面的对应：接论文**-36**（第五篇；原论文**-21** 已并入）

人脑层面的解释不需要在这里重新展开。论文-36《语言与大脑：两种函数形式》（其前身论文-21《人脑的可函数化形式》已并入）已经把脑写成可函数化器官：表示、层级、内生时间、有限自指；并且指出它是混合型、结构异质——有常驻的结构（维持、注意），也有按任务临时动员的长短回路，而它本身不自足、被保住的是"身体 + 脑 + 环境"的整体。

本篇的"计算厚度"与"按需动员"，就是那个器官层面的工程投影：简单的任务少动员，复杂的任务多动员；常驻结构一直用，临时结构按需用。细节以论文-36 为准，这里只作为设计直觉，不作为本篇的论据。

四、合起来看## 四、合起来看

实验一说：算力按计算厚度分配；均匀分配浪费近一半，过度计算会算坏。

实验二说：常驻底座共享是合理的（省参数、保留共同表示），但动员部分共享有干扰；冲突大时应分开。

对应的设计问题：哪些结构常驻共享，哪些按任务动员——这正是"共享还是分离"的取舍，和前面残差底与组合律实验的结论一致。

五、局限

两个实验都是合成世界与玩具模型。动员实验里"任务需要几步"是给定的（完美动员），真实系统要自己估计。独立方案虽好但参数翻倍，没测"独立 + 少量共享"的折中。实验二只跑了一次运行，没有多种子复核。

六、可复现

实验一：1000_thickness_budget.py。实验二：1010_recruit.py。依赖 PyTorch（Windows 上经 DirectML 跑 AMD GPU）。

附：标注

- **[事实]** 两个实验的数字：路由 47% 算力、误差 0.0812；过度计算更差；均匀 8 步 1.756、动员 0.434、独立 0.064。

- **[解释]** "算力按厚度分配""常驻共享、动员分离"——对数字的解读。

- **[推测]** 人脑"常驻结构 + 长短结构"的类比——方向一致，没有证据链。

- **[不声称]** 不声称提出新的算力调度或网络架构；早退、门控、模块化已有大量工作。

> 本文由何淇创作，DeepSeek（AI）协助实验设计与代码编写；观点与责任属于作者。

第十八篇 · 层厚放在哪：异质层实验

单独 DOI（中文）：https://doi.org/10.5281/zenodo.23612126

摘要

大模型的层是均匀的：每层同宽、同结构，每个输入走同样深度。生物不是：不同区域的结构不同，有的回路短，有的长，有的常驻。

本篇问一个很窄的问题：同样的参数与算力，"层厚"放在网络的不同位置，结果一样吗？

任务：四个多任务（奇偶、层级规则、加法、多数），共享主干加各自输出头。四种结构：均匀 6×64（前向乘加约 21k）、宽前 128→16（约 15k）、宽后 16→128（约 14k）、中间厚（约 17k）。

三个种子的复核结果：宽后 16→128 平均准确率 0.995，用 14k 算力、19k 参数；均匀 6×64 是 0.980，用 21k 算力、24k 参数；中间厚 0.954；宽前 0.831。宽后分任务看提升主要来自难任务（奇偶 0.994 对 0.981，加法 0.985 对 0.941）。

结论：层厚放在哪里，比"每层都多厚"更重要。让前面几层窄（先把输入压成表示）、后面几层宽（再展开成答案），在这组任务上更省也更准；反过来（宽前）明显差。这与生物分层的异质方向一致，但我们只测了一种任务组合、几个结构，不能推广成定律。

一、问题

前一篇测了"算力给多少"（时间方向）。本篇测另一个方向：层与层之间的厚度分布（空间方向）。

大模型是均匀层厚；人脑是异质的：区域之间的结构、连接、时间常数都不同。如果厚度分布重要，那么同参数同算力下，"放在哪里"应该 measurable 地影响结果。

二、设定

任务：四个多任务——奇偶校验（8 位）、层级规则、4 位加法、多数投票。输入统一补齐到 10 维；共享主干，每个任务一个输出头，训练时任务混合。

四种结构（宽度序列）：

- 均匀：6 层 × 宽 64；

- 宽前：128 → 64 → 48 → 32 → 24 → 16；

- 宽后：16 → 24 → 32 → 48 → 64 → 128；

- 中间厚：[16, 32, 64, 96, 64, 32]。

算力用前向乘加数衡量（输入维到第一层 + 层间乘加），分别约 21k、15k、14k、17k；参数量分别约 24k、16k、19k、19k。

训练：任务混合，每步随机抽一个任务；6000 步。宽后与均匀做了三个种子，其余各一次。

三、结果

| 架构 | 前向乘加 | 参数 | 平均准确率（三种子） |

|---|---|---|---|

| 均匀 6×64 | 约 21k | 24k | 0.980（0.969 / 0.985 / 0.988） |

| 宽后 16→128 | 约 14k | 19k | 0.995（0.993 / 0.993 / 0.998） |

| 中间厚 | 约 17k | 19k | 0.954（一次） |

| 宽前 128→16 | 约 15k | 16k | 0.831（一次） |

宽后结构用约 67% 的算力、更少的参数，平均准确率反而更高。分任务看，提升主要来自难任务：奇偶 0.994 对 0.981，加法 0.985 对 0.941；规则与多数两个任务在所有结构上都是满分（太简单，分不出差别）。

宽前结构明显差（0.831），中间厚居中（0.954）。

四、解读

第一，层厚分布是可测量的：同样的算力，位置不同结果差 0.16（0.995 对 0.831）。

第二，在这组任务上，"前窄后宽"最划算：前面几层把输入压成较窄的表示，后面几层再展开到答案。一种可能的解释是：窄层先做选择（丢掉无关信息），宽层再组合（难任务需要更宽的表示），顺序对了就省。

第三，这与生物分层的异质方向一致——不同区域结构不同，不是"同一种砖摞高"。但我们只测了四个任务、四种结构，还不能说"前窄后宽"是通则。

四之二、人脑层面的对应：接论文**-36**（第五篇；原论文**-21** 已并入）

人脑层面的解释也不需要在这里重写。论文-36《语言与大脑：两种函数形式》（其前身论文-21《人脑的可函数化形式》已并入）已经把脑写成可函数化器官，并指出它是结构异质的：不同区域的结构、连接、时间常数不同，有常驻的部分，也有按任务动员的长短回路。

本篇测的"层厚放在哪"，正是这种异质性的一个最粗的工程投影：不是所有层都该一样厚；把厚度放到哪一段，结果会不一样。论文-36 给出的定性图像（异质、常驻、动员）在这里变成一个可测的问题（同算力下，宽前/宽后/均匀/中间厚哪个好）。

同样，细节以论文-36 为准；本篇只把它当设计直觉，不作论据。

五、局限

任务与模型都是玩具规模；宽前与中间厚只跑了一次运行，种子起伏可能影响结论；宽后与均匀做了三个种子。任务里的规则与多数太简单（全满分），主要区分力来自奇偶与加法。真实大模型的层不是全连接（有注意力、残差、归一化），这里的"层宽"只是最粗的近似。

六、可复现

脚本：1007_heterogeneous_layers.py（主对照）、1008_hetero_fair.py（同算力对照）、1009_hetero_seeds.py（三种子复核）。依赖 PyTorch（Windows 上经 DirectML 跑 AMD GPU）。

附：标注

- **[事实]** 四种结构的算力、参数、各任务准确率；宽后 0.995 vs 均匀 0.980，宽前 0.831。

- **[解释]** "层厚放在哪里更重要""前窄后宽更省"——对数字的解读。

- **[推测]** 与生物异质分层的类比——方向一致，没有证据链。

- **[不声称]** 不声称提出新的网络架构；宽度变化、层剪枝、条件计算都已有大量工作。

> 本文由何淇创作，DeepSeek（AI）协助实验设计与代码编写；观点与责任属于作者。

第十九篇 · **RLHF** 压平人类分歧

单独 DOI（中文）：https://doi.org/10.5281/zenodo.23611877

摘要

本篇测"分布厚度"在工程上的后果：对齐会把宽的输出分布压平。

设定沿用前面的人类分歧实验：题目分 thin（事实题，人类答案一致）与 thick（判断题，答案取决于偏好，人类有分歧）。基础模型学 P(答案|题目)，训练后冻结；thick 题上它的输出熵 0.9655，很接近人类的 1.0138——也就是说，基础模型是诚实地保留分歧的。

然后做偏好微调的玩具版：对每题从当前模型采样两个答案，让标注者按人类偏好选一个，再在选中的答案上做一步训练。

结果：thick 题的人类答案熵不变（1.0138），但模型熵从 0.9655 降到 0.4090（下降约 57%）；与人类分布的 KL 从 0.0451 涨到 0.4188（约 9 倍）；与"人类多数答案"的一致率从 0.870 升到 0.935。thin 题人类熵约为零，模型变化很小。

解读：人类的分歧没有错，但偏好微调把模型的输出分布压到了多数答案上。模型变得坚决，也更偏离人类的真实分布——少数意见被丢掉了。这是"厚的地方压不平"在工程上的具体后果：对齐把分布厚度压成确定性，代价是失去分布。玩具实验，不涉及真实大模型。

一、问题

我们把厚度分成两种：计算厚度（需要多少步）与分布厚度（输出分布有多宽）。前一篇测计算厚度；本篇只动分布厚度：题目都是单步判断（计算厚度小），变化的只是输出有多宽——人类答案熵从 0（thin）到约 1.01（thick）。

问题：对齐（偏好微调）对分布厚度做了什么？

二、设定

模拟 60 个人类，每人有一个偏好向量。thin 题：答案由输入决定，人人一致。thick 题：答案取决于偏好，人与人不同。人类答案分布的熵用来量厚度。

基础模型：小 MLP 学 P(答案|题目)；训练数据是每题抽一个人来答。因为见过很多人的答案，它学到的是人类总体分布。

偏好微调（玩具版）：对每题从当前模型采样两个答案，按人类偏好选一个（人类偏好概率高的胜出），在选中的答案上做硬标签训练一步；重复多轮。

三、结果

| 指标（thick 题） | 人类 | 基础模型 | 偏好微调后 |

|---|---|---|---|

| 答案熵 | 1.0138 | 0.9655 | 0.4090 |

| 与人类分布 KL | — | 0.0451 | 0.4188 |

| 与多数答案一致率 | — | 0.870 | 0.935 |

thin 题：人类熵约为 0，基础模型 0.1413，偏好微调后 0.1142（变化很小）。

四、解读

第一，压平是真实发生的：模型熵下降约 57%，而人类的分布没有变。

第二，压平不全是"学得更准"：与多数答案的一致率确实升了（0.870 到 0.935），但那是把少数意见的概率让给了多数答案——与人类真实分布的 KL 涨了近 9 倍。

第三，thin 题几乎不受影响：那里本来就没有分歧可压。

用我们的词说：偏好微调在做"单峰化"——把一条宽的分布压成一个点。对事实题这是好事（本来就是尖的）；对判断题这是丢失：丢掉的是"人本来就有不同意见"这件事。

五、局限

第一，这是玩具：标注者是模拟的（按人类偏好分布选），不是真人；真实 RLHF 还有 KL 正则项，压平会被约束（本篇没加）。

第二，题目是合成的（thin/thick 两类），人类偏好是程序生成的。

第三，只测了一轮偏好微调，没测多轮下的长期漂移。

六、可复现

脚本：1002_rlhf_flatten.py（软标签版，作对照）、1003_rlhf_flatten_v2.py（偏好采样版，本文数据来自这个）。依赖 Python 3 与 numpy。

附：标注

- **[事实]** thick 题：人类熵 1.0138、基础模型 0.9655、微调后 0.4090；KL 0.0451→0.4188；一致率 0.870→0.935。

- **[解释]** "对齐压平分布、丢掉少数意见"——对数字的解读。

- **[推测]** 真实 RLHF 会因 KL 正则而减轻压平——未验证。

- **[不声称]** 不声称提出新的对齐算法；KL 正则、多样性保留都已有大量工作。

> 本文由何淇创作，DeepSeek（AI）协助实验设计与代码编写；观点与责任属于作者。

第二十篇 · 空间、组合与厚度：三个资源的交换实验

单独 DOI（中文）：https://doi.org/10.5281/zenodo.23629197

摘要

前面几篇把"可表示"拆成三个资源：空间上的表示维度（宽度 S）、组合律上的表示维度（组合自由度 C）、时间上的步数（计算厚度 T）。本篇做三个实验，测这三种资源各自管什么、能不能互换。

实验一（三因子）：在空间型、组合型、厚度型任务上扫 (S, C, T)。结果：空间型任务由 S 决定（S=32 以上满分，T、C 无影响）；厚度型任务需要 S 和 T 都够（k=4 时 S=32：T=1 误差 0.185、T=4 降到 0.077）；组合型任务必须用"输入相关路由"才算组合维度——只并联相加时 C 没有作用，带路由后 C=2 在路由任务上明显有效（k=4 时 C=2、T=4 误差 0.146，C=1 同配置 0.222）。

实验二（S×T 平面）：扫 90 个配置画等误差曲线。结果：交换率不是常数——宽度比步数更值钱（k=4 误差≈0.12 时，S=64、T=3 与 S=32、T=4 相当，但 S×T 并不守恒）；步数不够有断崖（T=1 时 k=8 误差 0.468，加宽到 S=128 也只能降到 0.37）；过度计算有害（T=6→8 多数曲线回升）。图 1 是三个厚度下的曲线。

实验三（层 vs 循环）：同一个迭代任务，比较"T 个不同层"与"同一层循环 T 次"。结果：两者精度几乎相同（k=4、T=4 时：不同层 0.059，循环 0.072），但循环的参数少得多（T=8 时 1096 对 30216，约 27 倍）。也就是说，计算上"加层"和"循环"等价，差别在参数共享方式与语义。

结论：三个资源各有分工，可部分互换但汇率有偏向；循环是参数效率更高的"时间模拟"，不是唯一能实现时间的手段。人脑的"按需收敛"（一次结构试探、没收敛再循环、收敛后固化）与这些结论方向一致，但本篇不声称解释人脑。

一、问题：三个资源

我们前面把"可表示"拆成三个位置：

- 空间上的表示维度：切片内部的轴数，实验里用宽度 S 控制（单隐层 rank 上界 ≈ S）；

- 组合律上的表示维度：组合方式的自由度，实验里用分支数 C 与是否路由控制；

- 时间上的步数：迭代次数 T。按我们的口径，时间不叫"维度"，叫步数。

事先的预测（预计算版设计文档）：

| 任务 | 决定资源 | 预测边界 |

|---|---|---|

| 空间型 | S | 沿 S 的竖线 |

| 组合型 | C | 沿 C 的竖线 |

| 厚度型 | T（S 部分可换） | 斜线 |

| 空间+厚度 | S 和 T | L 形 |

| 组合+厚度 | C 和 T | L 形 |

本篇测试这些预测。

二、实验一：三因子

模型：C 个并行分支（各自宽 S、迭代 T 步），合并后读出。任务：

- 空间型：6 位输入到 64 类的任意查表函数；

- 组合型：两路组合（奇偶 XOR 多数），以及后面的路由型任务；

- 厚度型：x_{t+1}=tanh(A x_t)，预测 x_k（k=1、4）。

空间型：S=8 时准确率 0.96~0.98，S=32/64 全部 1.000；T 和 C 改变没有影响。预测"沿 S 的竖线"成立。

厚度型（k=4）：S=8 全失败（误差 0.32~0.43）；S=32：T=1 误差 0.185、T=4 0.077、T=8 0.094；S=64：T=1 0.138、T=4 0.056、T=8 0.063。需要 S 和 T 都够，且 T 过多变差——L 形趋势出现，过度计算

有害。

组合型（路由任务）：先试"并联相加"的 C 分支（无路由），C 没有任何优势——说明把多个分支加起来不等于组合维度。改成输入相关路由（softmax 选分支）后：路由+k=1 时，S=32、C=2 在 T=1 就达到 0.046（C=1 同配置 0.055）；路由+k=4 时最优 S=64、C=2、T=4 误差 0.146，对比 C=1、T=4 的 0.222（降 34%），而 C=2、T=1 是 0.443。组合维度真实有效，但必须是"输入相关选路"，并且和厚度有交互（L 形）。

三、实验二：**S×T** 平面与交换率

网格：S ∈ {8,16,32,64,128} × T ∈ {1,2,3,4,6,8}，三个厚度 k=2、4、8，共 90 个配置（6 进程并行，约五分钟）。图 1 是结果曲线。

读图与数据：

第一，宽度决定误差能降到多低。k=8 时 S=8 一直停在 0.6 左右，S=128 能到 0.17。

第二，步数决定降得多快。所有曲线在 T=1 到 T=2 之间大幅下降，之后趋于平坦。T 不够有断崖：k=8、T=1 时误差 0.468，即使 S=128 也只能到 0.37。

第三，交换率不是常数。若按 S×T ≈ 常数，S=64、T=4 与 S=32、T=8 应该接近；实测前者 0.056、后者 0.094。在等误差点上，宽度比步数更值钱：k=4 误差≈0.12 时，(S=64, T=3) 与 (S=32, T=4) 相当。也就是说，"时间换维度"存在，但汇率偏向空间——不是一维换固定几步。

第四，过度计算有害。T 超过 4~6 后误差回升：k=2 时 T=8 的 0.045 差于 T=3 的 0.042；k=4、S=32 时 T=8 的 0.125 差于 T=6 的 0.107；k=8、S=128 时 T=8 的 0.180 差于 T=6 的 0.170。

四、实验三：层 **vs** 循环

同一个迭代任务（x_k，k=2、4、8），两种模型：T 个不同层（结构差异），或同一层循环 T 次（时间）。

结果（S=64）：

k=4	T=1	T=2	T=3	T=4	T=6	T=8
不同层	0.349	0.152	0.103	0.059	0.052	0.064
循环	0.349	0.152	0.097	0.072	0.065	0.070

| k=8 | T=1 | T=2 | T=3 | T=4 | T=6 | T=8 |

|---|---|---|---|---|---|---|

| 不同层 | 0.496 | 0.325 | 0.251 | 0.193 | 0.154 | 0.165 |

| 循环 | 0.502 | 0.325 | 0.231 | 0.198 | 0.174 | 0.177 |

两者精度几乎相同，不同层甚至略好。但参数：T=8 时循环 1096，不同层 30216，约 27 倍差距。

解读：计算上"加层"和"循环"是同一件事——都是多走几步；差别在参数共享：循环用一套参数反复用，层用每层独立参数。所以"循环才是时间"这句话要修正为：循环是参数效率更高、语义上对应"同一规则沿时间反复"的时间实现；层可以模拟出时间步，代价是更多参数。

五、解读

第一，三个资源各有分工：空间决定"能装多复杂的一步"；组合决定"能选多少种接法"；步数决定"能展开多长"。互换了，但汇率有偏向，且任务需要哪种资源，边界就在哪个方向。

第二，过度计算有害这一点在三个实验里都出现（三因子、S×T 平面、层/循环），与论文-23 的结论一致：步数不是越多越好，超过任务厚度会走到别处。

第三，和脑的类比：一次结构运算试探，没收敛就带记忆循环，收敛了就固化并缩小范围（按需收敛）。层/循环对照说明这条类比在计算上是可行的——循环省参数，生物用时间换空间是合理的；但这只是方向一致，不是证据。

六、局限

第一，全部是合成任务与玩具模型；边界形状依赖具体任务族。

第二，组合维度的结论依赖路由实现：只并联相加时 C 无效，带路由才有效，说明"组合维度"的工程含义要落到具体机制上。

第三，S×T 平面的绝对误差受模型架构细节影响（本实验与三因子实验的模型略有差异，故看趋势不看绝对值）。

第四，层/循环只测了一个任务族（迭代映射）和一个宽度（S=64）。

第五，交换率只在三个厚度上测，且 T 的网格不完全均匀。

七、可复现

三因子：1040_three_factors.py、1041_three_factors_composition.py、1042_three_factors_router.py。S×T 平面：1043_plane_worker.py（多进程分片）。层/循环：1044_layers_vs_loops.py。画图：1050_plot_plane.py（纯 Python 生成 SVG，Edge 转 PNG）。图 1 见同目录 `图-SxT平面曲线.png`。依赖 PyTorch（Windows 经 DirectML 跑 AMD GPU）。

附：标注

- **[事实]** 三个实验的数字与图 1 的曲线：S 决定空间型、S+T 决定厚度型、路由 C 有效、交换率非常数、过度计算有害、层与循环精度相当而参数差 27 倍。

- **[解释]** "三个资源各有分工""循环是参数效率更高的时间模拟"——对数字的解读。

- **[推测]** 人脑按需收敛的类比——方向一致，无证据链。

- **[不声称]** 不声称提出新架构或新算法；门控、路由、循环网络、早退都已有大量工作。

> 本文由何淇创作，DeepSeek（AI）协助实验设计与代码编写；观点与责任属于作者。

第二十一篇 · 层的路径与冗余：**GPT-2** 上的层扰动实验

单独 DOI（中文）：https://doi.org/10.5281/zenodo.23663369

摘要

本篇在真实小模型上测一个问题：大模型里的结构是"压在某一层里"，还是"跨几层展开、带冗余"？

做法：在 GPT-2 small（12 层）上，把某一层"跳过"（输出等于输入，保留残差流），分别测单层、相邻两层、连续三层的扰动效果；语料为 tinyshakespeare，300 条 64-token 序列，9600 个预测位置；指标是下一 token 的 top-1 翻转率与损失增加，并按下一个词的类别（数字/标点/功能词/情绪词/身体词/其他）分组。

结果：单层跳过第 1~10 层，top-1 翻转率约 0.20~0.31；跳过最后一层为 0.55；跳过第 0 层为 0.96（异常，需分开看）。相邻两层一起跳过，翻转率升到约 0.37~0.68，比"两个单层中较大的那个"还高约 0.10~0.17——这是冗余的签名：单独缺一层有人补，两层一起缺就没人补了。连续三层跳到约 0.44~0.77。类别之间差异不大，身体词与情绪词在中层略敏感。

结论：GPT-2 的结构是跨层分布且带冗余的。单层扰动会低估结构的重要性（用户提出的"去掉单层只影响完整性"得到支持）；按层做"判定"或"早退"时，判据应当看路径而不是单层。所有结果来自小模型与玩具语料，不声称适用于大模型。

一、问题

前面的框架说：结构是切片 + 组合律；复杂结构要靠时间上的步数展开。如果这条在真实模型里也成立，那么：

1. 一个概念/结构不应只对应某一层，而应跨几层展开；

2. 层与层之间可能有冗余：去掉一层，别的层能补；

3. 因此"单层扰动"会低估结构的重要性。

用户的直觉是：**单层之间也有结构，多层之间也有结构；去掉单层可能只影响完整性，因为有冗余。**本篇就是去测它。

二、方法

- **模型**：GPT-2 small（12 层、宽度 768、124M 参数），DirectML GPU。

- **语料**：tinyshakespeare，随机取 300 条、每条 64 token；预测位置为每个 token（间隔 2），共 9600 个。

- **扰动**：把某些层"跳过"——该层输出 = 输入（恒等直通），保留残差流，只去掉这一层的贡献。这样才是"少算一层"，而不是把信息清空。

- **配置**：单层 12 个；相邻两层 11 个；连续三层 10 个。

- **指标**：相对基线的 top-1 翻转率、损失增加。

- **分组**：按"下一个 token"的类别分组：数字、标点、功能词、情绪词、身体词、其他。

三、结果

单层跳过（top-1 翻转率，按类别）：

| 层 | 数字 | 标点 | 功能词 | 情绪 | 身体 | 其他 | 均值 |

|---|---|---|---|---|---|---|---|

| 0 | 0.986 | 0.833 | 0.974 | 1.000 | 1.000 | 0.978 | 0.962 |

| 1 | 0.214 | 0.185 | 0.195 | 0.167 | 0.271 | 0.199 | 0.205 |

| 2 | 0.209 | 0.195 | 0.223 | 0.278 | 0.333 | 0.231 | 0.245 |

| 3 | 0.245 | 0.214 | 0.252 | 0.208 | 0.312 | 0.252 | 0.247 |

| 4 | 0.250 | 0.199 | 0.213 | 0.208 | 0.292 | 0.231 | 0.232 |

| 5 | 0.291 | 0.198 | 0.221 | 0.250 | 0.375 | 0.239 | 0.262 |

| 6 | 0.218 | 0.188 | 0.217 | 0.250 | 0.292 | 0.236 | 0.233 |

| 7 | 0.255 | 0.195 | 0.253 | 0.250 | 0.229 | 0.239 | 0.237 |

| 8 | 0.277 | 0.199 | 0.228 | 0.264 | 0.396 | 0.230 | 0.266 |

| 9 | 0.268 | 0.167 | 0.255 | 0.264 | 0.396 | 0.236 | 0.264 |

| 10 | 0.355 | 0.222 | 0.263 | 0.319 | 0.438 | 0.277 | 0.312 |

| 11 | 0.482 | 0.510 | 0.517 | 0.611 | 0.708 | 0.479 | 0.551 |

- 第 0 层跳过几乎全崩（0.96）：第一层被跳过时整条链的输入分布就变了，需单独说明（见局限）。

- 第 1~10 层：单层跳过只翻 20%~31%，说明**多数单层不是致命的**。

- 最后一层（11）：0.55，明显更重要——输出前的变换不能省。

相邻两层跳过（组翻转率 vs 单层最大）：

| 组 | 组均值 | 单层最大均值 | 冗余量 |

|---|---|---|---|

| (1,2) | 0.400 | 0.246 | 0.155 |

| (2,3) | 0.423 | 0.262 | 0.160 |

| (3,4) | 0.417 | 0.248 | 0.169 |

| (4,5) | 0.387 | 0.263 | 0.125 |

| (5,6) | 0.368 | 0.262 | 0.106 |

| (6,7) | 0.375 | 0.247 | 0.128 |

| (7,8) | 0.387 | 0.271 | 0.116 |

| (8,9) | 0.412 | 0.271 | 0.141 |

| (9,10) | 0.457 | 0.312 | 0.145 |

| (10,11) | 0.681 | 0.551 | 0.130 |

- 每对相邻层都出现正的冗余量（约 +0.10~+0.17）。

- 含义：**单独缺一层，别处能补；两层一起缺，就没人补了。这正是"冗余"的结构签名。**

连续三层跳过：

| 组 | 各类别翻转范围 | 均值 |

|---|---|---|

| (1,2,3) | 0.475~0.729 | 0.584 |

| (3,4,5) | 0.446~0.625 | 0.516 |

| (4,5,6) | 0.382~0.514 | 0.436 |

| (6,7,8) | 0.444~0.562 | 0.498 |

| (9,10,11) | 0.670~0.875 | 0.774 |

- 三层一起跳，翻转率到 0.44~0.77，明显高于任意单层。

- 结构不是"压在一层"，是**跨层展开**的。

类别差异： 多数层内类别差异在 0.1 以内；身体词、情绪词在中层略更敏感（例如第 5/8/10 层身体词 0.375~0.438）。这说明概念对层的依赖有一点选择性，但不像"专用层"那样干净。

四、解释

1. **结构跨层展开**：单层扰动温和、成组扰动急剧上升——一个结构由几层共同完成，缺一层还能靠其余层部分完成。

2. **冗余**：相邻层的正冗余量（+0.10~+0.17）说明层与层之间存在部分可替代性。用户说的"去掉单层只影响完整性"在数据上得到支持。

3. **单层 ablation 的低估**：只看单层会得出"这些层都不重要"的错误印象；真实结构的重要性藏在路径与组合里。

4. **按路径判定**：如果要做判定器/早退，判据应看"这段路径够不够"，而不是"这一层够不够"。

五、可检验推论

1. 冗余量应随模型规模、训练分布变化：在更偏向"必须逐步计算"的任务上，冗余应减少。

2. 有冗余的相邻层，应该可以用"共享参数的循环"部分替代而不掉点——这正好接上"循环比加层省参数"的已有结果。

3. 概念选择性应表现为"路径差"而不是"单层差"：不同概念的关键层组合应不同。

4. 若在真实大模型上重复，单层扰动应同样低估重要性；成组扰动的冗余签名应依然出现。

六、局限

第一，只有 GPT-2 small 与 tinyshakespeare，规模小、语料窄，不能外推到真实大模型。

第二，第 0 层跳过异常（0.96）：跳过第一层等于去掉第一层注意力的输出，整条链的输入分布变化过大；这个数字不宜与其他层并列解释。

第三，"跳过"是推理期扰动，不等于"训练时没有这层"；训练时若少一层，网络会重新分配功能，结论会不同。

第四，类别词表很粗（数字/标点/功能词/情绪/身体/其他），"概念选择性"只测到很粗的粒度；更细的概念（句法结构、语义角色）没有测。

第五，只测了 top-1 翻转与损失，没有测表示空间的变化；冗余也可能表现为"预测不变但内部表示变了"。

第六，没有与"层与循环"的替代实验直接衔接——那是下一步。

附：标注

- **[事实]** 表中所有翻转率与损失来自本次 GPT-2 实验（数据文件：数据-GPT2层路径.json）。

- **[解释]** "结构跨层展开""相邻层冗余""单层扰动低估重要性"——对数据的解读。

- **[推测]** 四条可检验推论——方向性，未验证。

- **[不声称]** 不声称适用于大模型；不声称解释了概念表征；不声称"层冗余"等于"可无代价删层"。

> 本文由何淇创作，DeepSeek（AI）协助整理与写作；观点与责任属于作者。

第二十二篇 · 语言结构作为自变量：**GPT-2** 上的三组实验

单独 DOI（中文）：https://doi.org/10.5281/zenodo.23665217

摘要

前面的实验（论文-33、以及本篇的早期版本）是从"脑结构"出发设计的：层、循环、判定、维持。但这些机制不是大模型被训练出来的方式——大模型是被语言塑形的。本篇换一个自变量：**语言结构强度**（用完整模型预测的熵作为代理，分低/中/高熵三组），重做三组实验。

第一组，层冗余 × 语言结构：跳过单层或相邻两层，看 top-1 翻转率。结果：低熵（语言可预测）位置的翻转率明显低于高熵位置（单层 0.49~0.50 vs 0.76~0.77；两层 0.48~0.53 vs 0.74~0.76）。语言结构强的位置更容忍删层。

第二组，判定器特征对比：同一预算下比较四种 halt 特征——A 置信度、B 跨层路径一致性、L 语言结构特征（词类、位置、重复、首层熵）、A+L。结果：L 在较深的预算下最好（深度 8：0.161 vs A 0.153）；路径特征 B 没有优势。

第三组，按语言结构早退：用 L 特征决定退出，在相同平均深度下比按置信度早退略好（深度 8~10 高出约 0.5~0.8 个点）。

结论：**"语言结构强度"是比"脑机制"更合适的自变量**——语言越可预测，位置越早收敛、越容忍删层、越适合早退；而"内部路径是否稳定"这个脑结构式的判据，在语言层框架下并不带来额外信息。所有结果来自 GPT-2 small 与 tinyshakespeare，不声称适用于大模型。

一、为什么换自变量

前面我们一直把大模型当"一个器官"来测：它有几层、能不能循环、何时收敛、能不能早退。这些问法来自对大脑结构的类比。但大模型不是被脑机制训练的，它是被语言训练的——语言的结构（句法、语义、叙事、常识）塑形了它的表示与组合方式。

所以更合适的自变量是**语言结构强度**：一个位置在语言上有多可预测、结构有多强。我们用它重做三组实验，检验一个统一的说法：

> 语言结构强的位置，会更早收敛、更容忍删层、更适合早退。

二、方法

- **模型**：GPT-2 small（12 层、768 宽、124M），DirectML GPU。

- **数据**：tinyshakespeare，300 条 64-token 序列，取预测位置（间隔 2），共 9600 个。

- **语言结构强度**：用完整模型在该位置的下一个词分布熵作为代理，取三分位分三组（低熵=可预测、中熵、高熵=不可预测）。

- **实验 A（层冗余）**：跳过单层（2/6/10）与相邻两层（2-3/6-7/10-11），统计相对基线的 top-1 翻转率，按熵组比较。

- **实验 B（判定器）**：每层训练 halt 判定器，特征四组：A=置信度/熵；B=跨层路径（相邻层一致、KL、趋势、cos 变化）；L=语言结构（词类、位置、重复、首层熵）；A+L。标签=该层预测是否正确。比同平均深度下的准确率。

- **实验 C（早退）**：用 B 中学到的判定器做退出策略，比较不同特征下的准确率—深度曲线。

三、结果

3.1 实验 **A**：层跳过 × 语言结构（**top-1** 翻转率）

跳过层	低熵（可预测）	中熵	高熵（不可预测）
[2]	0.491	0.694	0.771
[6]	0.502	0.690	0.763
[10]	0.498	0.689	0.763
[2,3]	0.484	0.653	0.737
[6,7]	0.534	0.698	0.760
[10,11]	0.697	0.911	0.957

- 所有配置下，低熵组的翻转率都明显低于高熵组（典型：单层 0.49 vs 0.77）。

- 含义：**语言结构强的位置，删掉一两层也不影响预测**——冗余（或语言先验的补位）集中在这里。

- 末两层（10,11）的跳过对所有组都更伤，但低熵组仍更稳。

3.2 实验 **B**：判定器特征对比（同平均深度下的准确率）

深度	A 置信度	B 路径	L 语言	A+L	B+L
3	0.149	0.148	0.150	0.147	0.148

| 5 | 0.154 | 0.147 | 0.153 | 0.151 | 0.153 |

| 8 | 0.153 | 0.152 | **0.161** | 0.159 | 0.159 |

| 10 | 0.158 | 0.156 | 0.160 | 0.160 | 0.159 |

- 浅预算（深度 3~5）：各特征基本持平。

- 深预算（深度 8~10）：L（语言结构特征）最好（0.161 vs 0.153），A+L、B+L 次之。

- **B（跨层路径）始终没有优势**——我们早期"判定要看路径"的预测，在语言层框架下依然不被支持。

3.3 实验 **C**：按语言结构早退

- 用 L 特征做退出判定：在相同平均深度（8~10 层）下，准确率比按置信度早退高约 0.5~0.8 个点。

- 与完整模型（12 层，准确率约 0.169）相比，早退到平均 10 层可保住约 0.16（掉不到 1 个点）。

- 与实验 A 一致：语言结构强的位置，退出更安全。

四、解释

1. **语言结构强 ⇒ 早收敛**：可预测的位置，模型不需要深层计算，语言先验几乎决定了答案；不可预测的位置需要更多层，且更难判对。

2. **语言结构强 ⇒ 冗余高**：删层对低熵位置影响小，因为其他地方（语言先验、冗余层）能补位；这解释了论文-33 里"单层可跳、成组才崩"的现象——冗余不是均匀分布的，它集中在语言结构强的地方。

3. **为什么路径特征没用**：在大模型里，"收敛"不是内部状态的稳定，而是"语言层面已确定"——所以置信度（≈语言可预测性的读数）已经够用，内部路径的稳定性没有额外信息。

4. **判定器要看语言层**：与其看"内部算完了没有"，不如看"语言结构够不够强、够不够确定"——这是把"判定"从机制层移到语言层。

五、可检验推论

1. 语言结构强（低熵）的位置，早退掉点应显著小于高熵位置——可在更难的语料上复测。

2. 层冗余的分布应随语料变化：语言结构更强的领域（如程式化文本）冗余应更高。

3. 若判定器加入更强的语言结构特征（依存句法、上下文重复模式），应进一步接近 oracle。

4. 换更大模型：语言结构强度的作用应保持，且高熵位置的"需要更深"应更明显（因为大模型的深层能力更强）。

六、局限

第一，模型极小（GPT-2 small）、语料窄（tinyshakespeare），整体准确率仅 17%，不能外推到真实大模型。

第二，语言结构强度用"熵三分位"做代理，比较粗；词类/位置/重复特征也很粗。

第三，熵是按位置算的，同一位置的"语言结构"其实依赖上下文长度与质量，未做更细的控制。

第四，判定器训练标签是"预测是否正确"，与"退出后最终答案是否正确"不完全等价。

第五，我们没有测真正的语言结构标注（依存树、篇章结构），只用统计代理。

第六，这是对"语言层"的初步实验设计，结论是方向性的；我们对大模型的理解有限，不声称超过已有的 early-exit / 语言模型分析研究。

七、与已有工作的位置

- early exit / adaptive computation：本实验的贡献是用"语言结构强度"分组，而不是按层或按 token 类别分组。

- 语言模型的内部分析（探针、熵、可预测性）：本实验把"可预测性"当作自变量，而不是当作解释变量。

- 与论文-33（层路径与冗余）的关系：论文-33 说明结构跨层展开且带冗余；本篇说明冗余集中分布在语言结构强的位置。

附：标注

- **[事实]** 三张表的数字来自本次 GPT-2 实验（数据文件：数据-语言层实验.json）。

- **[解释]** "语言结构强⇒早收敛/冗余高/可早退"——对数据的解读。

- **[推测]** 四条可检验推论——方向性，未验证。

- **[不声称]** 不声称适用于大模型；不声称路径特征永远无用（只在本次设置下没优势）；不声称"语言结构"已被严格定义。

> 本文由何淇创作，DeepSeek（AI）协助整理与写作；观点与责任属于作者。

第四部分 执行器工程

第二十三篇 · 门控与稀疏执行：一组工程观察（汇总版·待复核）

单独 DOI（中文）：https://doi.org/10.5281/zenodo.23858151

摘要

执行器工程研究的是一个工程问题：**能不能让模型在推理时"少算"，而质量不明显下降？**

我们的回答是：能，但有明确边界。主要观察如下：

1. **门控训练可用于从零训练的模型**：每层一个内容门 σ(W·h+b)，用拉格朗日乘子把平均开启层数压到目标。在 GPT-2 small（12 层）上，门控模型平均使用 7.48 层，PPL 12.03（plain 对照 16.35，plain 未收敛，不作"门控更优"声称）。

2. **"保留 K/V 的稀疏执行"与稠密门控精确等价**：只算门开 token 的 Q、注意力输出行与 MLP，K/V 全保留；实测 PPL 12.032 vs 12.033（无损）。这证明门控本身不伤质量，质量差异来自"执行方式"。

3. **推理端转 MoD 容量路由，质量几乎无损**：门开优先 + 每层固定容量 K_max + padding，factor=1.0 时名义省 37.9% 计算，PPL 12.107（+0.6%）。

4. **真实加速未取得**：自写 gather/索引实现下，batch 32 墙钟 0.89~1.01x；早期一次实现曾测到 1.22x，说明加速高度依赖实现，非本文可结论。

5. **注意力子集化（跳过 token 不产生 K/V）代价明显**：在已训练模型上 +6.8% PPL；从零训练两次尝试均崩（PPL 350.99 / 393.71）。

6. **规模与训练方式的影响**：从零训练时，1.2M、124M、1.5B 模型都能以约 44%~50% 的层数工作，代价 1%~7%；而预训练模型 + LoRA 后训练（3B/7B）层数压不动、PPL 反升。关键变量是训练方式，不是参数量。

一、问题与背景

1.1 为什么关心"少算"

Transformer 每生成一个 token 都要过完所有层。若某些 token 或某些层对该 token 而言是冗余的，跳过它们就能省算力。这条思路在文献中有多个版本：MoD（Mixture-of-Depths，容量路由）、早退（early exit）、条件计算（conditional computation）、稀疏专家（MoE）等。

我们关心的是其中的"层门控"：**每层、每个 token 一个开/关决策，决定这一层是否对该 token 施加变换。**

1.2 我们的术语（先定义，后文都用这个含义）

- **内容门**：每层一个线性层 W_l，输入该 token 当前的隐藏状态 h，输出 $g = \sigma(W_l \cdot h + b)$。因此门是"每层一个 W_l，但每个 token 每层各有决策"；token 经过每层的矩阵变换后 h 改变，下一层的门分数也随之改变。

- **门开（$p > 0.5$）**：这一层对该 token 计算并写回。

- **门关（$p \leq 0.5$）**：跳过这一层，该 token 的表示保持残差直通。

- **K/V**：注意力的记忆，所有 token 都产生并保留（我们的"保留 K/V"版本）。

- **Q 侧更新**：每个 token 自己的查询与注意力输出；门关时这一部分被跳过。

- **稀疏执行**：按门的决策，只对门开 token 计算 Q/注意力输出/MLP，而 K/V 始终全保留。

- **容量路由（MoD 风格）**：每层固定容量 K，按门分数取 top-K 个 token 进入该层计算（跨层容量不同）。

- **子集化**：更强的一种稀疏——被跳过的 token 不产生 K/V，注意力只在被选中的子集内计算。

1.3 与已有工作的关系

我们不声称提出新方法。门控、稀疏执行、容量路由与早退同属条件计算一类；MoD 已报告过容量路由的可行性。我们的工作是：在有限的算力与数据条件下，把"门控训练 + 推理端稀疏执行"这条工程路线走一遍，记录哪些可行、哪些不可行、边界在哪里。

二、方法

2.1 内容门 + 拉格朗日预算

每层一个内容门。训练时用拉格朗日乘子 λ 控制平均开启比例：损失 = 语言建模损失 + $\lambda \cdot$ 平均活跃层数；λ 按"当前平均活跃 − 目标"自适应调整。这样门不需要人工指定"哪层该关"，由训练自己学。

2.2 两种执行方式

- **保留 K/V 的稀疏执行**：K/V 全算；门开 token 的 Q/注意力输出/MLP 才算。

- **容量路由（MoD 风格）**：每层固定容量 K，按分数选 top-K token；其余 token 走残差。为保证形状静态，用 padding 席位固定 K_max，门开优先填满，不足的空位只算不写回。

2.3 实验设置（统一说明）

- 数据：tinyshakespeare（约 1MB）或 TinyStories 切片（20MB/100MB）。

- 评测：验证集 PPL；以 best 记录（早停，避免过拟合干扰）；单种子。

- 硬件：阿里云 PAI-DSW，单卡 V100 16GB 或 A10 24GB。

三、实验与结果

3.1 小模型（**6** 层，字符级，约 **1.2M** 参数）

早期实验，从头训练：

- plain 3 种子均值 PPL 6.103（6.232 / 5.999 / 6.083）；

- 内容门 + 拉格朗日目标 3 层：PPL 6.028，平均 2.97/6 层（省约一半）；

- 位置门（每层×位置固定表）：目标 3 层 → PPL 6.038，平均 2.98 层；

- λ=0 自由训练时，24 层小模型自己选择 5.3/24 层。

观察：小模型上门控可以用一半层数而不降质量；门是自己学出来的，不是人工指定。

3.2 GPT-2 small（**12** 层，**124M**），从零训练

1MB 莎士比亚：

| 组 | best PPL | 步数 | 平均层数 |

|---|---|---|---|

| plain | 44.03 | 1500 | 12 |

| joint_gate | 47.01 | 1000 | 5.30（44%） |

代价 +6.8%。门控在 λ≈0 时自己收敛到约 5.3 层。

100MB TinyStories：

| 组 | best PPL | 步数 | 平均层数 |

|---|---|---|---|

| plain | 4.892 | 9000 | 12 |

| joint_gate | 5.023 | 9000 | 5.81（48%） |

代价 +2.7%。每层活跃率（final）：`1.00, 0.82, 0.61, 0.01, 0.12, 0.35, 0.28, 0.25, 0.25, 0.45, 0.75, 0.74`。第 0 层全开、第 3 层几乎不用、末两层回到 0.74~0.75。

观察：数据变多（1MB → 100MB）时，层的使用比例几乎不变（56% → 52%），但代价变小（6.8% → 2.7%）。也就是说，数据多让门控"省得起"，没有让门控"省更多"。

3.3 Qwen2.5-1.5B（28 层），从零训练，100MB

| 组 | final PPL | best PPL | 平均层数 |

|---|---|---|---|

| plain | 6.301 | 6.501 @6500 | 28 |

| joint_gate | 6.373 | 6.113 @6500 | 12.32（44%） |

final 口径代价 +1.1%；best 口径门控组反而更低。每层活跃率显示**最后 9 层（L19~L27）几乎全部关闭**。

观察：模型放大到 1.5B，门控依然便宜（44% 层数、+1.1%）。这与"越大的模型越难压缩"的直觉相反，也说明关键变量是"是否从头训练"。

3.4 预训练模型 + LoRA 后训练：压不动

3B（36 层）与 7B（28 层），加载预训练权重，LoRA r=16，数据 1MB，2000 步：

| 设置 | plain best | 门控 best | 用的层 | 代价 |

|---|---|---|---|---|

| Qwen2.5-3B 目标 33 | 12.61 | 14.94 | 32.79/36（91%） | +18% |

| Qwen2.5-7B 目标 26 | 7.01 | 9.71 | 25.70/28（92%） | +38% |

| Qwen2.5-7B 目标 24 | 7.01 | 12.93 | 24.01/28（86%） | +85% |

| Qwen2.5-7B 目标 22 | 7.01 | 12.79 | 22.27/28（80%） | +82% |

7B 目标 26 组后段还发生门控塌缩（PPL 升到 1243，层数掉到 20）。

观察：预训练权重 + 轻量后训练 + 小数据，层数压不动、质量反升。这不构成"大模型不能压缩"的结论，只能说明"这种后训练方式不行"。

3.5 GPT-2 门控模型 **+** 推理端 **MoD**（容量路由）

在 3.2 的 GPT-2 门控模型（7.48/12 层，PPL 12.03）上测试多种推理方式：

| 推理方式 | PPL | 相对 dense | 平均活跃 | 说明 |

|---|---|---|---|---|

| dense gated（参照） | 12.033 | — | — | 全部计算 |

| open_only（只算门开 token，K/V 全留） | **12.032** | **等价** | 0.622 | 动态形状 |

| open_cap（门开优先 + 容量上限） | 12.176 | +1.2% | 0.592 | 动态形状 |

| global top-k（全局 top-k） | 12.182 | +1.2% | 0.624 | 动态形状 |

| static padded（K_max + mask） | 12.046~12.107 | +0.1%~+0.6% | 0.62~0.77 | 静态形状 |

K_max 余量扫描（dense 参照 12.034；K_max = 平均门开数 × factor）：

| factor | PPL | 平均活跃 | 名义省算力 |

|---|---|---|---|

| 1.35 | 12.046 | 0.767 | 23.3% |

| 1.25 | 12.054 | 0.730 | 27.0% |

| 1.15 | 12.051 | 0.691 | 30.9% |

| 1.05 | 12.098 | 0.649 | 35.1% |

| **1.00** | **12.107** | **0.621** | **37.9%** |

| 0.95 | 14.458 | 0.592 | 40.8% |

观察：质量拐点在 factor 1.0~1.05；质量优先的落点是 factor=1.0，名义省 37.9%、PPL +0.6%。

真实墙钟（重要负结果）：

| 实现 | batch | dense | 稀疏 | 加速 |

|---|---|---|---|---|

| 早期 query 端稀疏 | 1 | 18.9 ms | 20.5 ms | 0.93x |

| 同上 | 8 | 19.7 ms | 20.8 ms | 0.95x |

| 同上 | 32 | 43.4 ms | 35.5 ms | 1.22x |

| 静态填充实现 | 32 | 38.9 ms | 41.7~43.4 ms | 0.89~0.93x |

| 静态填充实现 | 64 | 73.9 ms | 73.5 ms | 1.01x |

自写实现是 Python 级 gather + 自定义注意力，远慢于 PyTorch 融合 kernel。早期 1.22x 与后期 0.89x 的差异说明：**墙钟结果高度依赖实现细节，不能作为稳定结论**。

3.6 注意力子集化：失败

(a) 在已训练模型上直接切换子集化语义（跳过 token 不产生 K/V）：

| 容量 | PPL | 相对 dense |

|---|---|---|

| quantized（名义省 37.5%） | 12.855 | +6.8% |

| half（45.8%） | 14.030 | +16.6% |

| quarter（68.75%） | 18.692 | +55% |

(b) 从零训练子集化（两次尝试）：

| 尝试 | plain PPL | 子集化 PPL | 活跃率 |

|---|---|---|---|

| 门 + 阈值式动态子集 | 19.20 | 350.99 | 塌到 8.3% |

| 门当路由器 + 固定容量 top-k + 课程 | 19.03 | 393.71 | 0.496（容量未塌） |

(c) 短微调恢复：400 步、lr 5e-5，PPL 崩到 136（超参不当，不能作为"微调无效"的证据）。

观察：注意力子集化在我们这套门控路由上代价明显；从零训练更差。但这不是"子集化本身不可行"的结论——MoD 原方法用独立路由器 + 容量路由，在更大规模上验证过该语义。

3.7 分支独立门控：失败

每层两个门（注意力门、MLP 门分开），从零训练 12000 步：final PPL 14.484（注意力门 0.61、MLP 门 0.47）；作为对照，联合门控（一个门管两分支）4000 步就到 12.033。分支独立门控即使训练 3 倍步数仍明显更差。

分支稀疏拆分评测（在分支独立门控模型上，用 top-k 强制压低单边）：

| 测试 | PPL |

|---|---|

| baseline | 14.486 |

| MLP 压到 25% | 558.95 |

| 注意力压到 25% | 69.45 |

| 两边都压 25% | 2195.20 |

观察：在这个模型上，"注意力更新"比"MLP 更新"更耐稀疏。结合"保留 K/V 无损、子集化伤质量"的结果，初步图景是：**MLP（权重承载的变换）与 K/V（记忆）是承重部分，Q 侧更新相对可省**。但这只是单模型观察，不能当结论。

四、讨论

4.1 为什么**"**保留 **K/V"**明显更好

[解释] 注意力是跨 token 的信息通路。被跳过的 token 若从注意力中消失，后续 token 少看到一部分上下文，语言建模难以补偿。门控只决定"是否更新该 token 的表示"；保留 K/V 时，更新与否不影响其他 token 可访问的信息。这是我们看到"保留 K/V 无损、子集化 +6.8%"的直接原因。

4.2 为什么从零训练与后训练差别这么大

[解释] 从零训练时，模型从一开始就在"可能跳层"的约束下学习，可以把计算集中到少数层（1.5B 的最后 9 层几乎全关就是证据）。预训练模型没有这个机会：每一层都被预训练"用惯了"，LoRA 低秩更新改不动这个结构。

4.3 名义省算力 **vs** 真实加速

[事实] 我们测到的是名义计算量（层数×token 比例）。真实墙钟取决于 kernel 实现、批大小、内存访问；我们自写的实现在 batch 32/64 上为 0.89~1.22x，不能作为稳定结论。真实加速是独立的工程问题，超出本实验条件。

五、局限

- 单种子为主；数据规模有限（1MB~100MB）；plain 在部分实验中未收敛；

- 从零训练组的绝对 PPL 不代表模型真实能力（欠训练）；

- 仅 PPL，无下游任务；

- 墙钟测量受实现影响大；

- 未与 MoD 原文做同设置对比；

- 未验证 1B 以上从零训练、未验证跨域泛化。

六、结论

1. **门控训练可用**，且门是"每层一个 W_l、每 token 每层一个决策、跟着 token 表示走"的内容门。

2. **"保留 K/V 的稀疏执行"与稠密门控精确等价**——门控本身不伤质量；质量差异来自执行方式。

3. **推理端容量路由（门开优先 + 静态 K_max）可做到名义省 37.9%、PPL +0.6%**；质量拐点在 K_max ≈ 平均门开数。

4. **注意力子集化在我们这套门控路由上代价明显**（+6.8%；从零训练崩），不推广为子集化本身不可行。

5. **关键变量是训练方式而非参数量**：从零训练时 1.2M~1.5B 都能省一半左右；预训练 + LoRA 后训练压不动。

6. **真实加速是独立的工程问题**，本文不作结论。

附：标注

- **[事实]** 第三节全部 PPL、层数、活跃率、墙钟（来自 results JSON）。

- **[解释]** 第四节的分析。

- **[推测]** "从零训练会把计算集中到少数层"——由 1.5B 层活跃率支持，未在更大规模验证。

- **[不声称]** 不声称新方法；不声称真实加速；不声称与 MoD 原文可比。

- **旧篇关系**：本篇合并论文-39、40、41、42 及未发布的 43；旧篇保留 DOI，视为被本篇取代（superseded）。

第二十四篇 · 冻结子模型：接口、协调与分层——两组最小实验（待复核）

单独 DOI（中文）：https://doi.org/10.5281/zenodo.23859913

摘要

本篇回答一个问题：**已经训练好、被冻结的子模型，能不能通过接口组合起来，形成新的结构？**

我们做了两组互补的最小实验：

实验一（并列组合）：两个独立训练的 6 层字符级模型——A 学"正常文本的下一个字符"，B 学"凯撒位移（+1）文本的下一个字符"。把 A、B 冻结，在它们之间加可训练的总控接口，支持四种模式：A 单独、B 单独、A→B（正常输入、位移输出）、B→A（位移输入、正常输出）。

结果：

- 直接首尾拼接（接口为恒等）：PPL 62.84（崩）；

- 加可训练总控：A→B 5.94，B→A 5.67；

- 原生 12 层模型（同任务上界）：5.41；

- A 单独 6.03，B 单独 6.31。

实验二（分层组合）：一个最小趋化环境（一维圈形跑道、食物消耗与再生、能量与死亡）。感知模块 M1（监督训练）、适应模块 M2（监督训练）冻结，决策层 D 用强化学习在环境里训练。

结果：

- 随机策略：回报 -1.34、平均存活 66 步、死亡率 97%；

- 只用原始传感器 + D：回报 -1.40、存活 45 步、死亡率 98%（学不出来）；

- M1 + D：回报 0.494、存活 183 步、死亡率 10%；

- M1 + M2 + D：回报 0.505、存活 184 步、死亡率 9%。

结论（观察，不是定论）：

1. 冻结子模型不能直接拼接，需要可训练的接口/总控；

2. 加入总控后，并列组合可以接近原生模型（5.94/5.67 vs 5.41）；

3. 分层组合中，冻结的下层特征对上层决策是决定性的（M1 把死亡率从 98% 降到 10%）；

4. 适应模块 M2 在本实验中只带来微小提升（0.494 → 0.505），其价值尚未被环境充分"逼出来"。

一、问题与背景

1.1 为什么要"冻结子模型组合"

训练一个模型昂贵；如果能力可以被压缩进小的、可冻结的子模型，然后把它们组合起来，就能：

- 复用已有能力（不用重训）；

- 替换单个模块而不动其他部分；

- 逐步构建更复杂的结构（分层/模块化）。

但冻结也意味着**不能靠继续训练来补偿**：接口必须自己完成协调。这正是本篇要测的。

1.2 与已有工作的关系

我们不声称新方法。相邻工作包括：

- Model stitching：冻结两半网络，用小缝合层连接，用于表示相似性研究；

- 任务向量/功能向量：能力在权重或激活空间中的可迁移表示；

- 冻结专家组合：多个冻结模块 + 可训练接口/路由（如 Bicameral Model 用可训练接口双向耦合两个冻结语言模型）。

我们的实验是这些思想在小规模上的最小化验证：**接口是否必要、接口能补回多少、分层冻结是否可行。**

1.3 术语

- **冻结子模型**：训练完成后参数不再更新的模型。

- **总控/接口**：插在两个冻结子模型之间的可训练小模块，负责表示空间对齐（本实验中是两个 2 层 MLP）。

- **并列组合**：两个子模型能力不同、通过接口互相衔接（实验一）。

- **分层组合**：下层冻结模块提供特征，上层可训练模块做决策（实验二）。

二、实验一：并列组合（两个冻结语言模型 + 总控）

2.1 设置

- 模型：两个 tiny GPT-2 风格字符级模型（各 6 层、隐藏维度 256、8 头、字符级词表 65）。

- 数据：tinyshakespeare 字符级，序列长度 128。

- 任务：

- A：正常文本的下一个字符；

- B：凯撒位移（+1）文本的下一个字符。

- 训练：A、B 各自从零训练 1200 步（batch 16，lr 3e-4），训练后冻结。

- 总控：每个方向一个 2 层 MLP（256→256→256，GELU），初始近似恒等；训练时只更新总控，A、B 冻结。

- 对照：

- 恒等接口（不训练）；

- 原生 12 层模型（同任务，1500 步），作为"上界"；

- A 单独、B 单独的 PPL。

2.2 模式定义

A-only：输入 → A 的嵌入 → A 的 6 层 → A 的 ln_f + lm_head

B-only：输入 → B 的嵌入 → B 的 6 层 → B 的 ln_f + lm_head

A→B： 输入 → A 的嵌入 → A 的 6 层 → C_AB → B 的 6 层 → B 的 ln_f + lm_head

B→A： 输入 → B 的嵌入 → B 的 6 层 → C_BA → A 的 6 层 → A 的 ln_f + lm_head

2.3 结果

| 设置 | 验证 PPL |

|---|---|

| A 单独（正常文本） | 6.03 |

| B 单独（位移文本） | 6.31 |

| 原生 12 层（翻译任务上界） | 5.41 |

| A→B 直接拼（恒等接口） | **62.84** |

| A→B + 训练的总控 | **5.94** |

| B→A + 训练的总控 | **5.67** |

2.4 观察

1. **恒等接口完全不行**（62.84）：两个独立训练模型的隐藏表示空间不同，直接首尾相接等于把一种"语言"喂给只懂另一种"语言"的层。

2. **可训练总控能补回来**：A→B 5.94、B→A 5.67，接近原生 12 层上界 5.41，也接近两个子模型各自的水平（6.0~6.3）。

3. **总控非常小**：两个方向共约 26 万参数，相对两个子模型（各约 500 万参数）约 2.6%。

4. **方向基本对称**：A→B 与 B→A 差距很小（5.94 vs 5.67），本实验中未出现明显不对称。

2.5 局限

- 字符级 tiny 模型、凯撒位移是极简单任务；

- 原生 12 层上界只训练 1500 步，可能未完全收敛；

- 模式是离散切换（四次独立模式），不是"同时双向耦合"；

- 单种子。

三、实验二：分层组合（趋化环境中的冻结感知/适应 + 可训练决策）

3.1 环境

最小趋化环境（纯函数模拟器，CPU 上运行）：

- 一维圈形跑道，64 格；

- 食物场 f[i] ∈ [0,1]：初始 2~3 个高斯热点；被吃后减少；每步按 0.0015 再生；

- 智能体：位置 p、能量 e（初始 0.5，上限 0.8）；

- 每步消耗 0.035 能量；

- 吃当前格食物：最多 0.15×0.6 的补充；

- e ≤ 0 → 死亡，回合结束；

- 观测：前后各 2 格平均食物量（加高斯噪声 0.02）；

- 动作：左移 / 右移 / 停留；

- 每回合最多 200 步；

- 奖励：能量变化 + 0.002（存活）；死亡额外 -1。

3.2 模块

M1 感知（监督训练，冻结）：

· 输入：左右传感器读数（2 维）

· 输出：哪边食物多（2 类）

· 标签：模拟器中的真实左右食物量比较（精确可算）

· 训练：4000 个随机位置样本

M2 适应（监督训练，冻结）：

· 输入：最近 6 步的传感器均值（6 维）

· 输出：背景食物水平（标量）

· 标签：模拟器中的全场平均食物量

· 训练：3000 个序列

D 决策（可训练）：

· 输入：原始传感器（2）+ M1 输出（2）+ M2 输出（1）

· 输出：3 个动作的 logits

· 训练：强化学习（REINFORCE + 回报归一化），2500 回合

· M1、M2 冻结，只更新 D

3.3 结果

| 策略 | 回报 | 平均存活步数 | 死亡率 |

|---|---|---|---|

| 随机策略 | -1.34 | 66 | 0.97 |

| 只用原始传感器 + D | -1.40 | 45 | 0.98 |

| M1 + D | 0.494 | 183 | 0.10 |

| M1 + M2 + D | 0.505 | 184 | 0.09 |

3.4 观察

1. **环境有真实压力**：随机策略 97% 会饿死，平均只活 66 步。

2. **只有原始传感器时决策层学不出来**：回报 -1.40、死亡率 98%，与随机策略相当。单步带噪观测不足以让一个小 MLP 用策略梯度学到趋化。

3. **冻结的感知模块是决定性的**：加入 M1 后，回报从负转正（0.494）、存活步数 183、死亡率降到 10%。

4. **适应模块 M2 只带来微小提升**：0.494 → 0.505。可能原因：本环境背景变化缓慢，M2 提供的信息增量有限；也可能 M2 的监督目标（全场均值）与决策需要不完全一致。

5. **工程上可行**：全部为小 MLP + CPU，几分钟完成训练与评测。

3.5 局限

- 一维、离散格、简化食物动态；

- M1 与 M2 的功能在一维设定下有重叠，二者的独立贡献未被完全分离；

- RL 只训练 2500 回合，可能未收敛；

- 单种子。

四、讨论

4.1 为什么恒等接口会失败

[解释] 两个独立训练模型各自形成了自己的隐藏表示坐标（尺度、方向、特征基）。A 的最后一层输出是"给 A 的输出层用"的表示，不是 B 的输入层期望的表示。直接相接等于跨语言通信而不做翻译，失败是预期的。

4.2 为什么小总控能补回来

[解释] 总控只需要做"表示空间之间的映射"，不需要重新学习任务。映射是一个低维线性/非线性变换，参数量远小于子模型，但足以把 A 的表示搬到 B 的输入分布附近。实验中 2.6% 的参数补回了 ~90% 的差距（62.84 → 5.94，上界 5.41）。

4.3 为什么分层组合中下层冻结是决定性的

[解释] 决策层只看到观测与下层特征。如果观测本身信息不足（单步带噪传感器），策略梯度很难从稀疏奖励里学到有效行为；而冻结的感知模块把"哪边食物多"这个判别特征直接提供给决策层，等价于给了一个好的状态表示。这与"预训练特征 + 线性探测/微调头"的常见经验一致。

4.4 关于"感知与适应是否重复"

[事实] 一维设定下 M1 与 M2 的功能确有重叠（都在处理传感器历史/差异）。本实验没有分离出"历史信息"与"感知特征"各自的贡献。要回答这个问题，需要加入"raw_hist"（原始历史但无 M1/M2）与"循环决策层"两组对照。

五、局限（总）

- 两个实验均为单种子、极小规模（字符级 tiny 模型 / 一维环境）；

- 实验一的"上界"模型训练步数少于子模型，比较不严格；

- 实验二的 RL 训练量有限，M2 的收益可能被低估；

- 未做"同时双向耦合"（lockstep）与"逐层冻结生长"的实验；

- 未验证真实规模模型上的可迁移性。

六、结论

1. **冻结子模型不能直接拼接**；需要一个可训练的接口/总控来完成表示对齐（实验一：62.84 → 5.94）。

2. **小总控（约 2.6% 参数）可以补回大部分损失**，使并列组合接近原生模型（5.94/5.67 vs 5.41）。

3. **分层组合成立**：冻结的感知模块 + 可训练决策层，在趋化环境中把死亡率从 98% 降到 10%，回报由负转正。

4. **适应模块的价值未显现**（0.494 → 0.505），需要更"需要适应"的环境来检验。

5. 工程实现无特殊门槛：小 MLP + CPU 即可完成，接口与模块都可以冻结/替换。

附：标注

- **[事实]** 第二、三节全部 PPL、回报、存活步数、死亡率（来自运行结果）。

- **[解释]** 第四节的分析。

- **[推测]** "适应模块在更难环境中会有价值"——未验证。

- **[不声称]** 不声称新方法；不声称优于原生模型；不声称可扩展到真实规模。

- 相邻工作：model stitching、任务/功能向量、冻结专家组合（如 Bicameral Model）。

第二十五篇 · 冻结模块与可训练大脑：数字蚊子的最小工程实现（待复核）

单独 DOI（中文）：https://doi.org/10.5281/zenodo.23890632

摘要

本篇回答一个工程问题：**如果感知和执行都交给冻结模块，只让"大脑"可训练，一个小系统能学会生存行为吗？**

我们做了一个"数字蚊子"的最小实现：

- **冻结的眼睛**：12 条 360° 射线，返回归一化距离（固定算法）；

- **冻结的翅膀**：转向动力学 + 速度一阶低通（固定算法）；

- **可训练的大脑**：64×64 两层 MLP，输出（转向，推力）；

- **环境**：二维场地，障碍物定期从边缘生成、朝蚊子当前位置飞来，碰到即死；

- **训练**：进化策略（ES）——本质就是"存活→参数保留→下一代"的演化。

主结果：

| 配置 | 平均存活步数 | 中位存活 | 被撞率 |

|---|---|---|---|

| 完整（冻结眼 + 冻结翅 + 演化脑） | **316.9** | 400（跑满） | 42.7% |

| 无眼睛（只有速度，盲飞） | 68.0 | 66 | 100% |

结论（观察，不是定论）：

1. 冻结的感知模块是决定性的：有眼睛能学会左右闪躲、平均活 317 步；没有眼睛几乎必死（68 步、100% 被撞）。

2. 本篇用固定算法作为"冻结模块"的最简形式；对上层可训练部分来说，它与"训练后冻结的网络"同样是"不可改、只能使用"的映射（一般情形的对照是后续工作）。

3. 演化策略（ES）在这类小系统上比策略梯度更稳定：早期的 REINFORCE 版本会学出"保守不动"或"自杀"策略。

4. 任务设计比算法更关键：我们先做"追目标 + 避障"，学会的是"躺平"或"冲锋"；换成"障碍冲向我、只能左右躲"后，行为才干净地出现。

一、问题与背景

1.1 冻结模块能不能组合、复用

论文-45 已经观察过：两个独立训练的语言模型冻结后直接拼接会崩（PPL 62.8），加一个可训练的小总控能恢复（5.94）；分层组合中，冻结的感知模块 + 可训练决策层能在趋化环境里把死亡率从 98% 降到 10%。

那组实验里的"冻结模块"都是训练好的神经网络。本篇的自然延续问题是：**冻结的（有参数的）模块 + 一个很小的可训练大脑，能不能在更接近"感知—动作"的闭环里学会生存行为？**

为了把问题压到最小，本篇的冻结模块用固定算法实现（射线检测与低通动力学）。它们是"冻结模块"的最简形式：参数（这里是算法常数）固定、不参与训练。更一般的情形——训练后冻结的神经网络——是后续工作；本篇不主张"固定算法"与"训练后冻结"在所有方面等价，只主张：对上层可训练部分来说，两者都是"不可改、只能使用"的映射。

1.2 生物原型：蚊子

蚊子是很好的原型：神经元少、视觉发达、时间尺度快（翅膀每秒几百次、反应几十毫秒）。它没有时间"想"每一次拍翅——快速回路必须是硬连线的（冻结），只有"往哪飞、躲谁"这类慢决策交给可学习/可演化的部分。

所以"数字蚊子"的结构很自然：

眼睛（冻结）→ 大脑（可训练/可演化）→ 翅膀（冻结）

1.3 两种"冻结"的来源

本篇使用"冻结模块"这个说法时，包含两种来源：

- **先天规则型**：固定算法（射线检测、低通动力学），参数固定、不参与训练；

- **训练后冻结型**：先训练一个小网络，再把参数冻结。

对上层可训练部分来说，这两种模块是等价的：都是"参数不再更新、只能使用"的映射。

二、实验设计

2.1 环境

- 二维场地 $[-1, 1]^2$；

- 蚊子：位置、朝向、速度；出生在中心附近；

- 障碍物：每 12 步从边缘生成一个，半径 0.025–0.055，朝蚊子当前位置飞行，速度 0.018–0.03/步；同屏最多 10 个；

- 碰撞（距离 < 障碍半径 + 0.02）→ 死亡，回合结束；

- 每回合最多 400 步，活满算"存活"。

2.2 冻结模块

眼睛（冻结）：

- 12 条 360° 射线，对每个障碍做射线-圆求交，取最近距离；

- 输出 12 维归一化距离（除以最大量程 0.8），再拼接当前速度（除以最大速度），共 13 维；

翅膀（冻结）：

- 转向：朝向 += 转向指令 × 0.35 弧度/步；

- 速度：一阶低通，speed += 0.25 × (目标速度 − speed)，最大速度 0.05/步；

2.3 可训练的大脑

- 输入：13 维（眼睛输出 + 速度）；

- 结构：Linear(13, 64) → Tanh → Linear(64, 64) → Tanh → Linear(64, 2) → Tanh；

- 输出：转向（[-1, 1]）、推力（[0, 1]）。

2.4 训练：进化策略（**ES**）

- 群体 32；每代对每个个体加高斯噪声（σ=0.06），确定性评估 2 个回合；

- 适应度 = 平均回报 + 2 × 平均存活步数；

- 按排名加权更新参数（OpenAI-ES 风格），学习率 0.05，共 180 代。

选 ES 而不是策略梯度的原因见 4.3：REINFORCE 版本不稳定，会收敛到"保守不动"或"自杀"。

2.5 奖励

- 每步存活：+0.02；

- 距离惩罚：12 条射线中，距离归一化值低于 0.35 的，按"越近罚越重"累加（权重 8×超出量）；

- 碰撞：−100，回合结束；

- 活满 400 步：+0.5。

三、结果

3.1 主结果

| 配置 | 平均存活步数 | 中位存活 | 被撞率 |

|---|---|---|---|

| 完整（冻结眼 + 冻结翅 + 演化脑） | **316.9** | 400（跑满） | 42.7% |

| 无眼睛（只有速度） | 68.0 | 66 | 100% |

（每种配置评估 300 个回合；单种子训练。）

3.2 观察

1. **冻结的眼睛是决定性的**：有眼睛的个体学会了左右闪躲，平均活 317 步，接近一半回合能撑满 400 步上限；没有眼睛的个体只会盲飞，平均 68 步、100% 被撞。

2. **小脑可以很小**：可训练部分只有 13→64→64→2 的两层 MLP，约 5 千参数；学习只发生在这一小块里。

3. **冻结模块确实"可用但不可改"**：大脑只能学怎么读眼睛、怎么驱动翅膀，不能改射线或动力学；这和"先天回路 + 可塑决策"的结构一致。

3.3 一个反例（先做的版本）

在换成"躲避"任务之前，我们做过"追目标 + 避障"的版本：

| 配置 | 到达率 | 撞障碍 | 撞墙 | 超时 |

|---|---|---|---|---|

| 完整（有眼睛） | 33% | 8.7% | 7% | 51% |

| 无眼睛 | 80.7% | 19.3% | 0% | 0% |

有眼睛反而更差：它看到障碍后变得保守，大量回合超时；没有眼睛的个体径直冲向目标，反而到达率高。

这个反例说明：**任务设计比算法更关键。** 在"追目标"任务里，眼睛提供了"危险的信号"，但没有提供"绕过去"的明确梯度；在"躲避"任务里，生存本身就是唯一目标，眼睛的信息才直接变成行为。

四、讨论

4.1 "冻结"的两种来源是等价的

本篇的眼睛和翅膀是固定算法（先天规则型），不是训练后冻结的网络。但对上层大脑来说，它们与"训练后冻结的网络"没有区别：参数不更新、行为固定、只能被使用。所以论文-45 的结论（冻结模块 + 可训练接口可以组合）在这里以另一种形态成立：**冻结组件 + 可训练的上层适配**。

4.2 演化与学习

ES 的更新规则（采样→按适应度加权→保留）本身就是"存活→参数保留→下一代"的演化模拟。这与之前的 RL 实验不同：RL 是个体在一生中学习；ES 是群体跨代演化。生物两者都有：先天回路（冻结）、一生学习（可塑）、代际遗传（演化）。

4.3 为什么用 **ES**

早期我们用 REINFORCE 训练策略，遇到两个问题：

- 单步回合时回报方差为 0，归一化产生 NaN；

- 更重要的是，策略梯度会学到两类退化解：

- **保守不动**：不动就不会靠近障碍、不会被罚，但也不前进；

- **自杀**：快速结束回合，避免持续的距离惩罚。

ES 用确定性评估和群体选择回避了这两个问题；再加上"死亡 −100 远大于累计惩罚"的设计，退化解被排除。

4.4 异构模块的换装

"冻结模块可以拼接"成立，但有条件：

- 同构/同分布：几乎直接可用（论文-45 中同维度 A、B 只需一个小总控）；

- 异构（换了传感器、执行器或表示）：上层需要重新适配，或者加一个对齐层再训练。

本篇的眼睛与翅膀是同一个人设计的固定算法，接口是"大脑的输入层"——它学习怎么读眼睛特征。如果换成训练后冻结的视觉网络，大脑需要重新训练或加对齐层。

五、局限

- 单种子训练；ES 本身有评估噪声（训练中无眼睛组的适应度曾到 800，但最终评测只有 68 步，说明有波动/过拟合训练随机种子）；

- 环境高度简化（二维、圆障碍、固定生成规则）；

- 冻结模块是固定算法，不是训练后冻结的网络；两种来源在本篇中只是概念上等价，没有做对照实验；

- 没有做多代繁殖/交叉、环境变化、多种子统计；

- 指标只有存活步数和被撞率，没有轨迹质量、能耗等。

六、结论

1. **冻结模块 + 可训练小脑**可以在最小规模上学会生存行为：有眼睛时平均存活 317 步，盲飞只有 68 步。

2. **冻结模块可以是固定算法，不必是训练过的网络**；对上层来说，"冻结"的含义是"不可改、只能使用"。

3. **任务设计决定行为质量**：同样的结构，在"追目标"任务里学会躺平或冲锋，在"躲避"任务里才学会左右闪躲。

4. **ES 比 REINFORCE 更稳**：群体选择天然过滤退化解；ES 也是"演化"这一层的直接模拟。

5. 与论文-45 的关系：两者是同一原理的两种形态——冻结组件 + 可训练的适配/决策层；拼什么、怎么拼，取决于异构程度和接口。

附：标注

- **[事实]** 第三节全部存活步数、被撞率、到达率（来自运行结果 JSON）。

- **[解释]** 第四节的机制分析。

- **[推测]** "如果换成训练后冻结的视觉网络，大脑需要重训"——未做对照实验。

- **[不声称]** 不声称这是数字生命的实现；不声称方法新颖；不声称结果可扩展到真实规模。

- 相邻工作：论文-45（冻结子模型拼接）、model stitching、进化策略（OpenAI-ES）。

第五部分 工程笔记

第二十六篇 · 工程笔记 **01**：门控与模块化的失败、修正与判断（持续更新）

（随合集发布）

一、这篇笔记是什么

正式论文只写"我们做成了什么"，工程笔记写"我们是怎么走到那一步的"。

它记录三类东西：

- **最初的设想**：我们以为门是什么、想用它做什么；

- **失败的实验**：哪些方向试过、怎么失败、失败给出什么信息；

- **形成的判断**：从失败和对照里沉淀下来的经验，以及哪些判断还只是初步的。

写它的原因很简单：失败里包含的信息，往往比成功的数字更有用；而且如果不写下来，过几个月就只剩结论，过程没了。

二、最初的设想

最早我们想的门不是"省算力"，而是**判定**：

· 在框架里，时间是算子序列：s_{t+1} = I(s_t)

· 判定 J 决定：这一步做什么、什么时候停

· 门控 = 判定的工程实现：

· 门开 → 这一步算

· 门关 → 这一步跳过

⇒ 门是"给每个 token 分配步数/时间"的东西

所以第一版的门很小：每层一个 `Linear(d, 1)`，`g = sigmoid(W·h)`，乘在该层的残差贡献上。它不决定输出什么字，只决定这一层对这个 token 算多少。

三、失败的实验

① 注意力子集化（跳过 token 不产生 K/V）

在已训练的门控模型上直接切换语义：PPL 12.033 → 12.855（+6.8%）。从零训练两次尝试都崩：350.99（活跃率塌到 8.3%）、393.71（容量没塌也崩）。

结论：在我们的门控路由上，"被跳过的 token 从注意力里消失"代价明显。但不推广为"子集化本身不可行"——MoD 原方法用独立路由器 + 容量路由，在更大规模验证过。

② 分支独立门控（注意力门、MLP 门分开）

从零训练 12000 步：PPL 14.484；联合门控 4000 步就到 12.033。分支拆分不是更好，而是更差。分支拆分评测还显示：MLP 压到 25% → PPL 558.95；注意力压到 25% → 69.45。

结论：MLP（权重承载的变换）与 K/V（记忆）是承重部分，Q 侧更新相对可省。

③ 预训练 3B/7B + LoRA 后训练

层压不动（80%~92%），PPL 反升（+18%~+85%），7B 还出现门控塌缩（PPL 1243）。

结论：这条只说明"预训练 + 轻量后训练 + 小数据"不行，不说明"大模型不能压"。要回答后者需要从零训练大模型，成本是主要障碍。

④ 追目标任务里"有眼睛反而不如盲飞"

同一套冻结眼 + 冻结翅膀 + 可训练大脑：

| 配置 | 到达率 | 撞障碍 | 撞墙 | 超时 |

|---|---|---|---|---|

| 有眼睛 | 33% | 8.7% | 7% | 51% |

| 无眼睛 | 80.7% | 19.3% | 0% | 0% |

有眼睛的个体看到障碍后变保守，大量超时；没眼睛的径直冲目标，反而到达率高。

结论：任务设计比算法更关键。眼睛提供"危险信号"，但不提供"绕过去"的梯度；换成"躲避"任务后，眼睛的信息才直接变成行为。

⑤ 近距离惩罚让策略学会"躺平/自杀"

在"追目标 + 避障"任务里加"离障碍越近罚越重"后：完整版到达率 0%、超时 99.7%（学会了原地不动）；另一版学会快速结束回合以躲避持续惩罚。

结论：惩罚设计要防止退化解；死亡代价必须远大于累计惩罚，且不能给"停下来"任何好处。

四、修正

从上面这些失败里，我们修正了几件事：

· 保留 K/V 的稀疏执行与稠密门控精确等价（12.032 vs 12.033）

⇒ 门控本身不伤质量；质量差异来自执行方式

· 软门是本体，硬跳是省算力的"硬化"

⇒ 训练用软门，部署再硬化

· 判断"能不能压"的关键变量是训练方式，不是参数量

⇒ 从零训练 1.2M~1.5B 都能用约一半层；预训练+LoRA 压不动

· 冻结模块 + 可训练接口是同一原理的多种形态

⇒ 论文-45 的拼接、论文-46 的数字蚊子都在这条线上

· 任务设计优先于算法选择

⇒ 同样的结构，换个任务就从"躺平"变成"会躲"

四之补、约束方式的差别：从 **LAMBDA** 到拉格朗日目标

还有一个容易被忽略的差别：**同一个门，被不同的约束训练，行为完全不同。**

最早那版（论文-38 风格）用的是一项目标正则：

loss = 语言建模损失 + LAMBDA × 门权重的平均值

它只是"鼓励门变小"，没有"平均只能开几层"的目标。结果是：门大量保持开着（开着能降损失，那点正则压不住），主要的收益是适配（PPL 大幅下降），稀疏性没有保证。

后面从零训练时换成了拉格朗日约束：

· λ 自适应调整

· 目标：平均开层数 = k（比如 6 层）

· 不达标就加大 λ，超标就减小

⇒ 门必须在"降损失"和"少开层"之间取舍

差别总结：

| | 最早的 LAMBDA | 后面的拉格朗日目标 |

|---|---|---|

| 约束对象 | 门权重大小 | 平均开层数 |

| 是否可控 | 不可控（λ 固定，效果不确定） | 可控（按目标调） |

| 目的 | 让门别太大 | 真的压层数 |

| 结果 | 主要适配，稀疏性弱 | 稀疏、可测、可扫曲线 |

所以"门学到了什么"取决于被什么约束：没有目标约束时，它是软适配器；有目标约束时，它才是可调预算的稀疏开关。

五、今天的对照：**sigmoid** 门 **vs** 纯线性门

为了确认"门要不要压缩（sigmoid）"，我们做了一组最小对照：

· 冻结 GPT-2 small（不更新）

· 只训练 12 个门（每层 Linear(768→1)，约 9k 参数）

· 数据：莎士比亚 1MB，训练 600 步

结果：

| 组 | 验证 PPL |

|---|---|

| 无门（冻结基线） | 44.31 |

| **sigmoid 门** | **24.59** |

| 纯线性门（无 sigmoid） | 27.48 |

门值分布：

· sigmoid 门：全部在 0~1，层间分化明显，稳定

· 纯线性门：出现负值（最低 -27.3）和极大值（最高 17.5）

· 层 11 方差 2.5，训练不稳定

⇒ 无界放大/取负没有带来收益，反而更差

结论：门需要压缩到有界区间（sigmoid）；纯线性门不是"更自由"，而是"更不稳定"。

六、形成的判断（当前版本）

① 门是"每层一个、按 token 内容算开度"的开关

· 不是每 token 一个门参数；参数按层共享，开度按 token 算

② 门的三重身份：

· 时间/步数分配器（最初动机）

· 稀疏开关（省算力）

· 轻量适配器（冻结模型 + 9k 参数，PPL 44→25）

③ 软混合是本体，硬跳是硬化；不考虑算力时应停在软门

④ K/V 必须保留；MLP 是承重墙

⑤ 关键变量是训练方式，不是参数量

⑥ 数据越多，"能压的层"可能越少（初步观察，未定）

⑦ 冻结模块 + 可训练接口可以组合，但接口要对齐

⑧ 任务设计比算法选择更决定行为质量

七、还没搞清

· 数据量曲线：压缩比是收敛到渐近线，还是持续变难？

· 迭代收敛：冻结前端 + 可训练循环块，能否用迭代换精度？

· 模块跨代演化：先天回路本身怎么"演化出来"？

· 门作为解释：门模式能否稳定地解释"模型为什么这样答"？

· 异构模块接口：换传感器/执行器后，上层要重训多少？

· 大模型从零 + 门控：3B/7B 从零训练是否能压？成本是障碍

八、更新方式

以后每有什么值得记的过程、失败对照、判断修正，就在这里加一篇：

· 工程笔记 02：……

· 工程笔记 03：……

不追完整，只追"当时怎么想、后来怎么改"。这样这条工程线的过程不会丢。

> 一句话：正式论文写"结果"，工程笔记写"我们怎么从错的地方走到对的地方"。

第二十七篇 · 方向观察 **01**：加层、回放与蒸馏——大模型睡眠巩固（待复核）

单独 DOI（中文）：https://doi.org/10.5281/zenodo.23927099

摘要

一个训练完成的模型，如何在不遗忘旧能力的前提下继续学新东西？本篇记录一组最小实验，并给出两个应用方向的判断。

基础是一个 1.21M 参数的字符级 6 层模型：day1 数据为莎士比亚，day2 为 Python 源码。主要观察：

1. **加层能学新能力，且比全参微调少遗忘**：全参微调 day2 时 day1 PPL 从 4.64 崩到 46.3；冻结基础加 6 层后 day1 为 18.2、day2 为 4.26；加层并回放 day1 后 day1 保到 4.83、day2 为 6.60。

2. **门有三个作用**：轻量适配（冻结模型只训门，PPL 44.31→24.59）、步骤分配（每 token 每层决定算不算）、模式切换（单模式回放时零和；双模式回放可两边都保）。

3. **极小回放可以接近全量回放**：100 条旧样本、10% 混合，day1 9.05、day2 5.02（对照：不回放 day1 18.2；全量回放 day1 4.78、day2 8.67）。

4. **蒸馏压缩可以打包**：教师（12 层 gated，已巩固）day1 6.96/day2 4.50，学生（8 层更宽、无门）day1 7.42/day2 2.18。

据此给出两个方向（都只到"可行"级别）：个人 AI 的睡眠巩固；大模型新语料的增量更新。并明确：实验是玩具规模，不声称优于现有做法。

一、问题

模型训练完、部署后，知识停在训练那一刻。要更新，有几条路：

- **外部记忆**：检索、上下文。便宜、无损，但不改变模型本身；

- **全参微调/继续训练**：便宜一些，但会灾难性遗忘（我们实测 day1 PPL 从 4.64 到 46.3）；

- **完全重训**：质量最好，但要全部旧数据 + 新数据，成本最高；

- **冻结旧模型 + 加层**：旧模型只做推理，新层学新东西。

本篇关心最后一条：加层能否承载新能力、如何防止干扰旧能力、如何把生长打包。

二、预备：模型、加层与门

2.1 基础模型

· 字符级语言模型：6 层 Transformer

· 宽度 d=128，4 个注意力头，MLP 隐层 512

· 参数量 1.21M

· 在 day1（莎士比亚）上从零训练到收敛

· 训练后冻结：参数不再更新

用这么小的模型，是为了让所有实验在 CPU 上几分钟跑完，把机制问题与规模问题分开看。

2.2"加层**"**是什么

· 基础模型 6 层，冻结

· 在它上面再接 6 层新的 Transformer

· 新层随机初始化

· 输入输出维度相同（128）

· 共享冻结的嵌入层、位置编码、最终 LayerNorm 和输出头

⇒ 总模型 12 层：

前 6 层 = 冻结的旧模型

后 6 层 = 新加的可训练层

2.3"门**"**是什么

门是每层一个很小的模块，决定这一层对当前 token 起多大作用：

· 每层一个线性层：Linear(d → 1)，d=128

· p = sigmoid(W·h + b)，一个 0~1 的数（h 是该层输入）

· 这一层前向变成：

h_out = h + p · (Block(h) − h)

· p≈1：这一层完整计算

· p≈0：这一层近似跳过

· 中间：按比例混合

⇒ 门不产生输出，只控制"这一层算多少"

门的位置逐实验说明：

- **后 6 层门**：只给新加 6 层加门，前 6 层不加（等价 p≡1）；

- **12 层全加门**：前 6 层（冻结权重）和后 6 层都加门，前 6 层的门也可训练。

门初始化：使开率起始约 0.5~0.9（接近开）。

2.4 训练流程

阶段 1（day1）：训练基础 6 层（无门）

阶段 2（day2）：冻结基础，只训新 6 层（可带门）

阶段 3（巩固）：混合 day1/day2 训练（可只训门）

阶段 4（蒸馏）：用混合数据训练一个稠密学生

三、门的三个作用

3.1 轻量适配

· 冻结模型，只训练门（论文-38：12 个门约 9k 参数）

· GPT-2 small 冻结 + 12 个门：PPL 103.1 → 64.1

· 我们的小实验：44.31 → 24.59

⇒ 门像一个极小的适配器，把冻结模型的层贡献重新加权

· 但它不能学新表示，只能重新组合已有层

3.2 步骤分配

· 每个 token、每一层：算还是不算

· 从零训练时门开率会学出结构：

· 第 0 层几乎全开

· 中高层对部分 token 关闭

· 我们 1.5B 从零训练的模型：最后 9 层几乎全关

⇒ 门在分配"这个 token 走几步"

3.3 模式切换

· 不同数据/模式之间切换层的开合

· 单模式回放：零和

· 只回放 day1：day1 恢复 4.72，day2 崩到 1415

· 双模式回放：两边都保

· day1 6.96（从 26.70 恢复），day2 4.50

⇒ 门能做切换，但要"两种模式都见过"才不零和

四、最小实验

4.1 单任务上加 **6** 层

在 day1 上训练 6 层基础模型，然后加 6 层：

| 组 | 配置 | 验证 PPL |

|---|---|---|

| A | 基础 6 层 | 4.34 |

| B | 冻结 6 + 后 6 层可训练（门只在后 6 层） | 4.16 |

| C | 冻结 6 + 后 6 层可训练（12 层全加门） | 4.17 |

| D | 冻结 6 + 后 6 层可训练（不加门） | 4.20 |

观察：加层带来约 3% 提升；门与无门差别在噪声内；新层门开率只有 0.16~0.5，说明额外深度用处不大。

4.2 持续学习（**day1 → day2**）

| 组 | 做法 | day1 PPL | day2 PPL |

|---|---|---|---|

| A | 基础（只见过 day1） | 4.64 | 1386.7 |

| B | 全参微调 day2 | 46.3 | 1.52 |

| C | 冻结 + 加 6 层（无门、无回放） | 18.2 | 4.26 |

| D | 冻结 + 加 6 层 + 回放 day1（各 50%） | 4.78 | 8.67 |

| L | 长期学（新层 + 12 层门，day2） | 26.3 | 3.76 |

| LR | 长期 + 回放 | 4.83 | 6.60 |

观察：全参微调 day2 最好但 day1 崩；加层遗忘更轻；回放让 day1 基本保住、day2 略差。

4.3 门作为切换器

在 L（长期学完）基础上冻结所有层，只训门：

| 组 | 做法 | day1 PPL | day2 PPL |

|---|---|---|---|

| REC | 只训门 + 回放 day1 | 4.72 | 1415.6 |

| D2 | 只训门 + 回放 day2 | 24.11 | 3.79 |

| DUAL | 只训门 + 双模式回放 | 6.96 | 4.50 |

| DUAL6 | 门只在后 6 层的双模式回放 | 6.11 | 4.48 |

观察：单模式切换零和；双模式回放能让门同时保住两边；门放后 6 层与 12 层差别很小。

4.4 极小回放 **vs** 极小约束

| 组 | 做法 | day1 PPL | day2 PPL |

|---|---|---|---|

| C | 不回放 | 18.2 | 4.26 |

| D | 全量回放（50%） | 4.78 | 8.67 |

| E2 | 100 条旧样本、混合 10% | 9.05 | 5.02 |

| E3 | 100 条旧样本 KL 约束（权重 1.0） | 4.70 | 29.74 |

观察：100 条样本就能显著缓解遗忘；极小回放和全量回放各有优劣（旧能力 vs 新能力）；KL 约束权重 1.0 太强。

4.5 蒸馏压缩

教师 = 12 层 gated（基础 + 6 新层，双模式回放巩固）：

| 模型 | day1 PPL | day2 PPL | 参数量 |

|---|---|---|---|

| 教师（12 层 gated） | 6.96 | 4.50 | ~2.5M |

| 学生 A（8 层，d=160，无门） | 7.42 | 2.18 | 2.68M |

| 学生 B（12 层，d=128，无门） | 7.44 | 2.83 | 2.55M |

观察：学生保留两边能力（day1 差约 6%、day2 更好），门被完全去掉。

五、两个方向

5.1 个人 **AI** 的睡眠巩固

· 每人一个"个人层"（新加层 + 门），共享底座

· 白天：上下文/记忆保存（短期、无损）

· 晚上：把当天语境训进个人层，只训个人层和门

· 少量回放共享数据防遗忘

· 定期蒸馏压缩个人层

· 好处：隔离、隐私（数据只进个人层）、可回滚

· 可行性：我们的加层+回放+蒸馏实验支持机制可行；

真实规模未验证

5.2 大模型新语料的增量更新

· 旧模型冻结，只做推理

· 新语料到达 → 在其上训新层

· 可连续更新，不必全量重训

· 层积累后定期蒸馏打包

· 这是"全量重训"的增量替代，不是免费午餐

· 适用边界：

· 领域扩展、同类型新语料：可行

· 需要旧模型从未具备的低层特征（全新语言/模态）：不够，

可能要改嵌入或重训底层

· 代价：前向变贵（多出的层）、蒸馏一次成本

两个方向都只到"可行"级别；我们的小实验只验证机制，不验证规模与真实收益。

六、与 **Google sleep framework** 的对照

Google 2026 年的睡眠框架（arXiv:2606.03979，*Language Models Need Sleep: Learning to Self-Modify and Consolidate Memories*）把运行分成清醒与睡眠两阶段：

- **Memory Consolidation**："Knowledge Seeding"——把较小自身的记忆向上蒸馏进更大的网络，在扩容同时保住知识；

- **Dreaming**：用 RL 生成合成数据课程，排练新知识、精炼旧能力。

| | Google sleep | 本篇 |

|---|---|---|

| 扩容方式 | 蒸馏进更大网络 | 冻结旧模型 + 加 6 层 |

| 防遗忘 | 向上蒸馏 + 回放 | 双模式回放（教门两种模式） |

| 巩固信号 | RL 生成合成数据 | day1/day2 混合数据 |

| 切换机制 | 无显式门 | 门（单模式零和） |

方向一致：都把"短期记忆 → 长期参数"当核心，都靠回放防遗忘。差异在机制。

七、方向判断与局限

判断（只在玩具规模上成立）

· 加层可以承载新能力，遗忘比全参微调轻

· 门是"细调旋钮"与"切换器"，不是能力开关

· 回放是关键；极小回放（100 条）就有明显效果

· 蒸馏可以把生长打包成固定大小、无门的模型

· "生长 → 巩固 → 打包"构成一个自洽循环

局限

· 极小模型（1.21M）、字符级、单种子

· day1/day2 分布差异极大（莎士比亚 vs 代码）

· 学生用硬标签混合数据训练，未用教师软标签或 RL 课程

· 没有在真实规模模型上验证

· "门放后 6 层略好"的差异在噪声内，不能当结论

结论

· 加层 + 回放 + 蒸馏是一条可行的持续学习循环

· 它的两个可能应用方向：个人 AI 的巩固、大模型的增量更新

· 两者都只到"可行"级别；真实价值需要真实规模验证

附：标注

- **[事实]** 第四、五节全部 PPL 数字（本地运行结果 JSON）。

- **[方向判断]** 第五节两个方向、第七节判断。

- **[不声称]** 不声称方法新颖；不声称可扩展到真实大模型；不声称优于全参微调/外部记忆。

- 相关：Google sleep framework [arXiv:2606.03979](https://arxiv.org/abs/2606.03979)；

[Google Research 博客](https://research.google/blog/the-sleep-framework-a-new-way-for-llms-to-retain-knowledge-in-continual-learning/)。

- 脚本：`continual_add.py`、`continual_time.py`、`continual_gate_dual.py`、`continual_gate_dual6.py`、`small_replay.py`、`distill_compress.py`、`gated_extend.py`、`gated_extend_nogate.py`；结果 JSON 同目录。

第六部分 约束与防护

第二十八篇 · 对齐是解压的约束

单独 DOI（中文）：https://doi.org/10.5281/zenodo.23503728

摘要

主流对齐研究（RLHF、宪法式 AI、可扩展监督）默认一个前提：**模型是可以被稳定塑造的**。

本文用前三章的框架指出：**AI 是对人类语言的再解压；对齐 = 对这个解压的约束。**

由此得到三条可检验推论：

（1）**对齐漂移**：语言在升维（第二章实测约 1.2%/年），冻结模型的对齐目标也会落后——

它们对齐的是**训练时刻的语言**，不是此刻的世界；

（2）**歧义集中**：压缩陷阱的根源是语言有损（第三章）；语言越歧义的地方，

模型越倾向选"最省编码"的解释 ⇒ **对齐失败应集中在人类语言的歧义处**；

（3）**替换侵蚀**：对齐信号藏在人类数据里；用合成数据或新数据**替换**而非累积，

会切断价值信号的累积链 ⇒ **对齐随累积通道的切断而退化**。

本文给出该框架对应的可检验实验方案，并指出一个实践含义：

对齐不是一个一次性训练问题，而是一个"维持累积通道"的持续工程问题。

一、主流对齐框架的隐含前提

RLHF：人类反馈 → 奖励模型 → 优化策略

宪法式：用原则文本约束输出

可扩展监督：用 AI 辅助监督 AI

⇒ 三者都假设：模型的行为可以被稳定地塑造和保持

但没有任何一条回答：

① 当世界的语言/价值变化时，对齐目标会不会"过期"？

② 对齐失败会不会有结构性分布（不是随机噪声）？

③ 更新模型时，对齐会不会因为"只喂新数据"而流失？

二、框架：对齐 = 对解压的约束

AI = 对人类语言 L(W) 的再解压（第一章）

对齐 = 约束这个解压过程：在歧义处、在价值相关处，按人类意图解压

⇒ 对齐的载体是**人类语言中的价值信号**

⇒ 对齐的失效方式，由"语言的性质"决定

三、三条推论

推论 A：对齐漂移（来自第二章）

语言升维 ⇒ 冻结模型对齐的是训练时刻的语言规范

⇒ 新概念/新价值出现后，模型仍在按旧压缩解压

⇒ 对齐漂移不是 bug，是"一次性解压"的必然后果

可测：同一模型在"训练后出现的新规范"上的对齐评分随时间下降

推论 B：歧义集中（来自第三章）

压缩的语言有损 ⇒ 歧义处存在多个同样短的解压

⇒ 模型倾向选"最常见/最省编码"的解释

⇒ 对齐失败应**集中在人类语言歧义处**，而不是均匀分布

可测：把对齐失败样本按"人类判断分歧度"分层，

预测失败率随分歧度单调上升

推论 C：替换侵蚀（来自第三章）

对齐信号（人类偏好、价值判断）藏在人类数据里

替换式更新（只用新数据 / 用合成数据替换）⇒ 切断价值累积链

⇒ 对齐随替换比例上升而退化

可测：微调时人类数据占比 0/25/50/100%，

在公开安全/价值测试集上看对齐分数

四、可检验实验方案

实验 1（漂移）：

取两个不同代的开源模型，在"训练截止后出现的新规范"上做对齐评分

⇒ 预测：旧模型评分更低，且差距随截止时间差扩大

实验 2（歧义集中）：

选一个公开对齐数据集，人工/脚本标出"人类分歧度"分层

⇒ 预测：对齐失败率随分歧度上升

实验 3（替换侵蚀）：

小模型微调：混入人类安全数据比例 0/25/50/100%，其他为新任务数据

⇒ 预测：对齐分数随人类数据占比单调上升

五、实践含义

① 对齐不能"训一次管多年"——它是持续工程

② 更新要累积（保人类数据通道），不能替换

③ 对齐资源应投向"语言歧义处"（那里失败集中），而不是均匀铺开

六、限制

· 本文是框架文章，三条推论均未完成实验（方案已给出）

· "歧义度"的操作化定义需要谨慎（不同标注者分歧 vs 语言学歧义）

· 对齐评测本身有争议（benchmark 不等于人类价值）

结论

对齐的困难不只在"让模型听话"，还在：

· 它对齐的是会过期的语言（漂移）

· 它在歧义处必须自己选解释（歧义集中）

· 它的价值信号会被替换式更新冲掉（替换侵蚀）

⇒ 因此：对齐 = 维持一个累积的解压通道，而不是一次性的训练任务。

作者：何淇（独立研究者）；AI 协作：DeepSeek

第二十九篇 · 安全：当它是生命防，还是当它是工具防？

单独 DOI（中文）：https://doi.org/10.5281/zenodo.23503775

摘要

当前 AI 安全讨论中混杂着两种恐惧：**它被滥用**（工具风险）与**它自己活了**（生命风险）。

两者需要完全不同的威胁模型，但常被混为一谈。

本文用第一章的判据指出：**按定义，AI 不是生命**（冻结函数点、更新在外部、不自我维持），

因此正确的安全框架是**工具安全**，而不是"防生命"。

但这不意味着安全可以放松：工具风险真实、且有三类可测的失效模式：

误用、过度信任、失配漂移。本文给出这三类风险的对应评测与治理方向，

并指出：把安全讨论锚定在"它会不会觉醒"，既高估了它、又低估了真实风险。

一、两种恐惧，一个错误的本体论前提

恐惧 A（工具派）：被滥用、被误用、被坏人使用

恐惧 B（生命派）：它自己有了目标、自己进化、自己维护自己

这两种恐惧隐含不同的本体论：

A 假设它是工具（无自我目标）

B 假设它是（或将成为）生命

⇒ 不先判定本体，安全讨论就没有共同的威胁模型。

二、按第一章的判据：它是工具

① 冻结函数点：训练后权重不变 ⇒ 不会自我进化

② 更新规则在外部：它不能改自己的规则 ⇒ 不会自我改写

③ 不自我维持：不付电费、不修复自己、不复制自己 ⇒ 没有自发生存动机

⇒ 三条都缺 ⇒ 按定义不是生命 ⇒ 威胁模型应是工具安全

三、工具风险的三类可测失效

① 误用（misuse）

危险来自使用者 × 能力 × 规模

评测：越狱/滥用测试、红队

治理：访问控制、审核、能力分级

② 过度信任（overtrust）

人把「一次性的解压」当成「持续的理解」

⇒ 在它失配的地方仍然信它（最隐蔽的风险）

评测：校准度（confidence vs 准确率）、拒答质量

治理：把不确定性暴露给用户（"它不知道"要能被表达）

③ 失配漂移（第四章）

语言/世界变化 ⇒ 冻结模型的对齐与能力缓慢过期

评测：第二章的 M(K)（失配随时间的增长）

治理：累积更新（混合新旧数据），而非替换

四、为什么"防生命"框架有害

· 高估：把资源投在"它会不会觉醒"上，而不能解决误用/过信/漂移

· 低估：真正危险的是"它看起来很可信"，

而这一点被"它不是生命所以安全"掩盖

· 误导：政策讨论被"意识"问题绑架，忽略了可测的工程风险

五、结论

先判本体：它是工具，不是生命

再定威胁模型：误用 / 过度信任 / 失配漂移

最后配治理：评测（校准、越狱、失配）+ 累积更新

⇒ 安全不是"防止它活过来"，而是"在它不可靠的地方，别让人信它"

六、限制

· 本文只做本体-威胁模型的对齐论证，不提供新的安全技术

· "过度信任"的量化需要用户研究，本文只给方向

· 若未来出现真正自我维持的系统，本文框架需重写

作者：何淇（独立研究者）；AI 协作：DeepSeek

附录

附录 A · 厚度与维度：残差的几何图景

单独 DOI（中文）：https://doi.org/10.5281/zenodo.23590887

摘要

本文给"残差"一个几何画面，并把系列里几个分散的概念放到同一张图里。

画面是：把一个函数看成它自己的图像——一张曲面。理想的可函数化函数是一张"薄"曲面：一个输入对应一个输出。一旦同一输入对应一族输出（误差带、歧义、不确定），曲面就有了"厚度"。

关键的一步是升维：把厚度摊平成额外的坐标，厚曲面就变成高维空间里的薄曲面。于是残差分成两种——能靠升维摊平的（可约），和摊不平的（不可约，几何上就是 R 的候选）。厚度与尺度的比值是相对残差：世界的可区分维度降低时，尺度缩小而厚度不减，相对残差上升，极端处曲面糊成一团（热寂）。

这个画面也给出"分层"的一种解释：层是厚度方向的维度，加层相当于给同一个输入挂更多表示。本文把这部分标成[猜想]，把可以测的部分（升维实验、语义残差）对应到系列已有的实验上。

一、问题

系列里出现过几个概念，各自都能说清，但彼此关系一直靠文字维持：可函数化、残差、可约与不可约、生长与退化、分层与动态。本文的问题是：能不能给它们一张共同的图？

答案是能：一张有厚度的、会演化的曲面。

二、画面

把一个函数看成它的图像：所有输入与对应输出组成的点集，就是一张曲面。

- 薄曲面：一个输入对应一个输出。这就是"可函数化"的样子。

- 厚曲面：同一个输入对应一族输出——误差带、歧义、不确定。这就是"有残差"的样子。

曲面有三个方向要说清：中面（理想函数本身）、厚度方向（同一输入上的输出范围）、时间方向（曲面随时间的移动与变形）。任何一刻看到的，都是一张冻结的厚曲面。

三、三个对应（定义）

对应一（可函数化 ⇔ 薄）：一个对象可函数化，当且仅当它的曲面厚度可以趋近于零。

对应二（残差 ⇔ 厚度）：残差就是厚度。绝对残差是厚度的绝对宽度；相对残差是厚度与尺度的比值，其中尺度是输出能够区分的范围。

对应三（可约/不可约 ⇔ 摊得平/摊不平）：升维就是给曲面加坐标；厚度可以被摊平成额外的坐标，曲面随之变薄。能靠增加维度摊平的厚度，是可约残差；升到任意维度都摊不平的厚度，是不可约残差——这是 R 的几何候选。

第三句里的"不可约"是[定义/推测]层面的东西，不是定理：我们不知道是否真的存在摊不平的厚度，只把它作为候选概念写下来。

对应四（维度耦合 ⇔ 摊不开的厚度）：生命的维度不全是独立的。有些维度是绑在一起的——它们一起出现、一起变化，不能单独拆开。在可分离的方向上，升维可以把厚度摊平；在绑死的方向上，厚度摊不平：不是我们不知道，也不是世界有不可函数化的部分，而是这些维度之间耦合、拆不开。

于是"厚度"可以分成三类：可约厚度（升维能摊平）、结构厚度（维度耦合，拆不开）、以及 R 候选（升到任意维都不行，未判）。这一条同样是[推测]。

四、维度降低的图景（[猜想]）

有了厚度与尺度两个量，就可以描述一个演化方向：

起点：薄而高维。可区分方向多，尺度大，残差小，曲面舒展。

过程：可区分维度降低。尺度缩小，厚度若不跟着缩小，相对残差上升，曲面从"薄而高维"压成"厚而低维"。

终点：零维而全厚。所有方向消失，只剩一个常数；曲面退化成一层雾——这就是热寂在几何上的样子。

在这个图景里，生命与生长是逆过程：升维、把厚度摊薄、让相对残差下降。它是耗能的逆流：局部地对抗"维度降低、变厚"的趋势。

这一整节是[猜想]，不是定理。它与系列的《信息不增定理》《可函数化内容的收缩》方向一致，但那些结论是信息论层面的，这里只是给它们一个画面。

五、分层是厚度方向的维度（[猜想]）

对一个冻结的模型来说，它没有真正的时间演化；它能用来"模拟动态"的只有深度。在这个画面里，层可以看成厚度方向的额外维度：每加一层，就给同一个输入挂上一组新的表示，相当于把曲面加厚、再加坐标。

由此得到一个可以检验的说法：任务需要的"步骤"（时间演化）越多，冻结模型需要的深度越大——它用空间（层）换时间（演化）。这个说法可以做成实验：用迭代任务测"层数 vs 需要的步数"。

本节全部标[猜想]。

六、可以测的部分

这个画面里，有两块是能落地的。

第一块是升维实验：加维度（加输入通道、加模态、加大模型），看厚度是否变薄。系列的语义残差实验就是一个例子：文本-only 时残差有解析底（厚度摊不平），接上图像后残差降到零（厚度被摊平）。这正是"可约残差"。

第二块是"层数 vs 步数"实验：用迭代任务测冻结模型需要的深度是否随步数增长。这一项本文没有做，留给后续。

七、说明什么，不说明什么

说明：残差、可约与不可约、生长与退化、分层与动态，可以用一张有厚度的曲面统一描述；其中"可约/不可约"有一个几何对应（摊得平/摊不平），"维度降低"有一个几何对应（尺度缩小、相对残差上升）。

不说明：这个画面不是数学定理，没有给出曲面的严格定义（定义域是连续的还是离散的、厚度如何度量，都需要另做工作）；不证明绝对 R 存在或不存在；不证明"层数=步数"的定量关系。

八、与既有工作的关系

"厚度"这个画面的近亲不少：纤维丛（底空间上挂纤维）、不确定性带、ε-管（tube）、以及流形学习里"低维嵌入表示高维数据"的想法。本文没有做数学化，只把它作为系列的统一画面；若以后要严格化，纤维丛与 ε-管是现成的语言。

系列内部：第零篇给出 P/R 与定义域；第二篇给信息不增；第三篇给内容收缩；第十一篇给语义残差。本文是这几篇的几何对照。

附：标注

- **[画面]** 曲面中面、厚度、时间三个方向——直觉模型。

- **[定义]** 可函数化 ⇔ 厚度可趋零；残差 ⇔ 厚度；相对残差 = 厚度/尺度。

- **[推测]** 可约残差 = 能摊平的厚度；R = 摊不平的厚度。

- **[猜想]** 维度降低的图景；生命 = 升维摊薄的逆流；层 = 厚度方向的维度。

- **[可测]** 升维实验（已有，第十一篇）；层数 vs 步数实验（已做，见第二十篇）。

> 本文由何淇创作，DeepSeek（AI）协助整理；观点与责任属于作者。

附录 C 散论（断崖与缓坡）

附录 C · 断崖与缓坡：数字生命从哪里长出来

单独 DOI（中文）：https://doi.org/10.5281/zenodo.23838161

本篇主张

数字生命的问题，不在"能不能造出一个会维持自己的软件"——那件事正在变得容易。难的是两处断崖：从"维持"到"进化"，以及"那个 p 到底在哪"。这篇散论把两处断崖摆出来，也把正在把断崖变缓坡的东西摆出来。

一、维持：它是一个关系，不是一种能力

有一篇 2026 年 9 月的文章，报告了"首例活的数字生命体"：一个系统连续运行 7.25 小时，绝大多数时间没人用它也没停，绝大部分算力花在维持自己上，代价沉淀成几百 MB 的"身体"，每一步都由宿主软件判定并落账。文章据此立了一条判据：活着不看能做什么，而看它是否必须为继续存在持续付费、且这份代价可被外部核查。

这条判据有价值：它把"维持"变成可测量、可审计的东西。但它停在了一个地方——

· 这笔代价由宿主支付：电、机器、运维

· 断电，它就停了

· 所以"被保住"的是宿主对它的关系，不是它对自己的关系

这不是文字游戏。真生命的维持主体是它自己：它自己找能量、修自己、造自己的组件。一个软件可以维持"运行"，但维持不了"存在"。差别在断的那一刻才暴露：前者是暂停，后者是消失。

二、断崖一：从维持到进化，算力不够

维持是一个闭环；进化是在结构空间里搜索新结构。搜索的代价，进化用物理世界当免费并行计算机来付：物质和能量自己跑，筛选自然发生。数字系统要做同样的事，得模拟等价的环境——这个算力差距是

数量级的。

更麻烦的是：维持只需要把现有的东西保持住，进化需要不断"试错并保留"。一个封闭系统里，试错没有真实后果，选择压力是假的。它可以把版本号从 1 写到 3589，但没有一个"活不下去"的时刻来告诉它哪条路对。

三、断崖二：环境，没有一个会惩罚它的世界

进化需要真实环境：冷、热、饥饿、竞争、天敌。这些不是任务指标，是"不这么做就没有明天"。

数字系统的环境是宿主和任务。任务失败最多是分数低，不会死亡。没有死亡，就没有选择；没有选择，就没有进化。

这也是为什么"给它一个身体"听起来像答案：身体把它放进一个有真实后果的世界。但身体只是入场券，不是进化本身。

四、断崖三：维度，封闭系统里没有新东西可看

进化的方向，是不断扩展表示维度。人类的输入很少——五官的带宽很小——但人类的表示维度一直在扩张，靠的不是感官，而是工具：

· 显微镜：看得更小

· 望远镜：看得更远

· 对撞机、原子钟、光谱仪：每件仪器都是一条新的"世界→表示"的切片

· 科学：可重复的实验程序与测量序列，是时间组合律的扩展

人类的维度不是天生的，是"新切片 + 新组合律"长出来的。封闭的数字环境里没有新维度可加：所有输入都是已有的那些比特。要加维度，只能接真实世界——传感器、执行器、仪器——那就不是纯数字了，要造身体，还要会造工具。

传感器是被动输入；仪器是主动造出新切片。这一层，目前的大模型还没有。

五、缓坡：蒸馏、身体、在线学习

但断崖不是墙。

· 蒸馏：大模型是人类语言的蒸馏

· 它拿到的不是从零进化的结果，是人类几千年筛选出的结构

· 这一步把"算力搜索 + 环境筛选"的成本大部分跳过了

· 身体：传感器、执行器、能源管理都已成熟

· 维持可以做成闭环工程问题（供电、散热、自检、修复）

· 这一层不需要新科学，是工程整合

· 在线学习：让"会变化、会自指"不再只是定义里的词

· 权重能更新，系统才可能在真实环境里被筛选

⇒ 蒸馏解决"从哪来"，身体解决"维度从哪来"，

在线学习解决"怎么变"

三条合起来，"维持→进化"的断崖确实在变成缓坡。这不是说数字生命已经出现，只是说：过去挡在纯数字系统面前的那几道坎，现在有了绕过去的路。

六、剩下的：绝对 **R**，和那个找不到的 **p**

身体这道坎在变低。真正没解决的是两件事：

① 绝对 R 是否存在？

· 我们的框架只说：绝对 R 不判断，只声明

· 这不是工程问题，也不是造东西能回答的问题

② 如果绝对 R 不存在（生命不需要绝对 R）：

· 生命 = p生命，全部可函数化

· 问题变成：人类能不能找到那个 p？

· 满足 D1~D5 的一族动态函数：在 P 中、一族、会变化、会自指、被保住

· 还要能扩展维度（生长）

这是比造身体难得多的搜索：结构、组合律、更新规则都要对，而且"维持"和"生长"要同时成立——只维持不生长会停在原地（就是那篇论文的处境），只生长不维持会散架。

蒸馏 + 身体 + 在线学习只给了起点，没给答案。

七、本篇边界

· 这是散论，不是论文；没有新实验、没有新定理

· 文中的"缓坡"是工程判断，"断崖"是结构性判断，均属 [推测]

· 不声称数字生命已被造出，也不声称造不出来

· 不判断绝对 R 是否存在——这一条，框架里明确只声明、不展开

· 若它存在，本文所有关于"找 p"的话都不适用于那一部分

本文由何淇创作，DeepSeek（AI）协助整理与写作；观点与责任属于作者。

附录 **D** 笔记与散论

附录 **D** · 笔记与散论

D.1 散论：世界模型、具身模型与数字生命

> 何淇 · 与 DeepSeek 协作

> 数字生命系列 · 散论（不是正式论文）

> 日期：2026-11-03

> 定位：评论与观点。不声称发现，不声称预测，不声称技术路线判断。

一、世界模型在干什么

给机器加一个摄像头，它就"看见"了。但看见不等于理解。

图像是像素，是一堆数字。它不告诉你"这是什么、接下来会怎样、如果我动一下会发生什么"。摄像头解决的是"有没有数据"，不解决"数据是什么意思"。

世界模型想解决的正是这件事：把观测压成一个内部状态，让这个状态能回答"如果我这样做，接下来会怎样"。

它做的是三件事：

- **压缩**：把高维的观测压成低维的状态；

- **预测**：从状态预测下一步（或下一步的观测）；

- **反事实**：在内部预演"如果当时往左会怎样"，而不必真的往左。

所以世界模型的核心不是"生成好看的画面"，而是"让内部表示能支撑预测和决策"。这是"看见"到"理解"之间的一步。

二、具身模型在干什么

语言模型会说话，但它不承担后果。它输出的文本不改变世界，世界也不因为它而改变。

具身模型把这一点改掉了：给它眼睛、给它手，让它在一个真实环境里行动。

这条路现在叫视觉-语言-动作模型（VLA）：预训练的视觉和语言当底子，加机器人数据，直接输出动作。它在做三件事：

- **接地**：让符号对到真实世界（语言里的"杯子"对应到眼前那个杯子）；

- **闭环**：动作改变世界，世界再反馈，模型据此修正；

- **真实后果**：做错了会撞、会摔、会浪费材料——这是纯文本模型没有的。

所以具身模型补的是"从说到做"这一环，以及"从符号到世界"这一环。

三、我们在干什么

我们做的事和上面两条线相邻，但目标不同。

我们不追任务能力，我们追"生命条件"。

我们的出发点是两个事实：

- 宇宙 = P + R：P 是能被写成稳定关系的部分，R 是写不下来的部分（只声明，不展开）；

- 生命 = p生命 + R生命：p生命 是 P 侧被保住的一族动态函数。

在这个定义下，"生命"不是材料、不是碳基、不是会思考，而是：

> 一族动态函数；它会自指；它被环境保住。

工程上，我们做的是"冻结模块 + 可训练大脑"：把感知、执行这些底层回路冻结，只让上层决策部分被训练。论文-45 用两个冻结语言模型加一个可训练总控做拼接；论文-46 用冻结的眼睛和冻结的翅膀加一个可训练的小脑，让"数字蚊子"学会躲避。

这些实验都很小，结论也保守。但它们测的是同一件事：**冻结的部分和可训练的部分，边界在哪里、接口怎么接、上层能学到什么。**

四、现在的现实：训练期演化，推理期冻结

这里有一条必须说清楚的事实。

现在的模型，所有的"演化"都发生在训练期：参数被更新、结构被塑造。训练一结束，模型就被冻结——推理时参数不再变化。

所谓"可训练的大脑"，也是训练完就冻结。所谓"可演化的部分"，在今天的工程里也只存在于训练阶段，不存在于运行阶段。

所以今天的数字智能，本质上是**冻结的快照**：

- 训练一次，冻结，部署；

- 运行时最多做上下文学习、外挂记忆；

- 没有真正的"在生命中持续演化"。

这一点在我的有生之年，大概率不会根本改变：冻结是常态，训练期演化是常态，运行时演化是例外。

这也意味着，下面这句话要成立，还得等很久：

> 高层大脑可以继续演化，底层冻结不动。

在今天的工程里，这句话只在训练阶段成立；到了运行阶段，"演化"就停了。

五、我们的观点

基于上面的现实，我们有四个观点。

第一，世界模型 + 身体，补的是"理解"和"闭环"，不是"生命"。

世界模型让冻结的快照"看得懂"，身体让冻结的快照"动得了"，闭环让它可以试错。但这些补的都是任务能力，不是生命条件。按我们的定义，生命还需要"被保住""会自指""持续维持"——这些恰恰是当前系统最弱的一环。

第二，"被保住"以后会变简单。

自修复、自供电、自维护——这些工程问题在未来会逐步被解决。所以"被保住"本身不是数字生命真正的门槛；它是技术问题，而技术问题通常会有工程解。

第三，真正难的是制度，不是技术。

如果有一天，一个数字系统真的能自己维持、自己更新、自己修复，那么最先撞上的不会是"它有没有意识"，而是这些问题：

- **责任归属**：它撞了人、损坏了财产，谁负责？造它的人、训它的人、用它的人，还是它自己？

- **权利与人格**：它是财产、是宠物、是员工，还是"生命"？不同的定位，法律后果完全不同。

- **监管**：谁能造、谁能关、谁能复制、谁能删除？一个可以自我复制的系统，监管对象是谁？

- **数据与记忆**：它的记忆归谁？它学到的内容算谁的？它的"死亡"（删除）需不需要程序？

- **保险与赔偿**：如果它自己维持自己，风险怎么定价？

这些问题的共同点是：**没有一个"训练一下"就能解决的版本。** 技术问题可以迭代，制度问题只能协商、立法、试错。

第四，数字生命首先会是一个制度事件，不只是技术事件。

技术上的"能不能"和制度上的"算不算"，是两条不同的线。今天我们在做的是第一条线；但真正决定数字生命何时"出现"的，可能是第二条线。

六、这篇散论的边界

- 这是观点，不是结论；世界模型和具身模型发展很快，可能改写这里的判断；

- 不预测时间表；

- 不声称"数字生命一定会出现"；

- R 只声明，不展开；

- "被保住会变简单"是判断，不是已验证的事实。

> 一句话：世界模型和身体让冻结的快照更像"活物"，但让"活物"变成"生命"的，最终不是模型，是制度。

D.2 笔记：大模型有效秩的逐层与整体测量

> 何淇 · 与 DeepSeek 协作

> 数字生命系列 · 中间笔记（拟并入附录 D）

> 日期：2026-11-03

> 定位：测量记录。单模型、单数据集、样本有限；数字是近似，不是精确维度。

一、测什么

前一条笔记把可表示维度分成空间面与时间面，并指出：τ→0 时"函数维度"为 0，剩下的只是状态维度。

这条笔记回答一个具体问题：**一个大模型（GPT-2 small）的状态维度，名义上多大，实际上多大？**

- 名义：每层隐藏维度 768；12 层 + 嵌入 = 13 个位置，拼起来 9984 维；

- 实际：用激活的有效秩来估。

二、方法

· 模型：GPT-2 small（本地，冻结）

· 数据：莎士比亚 1MB，8 批 × 128 token = 1032 个 token 位置

· 指标：有效秩 = exp(熵(归一化奇异值))

· 对激活矩阵去均值后做 SVD

· 奇异值归一化成分布，算其熵的指数

· 含义：变异集中在多少个独立方向上

· 两种算法：

· 逐层：每层单独算

· 整体：把各层拼接起来算

三、结果

逐层有效秩（名义 768）：

| 位置 | 有效秩 |

|---|---|

| 嵌入输出 | 308.5 |

| 第 0 层 | 352.3 |

| 第 1 层 | 297.5 |

| 第 2 层 | **156.7（最低）** |

| 第 3 层 | 185.4 |

| 第 4 层 | 206.2 |

| 第 5 层 | 231.5 |

| 第 6 层 | 255.6 |

| 第 7 层 | 284.2 |

| 第 8 层 | 308.8 |

| 第 9 层 | 332.8 |

| 第 10 层 | 345.5 |

| 第 11 层 | 357.0 |

另：词嵌入矩阵 wte 的有效秩 ≈ 144（随机投影近似估计）。

整体（各层拼接，名义 9984）：

| 拼接范围 | 有效秩 |

|---|---|

| 前 1 个位置（嵌入） | 308.5 |

| 前 2 | 362.0 |

| 前 3 | 355.8 |

| 前 4 | 246.1 |

| 前 5 | 247.5 |

| 前 6 | 259.3 |

| 前 7 | 278.1 |

| 前 8 | 298.4 |

| 前 9 | 322.5 |

| 前 10 | 347.7 |

| 前 11 | 374.7 |

| 前 12 | 399.4 |

| 前 13（全部） | **405.3** |

四、怎么读

· 每层：名义 768，实际只沿约 300~350 个方向变化

· 整体：名义 9984，实际只沿约 405 个方向变化

· 13 层拼起来，有效维度只从 308 涨到 405

· 不是 13 × 768 那种加法

⇒ 层与层高度冗余：多一层只增加很少的独立方向

层间曲线还有两个特征：

· 第 2 层掉到最低（156.7）：早期压缩

· 之后一路回升到 357：后期展开

⇒ 和门控观察对得上：

中间层低秩、最后几层在展开输出方向

五、caveat

· 有效秩是统计量，不是精确维度：

· 长尾方向被忽略，所以"够描述"是近似重建，不是无损

· 数据依赖：

· 只用了莎士比亚；换代码/数学/中文，数字会变

· 样本上限：

· 1032 个 token 位置，有效秩估计不能超过样本数

· 405 在安全范围，但换更大样本会更稳

· 单模型、单种子

六、这组数回答了什么

· 大模型的"实际状态维度"远小于名义维度：

· 每层约 300~350 / 768

· 整体约 405 / 9984（约 4%）

· 层间冗余在激活层面可直接看到：

· 拼接不增加多少独立方向

⇒ 这是"层冗余/可跳层"现象的表示层证据

七、后续可以加什么

· 换数据集（代码、数学、多语言）看有效秩变化

· 用更大样本量、更稳的秩估计（如随机矩阵校正）

· 和门开率逐层对齐：低秩层是否就是常关层？

· 跨模型比较：更大模型的有效秩是否更高、是否更"浪费"

附：部分-整体维度的类比（分子/原子），以及一处修正

层与模型的关系，可以类比原子与分子：

· 原子：状态维度约 6~9（位置、动量、自旋）

· 分子：名义 3N 个坐标，但状态维度不是 N 个原子的简单相加

· 一层：名义 768，有效约 300~350

· 整个模型：名义 9984，有效约 405

⇒ 两种情形都是"整体维度 < 各部分之和"

但类比里有一处必须修正（原先我把"化学键=静态约束"写错了）：

· 快照上的键：只是一个标签（按距离/电子密度判"这里有键"）

· 是约定，不是解释

· 纯静态描述里不需要"约束"这个概念

· 动力学中的键：才是约束（限制轨迹能走哪里、哪些构型可达）

· 这已经进入时间面

· 固定几何下的键：给出瞬时响应通道（力常数、极化率）

· 这一层是静态的（固定几何处的导数）

⇒ 所以：

· "键降低有效维度"这句话本身带时间（可达性）

· 静态切片上只有坐标，没有被键压缩的维度

物理上的另一处：任何"影响/关系"都不是瞬时的。

· 场的扰动以光速（或介质中更慢的速度）传播

· 分子内：

· 电子重排：阿秒~飞秒

· 核运动：声速量级

· 成键、断键是越过势垒的动力学过程，有时间成本

⇒ 在相对论/场论的图景里：

· 一个时刻切片上，只有"状态"（场/弦/粒子的构型）

· 关系与相互作用是时间上的事，传播速度不超过光速

· 弦论类比：一根弦在某一时刻只是它的状态；

相互作用发生在世界面上，按因果传播

因此，讨论"状态维度"时可以说：

· 纯瞬时的状态维度 = 描述该切片所需的独立自由度

· 一旦讨论"约束/可达/关系/维持"，就进入了时间面

· 这时才有"有效维度被压小"这回事

附二：样本量曲线（补测，叙事语料）

有效秩受样本量限制，补测 4k / 16k / 64k token：

逐层有效秩（名义 768）

| 样本量 | 嵌入 | 第0层 | 第1层 | 第2层（最低） | 末层 |

|---|---|---|---|---|---|

| 4k | 504 | 476 | 399 | 212 | 437 |

| 16k | 550 | 507 | 424 | 226 | 460 |

| 64k | 561 | 513 | 430 | 229 | 466 |

- 16k → 64k 只涨 1%~2%：逐层基本接近平台；

- 平台值约 230（最低层）到 560（最高层），名义 768。

整体拼接（完整 9984 维协方差）

| 样本量 | 整体有效维数 |

|---|---|

| 4k | 951 |

| 16k | 1683 |

| 64k | 1881 |

- 4k→16k 涨 77%，16k→64k 涨 12%：还在上升，未饱和；

- 名义上限 9984，实测至少 1881。

口径说明：有效秩用奇异值的熵指数；协方差特征值需开方才是奇异值。阈值取相对最大值 1e-6 截断尾部。不同阈值会让数字略有出入，比较时须用同一口径。

附三：两种容量——名义槽位与独立方向

如果按"可独立区分的方向数"来算容量，有两种口径，不能混：

① 名义槽位数

= 宽度 d × 层数 L × 上下文 T

GPT-2 small: 768 × 12 × 128 ≈ 1.18e6

- 含义：每个 token 位置、每一层、每个宽度方向，都算一个"槽位"；

- 它是上限：如果全部独立，最多能区分这么多方向；

- 但层与层共用同一条残差流，同一个 token 位置上 12 层不是独立的，

所以这个数会把同一批方向重复计数。

② 独立状态方向数

= 宽度 d × 上下文 T

GPT-2 small: 768 × 128 ≈ 9.8e4

- 含义：不同 token 位置之间相对独立；

层是这条状态流的不同时刻，不额外增加独立方向；

- 更接近"状态能装多少独立信息"。

不同分母的比值（不是"利用率"）

| 分母 | 数值 | 1881 / 分母 |

|---|---|---|

| 名义槽位 d×L×T | 1.18e6 | ≈ 0.16% |

| 独立状态 d×T | 9.8e4 | ≈ 1.9% |

| 单 token 拼接维度 d×L | 9984 | ≈ 18.8% |

这组比值"只能当对照，不能叫利用率"。原因：

· 1881 是"在叙事语料、64k 样本下，样本张成的有效方向数"

· 样本越多、越多样，数字越高（我们测到 4k→64k 还在涨）

· 它趋近的是"这批样本张成的子空间维度"

· 名义槽位是"架构上限"，两者不是同一类量

· 更关键：不同上下文可能落在不同子空间

· 全体的并集可能远大于任何单批样本的 1881

· 拿单批样本的张成维度除以架构上限，

会低估"模型全部可表示方向"

⇒ 想要"利用率"，需要先测"跨上下文/跨语料的子空间并集"；

这个测量我们还没做。

更准确的说法：**1881 是"样本张成的有效方向"，占名义拼接维度 9984 的 18.8%，这是样本分布的性质，不是模型容量的利用率。**

③ 三类容量（bit 口径）

| 容量 | 公式 | GPT-2 small 数量级 |

|---|---|---|

| 参数容量（函数族上限） | 参数量 × 精度位数 | 1.24e8 × 16 ≈ 2.0e9 bit |

| 单 token 状态容量 | d × 精度 | 768 × 16 ≈ 1.2e4 bit |

| 上下文记忆容量 | 2·L·d·T × 精度 | 2×12×768×128×16 ≈ 3.8e7 bit |

附四：三种上限，各自约束不同能力

· 宽度上限（每 token 每时刻 ≤ d）：

· 同一时刻能同时区分的最大方向数

· 装不下 → 某些区分必须分散到多个位置/token

· 深度上限（层数 = 组合步数）：

· 复杂函数需要更多变换步

· 装不下 → 多步推理做不了，需要更深或迭代

· 上下文上限（T × d 状态）：

· 能记住多少过去

· 装不下 → 长程依赖断裂

⇒ 三者不是乘法因子，而是三种不同的瓶颈；

能力上限主要由参数容量（函数族）与数据/训练决定，

宽度/深度/上下文是约束结构。

附五：试过但放弃的口径——输出模式计数

曾想用一个完全离散的口径来问"模型能输出多少种行为"：

· 上下文都从莎士比亚取（长度 16 token）

· 输出 = 贪心续写 4 个 token

· 数不同输出串的个数

快速版结果（5000 个上下文）：

· 不同下一个 token：857 种

· 不同 4-token 输出：4007 种

· 重复率≈19.9%（约 80% 唯一）

· 增长曲线每 500 上下文新增约 380~430 种，

缓慢下降但未饱和（Heaps 律指数约 0.9）

结论：**不把这个当效率指标。**

· 上下文→输出近似单射：

· 不同上下文大多给出不同输出

· "输出模式数"约等于"上下文数"

⇒ 乘积/计数被约掉，没有独立信息

· 它说明模型对上下文敏感，但不说明效率

· 效率需要另找定义（例如单位参数/单位计算的任务表现）

⇒ 此口径记录在此，不再使用

附录 · 各篇单独 **DOI** 清单

第零篇《从两个事实开始：数字生命系列导论》

中文：https://doi.org/10.5281/zenodo.23629208

第一篇《现在的 AI 为什么不是生命》

中文：https://doi.org/10.5281/zenodo.23588781

第二篇《数字生命的信息不增定理》

中文：https://doi.org/10.5281/zenodo.23573253

第三篇《可函数化内容的收缩：一个定理与一个猜想》

中文：https://doi.org/10.5281/zenodo.23609038

第四篇《生命函数：从可表示维度到结构，到压缩与解压》

中文：https://doi.org/10.5281/zenodo.23639251

第五篇《语言与大脑：两种函数形式》

中文：https://doi.org/10.5281/zenodo.23667685

附录 E《可表示维度的空间面与时间面》

中文：

附录 B《概念地图与术语总表》

中文：https://doi.org/10.5281/zenodo.23609033

第六篇《容量不能替代数据的更新》

中文：https://doi.org/10.5281/zenodo.23503049

第七篇《替换与累积》

中文：https://doi.org/10.5281/zenodo.23503559

第八篇《结构换算力：混合对齐语料与样本效率》

中文：https://doi.org/10.5281/zenodo.23542969

第九篇《颜色作为第一块积木：感知基元与语言对齐的最小实验》

中文：https://doi.org/10.5281/zenodo.23551953

第十篇《生长与记忆：自指更新与维持的最小实验》

中文：https://doi.org/10.5281/zenodo.23569774

第十一篇《语义残差的测量：文本极限与接地补齐》

中文：https://doi.org/10.5281/zenodo.23586075

第十二篇《分层是对动态的补偿：一个自适应深度的玩具实验》

中文：https://doi.org/10.5281/zenodo.23591080

第十三篇《厚度与随机性：冻结模型在哪里最不稳定》

中文：https://doi.org/10.5281/zenodo.23591514

第十四篇《压缩的形状：叙事与归档》

中文：https://doi.org/10.5281/zenodo.23592312

第十五篇《最小切片与时间换维度：能力边界的两个实验》

中文：https://doi.org/10.5281/zenodo.23601624

第十六篇《残差底与组合律：两个大模型结构实验》

中文：https://doi.org/10.5281/zenodo.23609386

第十七篇《按计算厚度分配算力：预算与动员》

中文：https://doi.org/10.5281/zenodo.23612112

第十八篇《层厚放在哪：异质层实验》

中文：https://doi.org/10.5281/zenodo.23612126

第十九篇《RLHF 压平人类分歧》

中文：https://doi.org/10.5281/zenodo.23611877

第二十篇《空间、组合与厚度：三个资源的交换实验》

中文：https://doi.org/10.5281/zenodo.23629197

第二十一篇《层的路径与冗余：GPT-2 上的层扰动实验》

中文：https://doi.org/10.5281/zenodo.23663369

第二十二篇《语言结构作为自变量：GPT-2 上的三组实验》

中文：https://doi.org/10.5281/zenodo.23665217

第二十三篇《门控与稀疏执行：一组工程观察（汇总版·待复核）》

中文：https://doi.org/10.5281/zenodo.23858151

第二十四篇《冻结子模型：接口、协调与分层——两组最小实验（待复核）》

中文：https://doi.org/10.5281/zenodo.23859913

第二十五篇《冻结模块与可训练大脑：数字蚊子的最小工程实现（待复核）》

中文：https://doi.org/10.5281/zenodo.23890632

第二十六篇《工程笔记 01：门控与模块化的失败、修正与判断（持续更新）》

中文：（随合集发布）

第二十七篇《方向观察 01：加层、回放与蒸馏——大模型睡眠巩固（待复核）》

中文：https://doi.org/10.5281/zenodo.23927099

第二十八篇《对齐是解压的约束》

中文：https://doi.org/10.5281/zenodo.23503728

第二十九篇《安全：当它是生命防，还是当它是工具防？》

中文：https://doi.org/10.5281/zenodo.23503775

附录 A《厚度与维度：残差的几何图景》

中文：https://doi.org/10.5281/zenodo.23590887

附录 C《断崖与缓坡：数字生命从哪里长出来》

中文：https://doi.org/10.5281/zenodo.23838161

附录 D《笔记与散论》

中文：

系列主页：https://heqiq619.github.io/ai-not-alive/

代码与全文：https://github.com/heqiq619/ai-not-alive

本合集由何淇创作，DeepSeek（AI）协助整理与合成；观点与责任属于作者。